République Tunisienne

Ministère de l'Enseignement Supérieur

et de la Recherche Scientifique

Université de Jendouba

Institut Supérieur d'Informatique du Kef

Rapport de Projet de Fin d'Études

Présenté en vue de l'obtention du

Mastère Professionnel

Spécialité : Applications Web Intelligentes

Par

Iheb El Ech

Conception et développement d'une plateforme intelligente d'aide au recrutement et de mise en correspondance entre CV et offres d'emploi

Encadrante professionnelle : **Dr. Trabelsi Emna**

Encadrant académique : **Dr. Louati Aymen**

Réalisé au sein de CyberCloud Solutions

Soutenu le : .....

devant le jury composé de :

Président : .....

Rapporteur : .....

Année Universitaire 2025–2026

République Tunisienne

Ministère de l'Enseignement Supérieur

et de la Recherche Scientifique

Université de Jendouba

Institut Supérieur d'Informatique du Kef

Rapport de Projet de Fin d'Études

Présenté en vue de l'obtention du

Mastère Professionnel

Spécialité : Applications Web Intelligentes

Par

Iheb El Ech

Conception et développement d'une plateforme intelligente d'aide au recrutement et de mise en correspondance entre CV et offres d'emploi

Encadrante professionnelle : **Dr. Trabelsi Emna**

Encadrant académique : **Dr. Louati Aymen**

Réalisé au sein de CyberCloud Solutions

Année Universitaire 2025–2026

J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrante professionnelle, **Dr. Trabelsi Emna**

Signature et cachet

J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant académique, **Dr. Louati Aymen**

Signature

Dédicaces

C'est avec une profonde gratitude et des mots sincères que je dédie ce modeste travail.

À mes chers parents, Abedlwaheb El Ech et Habiba Omri. *Votre soutien indéfectible et votre amour inconditionnel ont été une source inestimable de force et d'inspiration. Que ce travail soit le témoignage de ma profonde reconnaissance et de ma gratitude infinie. Que Dieu vous bénisse, vous accorde santé et longue vie.*

À ma petite sœur, Hayfa El Ech. *Merci pour ton soutien, ta présence et tes encouragements tout au long de ce parcours.*

À ma famille. *Je vous remercie pour votre affection, votre confiance et votre soutien constant.*

À mes amis. *Pour tous les instants inoubliables que nous avons passés ensemble, je vous remercie du fond du cœur. Que ce travail soit également le témoignage de mon amitié et de ma reconnaissance pour votre soutien.*

El Ech Iheb

Remerciements

Je remercie Dieu le Tout-Puissant de m'avoir donné la santé et la volonté d'entamer et de terminer ce rapport.

J'exprime également ma sincère gratitude à mon encadrant à l'ISI Kef, **Dr. Aymen Louati**, pour son aide et ses conseils concernant l'acheminement du projet et l'architecture du rapport. Je le remercie pour le temps qu'il m'a consacré, pour les informations qu'il m'a fournies avec compréhension, ainsi que pour sa patience et son amabilité.

J'adresse également mes remerciements à **Dr. Emna Trabelsi** pour sa pédagogie, sa confiance et le soutien qu'elle m'a accordé tout au long de ce projet au sein de CyberCloud Solutions.

Je tiens de même à remercier tous les enseignants de l'ISI Kef qui m'ont enseigné pendant ces trois années successives et qui, par leurs compétences, m'ont soutenu dans la poursuite de mes études.

Enfin, je présente mes profonds remerciements et mon respect aux membres du jury qui ont accepté de juger mon travail.

Table des matières

Dédicaces **iv**

Remerciements **v**

Liste des abréviations **x**

Introduction générale **1**

1 Contexte général et étude préalable **2**

1.1 Présentation de l'organisme d'accueil . . . 3

1.1.1 Présentation de CyberCloud Solutions . . . 3

1.1.2 Activités et domaines d'expertise . . . 4

1.1.3 Cadre du stage . . . 4

1.2 Contexte du projet . . . 4

1.3 Étude de l'existant . . . 5

1.3.1 Processus manuel de présélection . . . 5

1.3.2 Approches lexicales et ATS traditionnels . . . 6

1.3.3 Solutions numériques étudiées . . . 6

1.3.4 Synthèse comparative . . . 7

1.4 Critique de l'existant . . . 7

1.5 Problématique . . . 8

1.6 Solution proposée : JobTest . . . 8

1.7 Objectifs du projet . . . 9

1.7.1 Objectif général . . . 9

1.7.2 Objectifs spécifiques . . . 9

1.8 Méthodologie de conduite du projet . . . 10

1.8.1 Justification du modèle en cascade . . . 10

1.8.2 Phases appliquées à JobTest . . . 10

2 Analyse et spécification des besoins **12**

2.1 Périmètre fonctionnel de JobTest . . . 13

2.2 Identification des acteurs et parties prenantes 13
2.2.1 Visiteur non authentifié 13
2.2.2 Recruteur authentifié 14
2.2.3 Candidat en tant qu'entité métier 14
2.2.4 Partie prenante technique 14
2.3 Spécification des besoins fonctionnels 15
2.3.1 Authentification du recruteur 17
2.3.2 Gestion des offres et des candidats 18
2.3.3 Évaluation et restitution 18
2.3.4 Historique, classement et réglages 19
2.4 Spécification des besoins non fonctionnels 19
2.5 Règles de gestion 22
2.6 Diagramme global des cas d'utilisation 22
2.7 Raffinement des cas d'utilisation principaux 23
2.8 Descriptions textuelles des cas d'utilisation 29
2.9 Matrice de traçabilité 34
2.10 Contraintes et hypothèses 35

3 Conception fonctionnelle et architecturale 38

Introduction 39
3.1 Démarche de conception 39
3.2 Vue fonctionnelle globale 39
3.3 Architecture logique de JobTest 39
3.4 Architecture applicative et responsabilités 41
3.5 Conception statique – diagramme de classes global 41
3.6 Conception dynamique – diagrammes de séquence 42
3.6.1 Authentification du recruteur 42
3.6.2 Création d'une offre d'emploi 43
3.6.3 Évaluation d'un CV par rapport à une offre 43
3.6.4 Analyse approfondie avec Ollama 46
3.6.5 Génération du rapport PDF 46
3.7 Conception des données 46

3.8 Principes de conception retenus 47
Conclusion 48

4 Conception et évaluation du moteur intelligent 49

Introduction 50
4.1 Objectifs et principes de conception 50
4.2 Prétraitement et structuration des documents 50
4.3 Similarité lexicale TF–IDF 51
4.4 Similarité sémantique par embeddings 51
4.5 Rapprochement structuré des compétences 51
4.6 Fusion hybride et décision 52
4.7 Explicabilité et couche d'analyse locale 52
4.7.1 Construction du contexte et format de réponse 53
4.7.2 Garde-fou de cohérence et tentative d'injection 53
4.8 Jeu de données et protocole expérimental 54
4.9 Résultats et analyse comparative 55
Deux lectures distinctes du même score 58
4.10 Corrections du moteur et du protocole en cours de projet 60
4.10.1 Sensibilité à la formulation et rétablissement des poids documentés 60
4.10.2 Intégrité du protocole d'évaluation 61
4.11 Limites, risques et perspectives 62
Conclusion 63

5 Réalisation et présentation de la plateforme 64

5.1 Environnement matériel et logiciel 65
5.1.1 Environnement matériel 65
5.1.2 Environnement logiciel 65
5.2 Choix des technologies 66
5.2.1 Technologies frontend 66
5.2.2 Technologies backend 67
5.2.3 Technologies d'intelligence artificielle 67
5.2.4 Technologies de données et d'infrastructure 68

5.3 Organisation du code source . . . 69
5.4 Réalisation du backend . . . 69
5.5 Réalisation du frontend . . . 70
5.6 Intégration frontend–backend . . . 71
5.7 Présentation des interfaces . . . 71
5.7.1 Connexion et identité visuelle . . . 72
5.7.2 Consultation sur mobile . . . 73
5.7.3 Vue d'ensemble, offres et candidats . . . 74
5.7.4 Évaluation, résultat et analyse approfondie . . . 77
5.7.5 Historique, comparaison et rapport PDF . . . 80
5.8 Parcours fonctionnel de démonstration . . . 82
5.9 Difficultés de réalisation et solutions apportées . . . 83

6 Tests, sécurité, DevOps et déploiement Azure 84

6.1 Stratégie de validation . . . 85
6.2 Environnement et données de test . . . 86
6.3 Tests fonctionnels . . . 87
6.4 Validation du moteur intelligent . . . 88
6.5 Tests de l'API et de la persistance . . . 88
6.6 Validation du rapport PDF . . . 89
6.7 Tests de santé et stabilité des services . . . 89
6.8 Résultats de validation de bout en bout . . . 89
6.9 Sécurité applicative . . . 89
6.10 Sécurité réseau et infrastructure . . . 90
6.11 Analyse honnête des limites de sécurité . . . 91
6.12 Traitement des données à caractère personnel . . . 92
6.13 Conteneurisation . . . 93
6.14 Déploiement sur Microsoft Azure . . . 94
6.15 Architecture de déploiement . . . 97
6.16 Exploitation et maîtrise des coûts . . . 97
6.17 Procédures d'exploitation . . . 98
6.18 Bilan du déploiement . . . 98

Conclusion générale **100**

Bibliographie **102**

Résumé **105**

Abstract **106**

Liste des figures

1.1 Logo de CyberCloud Solutions (source : page officielle de l'entreprise [1]) 3
1.2 Déroulement en cascade adapté au projet JobTest 11

2.1 Diagramme global des cas d'utilisation de JobTest 23
2.2 UC-01 : authentification et session 24
2.3 UC-02 : gestion des offres d'emploi 24
2.4 UC-03 : gestion des candidats 25
2.5 UC-04 : évaluation d'un CV 25
2.6 UC-05 : analyse approfondie optionnelle 26
2.7 UC-06 : génération du rapport PDF 26
2.8 UC-07 : classement de plusieurs candidats 27
2.9 UC-08 : historique et résultats 27

3.1 Vue de synthèse fonctionnelle de JobTest 40
3.2 Architecture logique et frontières réseau de JobTest 40
3.3 Diagramme de classes métier simplifié de JobTest 42
3.4 Séquence d'authentification du recruteur 43
3.5 Séquence de création d'une offre d'emploi 43
3.6 Séquence centrale d'évaluation et de persistance d'un CV 45
3.7 Séquence d'analyse approfondie, additive à la Layer 1 46
3.8 Séquence de génération et téléchargement d'un rapport PDF 47

4.1 Pipeline de rapprochement mis en œuvre dans JobTest. La couche 2 est additive et non décisionnelle. 50
4.2 F1 et AUC-ROC des quatre méthodes sur le test isolé. Données : `evaluation_results.json`. 55
4.3 Matrices de confusion des méthodes de rapprochement sur le jeu de test 56
4.4 Carte thermique des scores de rapprochement par domaine 56
4.5 Comparaison des signaux sémantique et de compétences 57
4.6 Scores F1 des méthodes évaluées 58

5.1 Écran réel de connexion de JobTest, sur l'instance locale de démonstration 72

5.2 Écran d'inscription du recruteur 72
5.3 Interface JobTest adaptée à la consultation sur mobile 73
5.4 Deuxième vue de l'interface JobTest sur mobile 74
5.5 Vue d'ensemble de l'interface JobTest 75
5.6 Fonction de recherche dans l'interface JobTest 75
5.7 Consultation des offres d'emploi 76
5.8 Formulaire de création d'une offre d'emploi 76
5.9 Suivi des candidats 77
5.10 Historique des évaluations 77
5.11 Saisie des éléments de l'évaluation d'un candidat 78
5.12 Progression du calcul et score global de l'évaluation 78
5.13 Composantes et explication du résultat d'évaluation 79
5.14 Visualisation complémentaire du résultat d'évaluation 79
5.15 Poursuite de l'analyse approfondie dans le parcours candidat 80
5.16 Sélection de l'offre et des CV dans Talent Leaderboard 80
5.17 Classement initial des candidats dans Talent Leaderboard 81
5.18 Podium et détail d'un classement multi-CV 81
5.19 Résultats complémentaires du classement multi-CV 82
5.20 Écran de paramètres et état des composants de la plateforme 82

6.1 Architecture concrète de déploiement : accès public, VM et réseau Docker interne . . 91
6.2 Vue de l'abonnement Azure, recadrée 95
6.3 Aperçu de consommation Azure, recadré 95
6.4 Ressources Azure associées à la démonstration 96
6.5 Propriétés observées de la machine virtuelle 96
6.6 État de la machine virtuelle, recadré 96
6.7 Ressource d'adresse IP publique, recadrée 97

Liste des tableaux

1.1 Comparaison fonctionnelle fondée sur les documentations officielles consultées [8], [9], [10], [11], [12], [13] . . . 7

2.1 Acteurs et parties prenantes de JobTest . . . 14
2.2 Besoins fonctionnels de JobTest . . . 15
2.3 Besoins non fonctionnels . . . 20
2.4 Règles de gestion . . . 22
2.5 UC-01 : s'authentifier . . . 29
2.6 UC-02 : gérer une offre d'emploi . . . 29
2.7 UC-03 : gérer un candidat . . . 30
2.8 UC-04 : évaluer un CV . . . 31
2.9 UC-05 : demander une analyse approfondie . . . 31
2.10 UC-06 : générer un rapport PDF . . . 32
2.11 UC-07 : classer plusieurs candidats . . . 33
2.12 UC-08 : consulter l'historique . . . 33
2.13 Matrice de traçabilité des besoins . . . 34
2.14 Contraintes et hypothèses spécifiques à JobTest . . . 36

3.1 Modules applicatifs et responsabilités . . . 41

4.1 Seuils de recommandation de la couche 1 . . . 52
4.2 Comparaison sur le jeu de test isolé . . . 55
4.3 Le même score hybride lu au point de calibration et aux bornes du produit (1 800 paires de test) . . . 59
4.4 Effet de la reformulation à compétences identiques, avant et après le rétablissement des poids . . . 60
4.5 Effet de la calibration par méthode sur les métriques publiées (exactitude / F1 / AUC-ROC) . . . 61
4.6 Synthèse des couches et de leur statut . . . 62

5.1 Environnement matériel local de réalisation . . . 65

5.2 Environnement de réalisation effectivement utilisé 66
5.3 Synthèse des choix techniques et de leur usage 68
5.4 Difficultés de réalisation et réponses apportées 83

6.1 Niveaux de validation retenus pour JobTest 86
6.2 Matrice des contrôles fonctionnels attestés 87
6.3 Services conteneurisés de JobTest 94
6.4 Configuration observée de la machine virtuelle de démonstration 95
6.5 Synthèse des résultats observés et des limites 99

Liste des abréviations

- **API** = Interface de programmation applicative
- **ASGI** = Interface asynchrone entre un serveur Web et une application Python
- **ATS** = Système de suivi des candidatures
- **AUC** = Aire sous la courbe
- **BERT** = Modèle de représentation contextuelle bidirectionnelle du texte
- **CPU** = Processeur central
- **CRUD** = Créer, lire, mettre à jour et supprimer
- **CSS** = Feuilles de style en cascade
- **CSV** = Fichier texte de valeurs séparées par des virgules
- **CV** = Curriculum vitae
- **DOCX** = Format de document Word fondé sur XML
- **DevOps** = Pratiques associant développement et exploitation
- **HTTP** = Protocole de transfert hypertexte
- **HTTPS** = Protocole de transfert hypertexte sécurisé par TLS
- **IA** = Intelligence artificielle
- **IP** = Protocole Internet, et par extension l'adresse qu'il attribue
- **JSON** = Format d'échange de données textuel
- **JWT** = JSON Web Token
- **LLM** = Grand modèle de langage
- **LTS** = Version bénéficiant d'un support à long terme
- **NSG** = Network Security Group

- **OAuth2** = Protocole de délégation d'autorisation, utilisé ici pour le formulaire de connexion
- **ORM** = Correspondance objet–relationnel
- **OWASP** = Open Worldwide Application Security Project
- **PDF** = Portable Document Format
- **RAM** = Mémoire vive
- **RH** = Ressources humaines
- **ROC** = Courbe caractéristique de fonctionnement du récepteur
- **SQL** = Langage de requête structuré pour bases relationnelles
- **SSD** = Disque à mémoire flash
- **SSH** = Protocole d'accès distant sécurisé à une machine
- **TCP** = Protocole de transport en mode connecté
- **TF–IDF** = Term Frequency–Inverse Document Frequency
- **TLS** = Protocole de chiffrement des échanges réseau
- **UFW** = Pare-feu en ligne de commande d'Ubuntu
- **UML** = Unified Modeling Language
- **URL** = Adresse d'une ressource sur le Web
- **VM** = Machine virtuelle
- **WAF** = Pare-feu applicatif Web
- **XDR** = Détection et réponse étendues aux incidents de sécurité
- **XML** = Langage de balisage extensible
- **XSS** = Injection de script dans une page Web

Introduction générale

Le recrutement exige d'examiner des CV nombreux et hétérogènes au regard d'offres dont les exigences sont souvent formulées de manière libre. Cette première lecture mobilise du temps, tout en devant rester justifiable et attentive au contexte de chaque candidature. Une assistance logicielle peut organiser les informations utiles, mais elle ne peut pas se substituer au jugement du recruteur ni produire seule une décision d'embauche.

Ce mémoire présente *JobTest*, réalisé dans le cadre du stage de PFE chez CyberCloud Solutions. Le projet répond au besoin d'une aide à la présélection qui rapproche un CV et une offre, rend visibles les éléments de correspondance et conserve les résultats d'évaluation. La plateforme permet au recruteur de gérer ses offres et ses candidats, de lancer des évaluations sur du texte ou des documents, puis de consulter un résultat composé d'un score, de sous-scores, de compétences détectées, d'écarts et de recommandations.

La solution associe une couche déterministe fondée sur TF–IDF, la similarité sémantique et les compétences, à une politique de décision explicite. Une analyse qualitative locale peut compléter la restitution, sans modifier le verdict officiel. L'application s'appuie sur un frontend Next.js, une API FastAPI, une persistance relationnelle et un export PDF. Les validations documentées couvrent les parcours principaux, un benchmark sur données synthétiques et un déploiement de démonstration conteneurisé sur Microsoft Azure.

Le premier chapitre présente le contexte, l'organisme d'accueil, la problématique et la démarche. Le deuxième précise les besoins et les règles de gestion. Le troisième expose la conception fonctionnelle et architecturale. Le quatrième décrit le moteur d'évaluation et son protocole expérimental. Le cinquième présente la réalisation de la plateforme, tandis que le sixième traite des tests, de la sécurité, de l'industrialisation et du déploiement. La conclusion générale synthétise les apports, les limites et les perspectives de JobTest.

Chapter 1

Contexte général et étude préalable

Plan

1 Présentation de l'organisme d'accueil 3
2 Contexte du projet 4
3 Étude de l'existant 5
4 Critique de l'existant 7
5 Problématique 8
6 Solution proposée : JobTest 8
7 Objectifs du projet 9
8 Méthodologie de conduite du projet 10

Introduction

Le présent chapitre fixe le cadre organisationnel, fonctionnel et méthodologique du projet *JobTest*. Il situe d'abord le stage au sein de CyberCloud Solutions, puis examine le problème de présélection de CV et les réponses actuellement proposées par les outils numériques. Cette étude préalable sert à délimiter précisément la contribution de JobTest : une aide à l'analyse et à la comparaison de candidatures, et non un mécanisme autonome de recrutement. Le chapitre formule ensuite la problématique, décrit la solution retenue, ses objectifs ainsi que la démarche de conduite du projet.

1.1 Présentation de l'organisme d'accueil

1.1.1 Présentation de CyberCloud Solutions

CyberCloud Solutions est une entreprise de services et de conseil en informatique fondée en 2017. Sa page publique la situe dans le Grand Tunis, indique un effectif de deux à dix personnes et présente une organisation tournée vers l'accompagnement technique, le conseil et la transformation numérique [1]. Dans ce contexte, l'entreprise peut réunir des besoins variés autour des environnements numériques sans que cela implique une spécialisation unique sur le recrutement.

Son positionnement associe prestation de services, assistance et conseil. La flexibilité du travail à distance ou hybride est cohérente avec le cadre du stage décrit dans ce mémoire : elle permet de structurer un travail individuel autour de livrables partagés et vérifiables, sans présumer d'une organisation interne, d'un rythme de réunion ou d'une répartition de services qui ne sont pas documentés publiquement.

La figure 1.1 reproduit le logo employé par l'entreprise sur sa page officielle. Il est utilisé ici uniquement pour identifier l'organisme d'accueil.



Figure 1.1: Logo de CyberCloud Solutions (source : page officielle de l'entreprise [1])

1.1.2 Activités et domaines d'expertise

Les activités publiées par CyberCloud Solutions peuvent être regroupées sans les confondre avec le périmètre de JobTest. Un premier ensemble concerne le conseil, l'assistance et la transformation numérique. Un deuxième porte sur l'infrastructure : équipements informatiques et bureautiques, serveurs, stockage, postes de travail, imprimantes et logiciels professionnels. La présentation publique mentionne également des solutions de sécurité Internet, de *network forensics* et d'interception légale [1].

Les spécialités affichées complètent ce panorama : cybersécurité, XDR, réseaux, serveurs, cloud, DevOps, logiciels et intelligence. Elles montrent un environnement technique pluridisciplinaire ; elles ne signifient pas que chacun de ces domaines a été directement mobilisé pour la plateforme. JobTest relève plus précisément de l'ingénierie d'une application d'aide au recrutement, à l'intersection du traitement de texte, du développement web, de la persistance des données et de l'évaluation de modèles.

1.1.3 Cadre du stage

Ce projet est réalisé par Iheb El Ech dans le cadre du Mastère Professionnel en Applications Web Intelligentes de l'Institut Supérieur d'Informatique du Kef, relevant de l'Université de Jendouba, pour l'année universitaire 2025–2026. Le stage de PFE s'étend de mars à août 2026. Le sujet, proposé par CyberCloud Solutions, est : *conception et développement d'une plateforme intelligente d'aide au recrutement et de mise en correspondance entre CV et offres d'emploi.*

L'étudiant intervient individuellement en qualité de stagiaire PFE, dans un mode principalement distant ou hybride. L'encadrement professionnel est assuré par Dr. Trabelsi Emna. Ce cadre précise l'origine professionnelle du besoin tout en conservant le caractère académique du travail : les choix techniques, les résultats et leurs limites doivent pouvoir être explicités et contrôlés.

1.2 Contexte du projet

Une candidature condense des informations de natures différentes : expériences, formations, compétences déclarées, réalisations, certifications et éléments libres. Les offres d'emploi présentent elles aussi des exigences hétérogènes, parfois sous forme de listes, parfois dans des descriptions rédigées. La généralisation des candidatures en ligne a accru le volume à examiner et rend la présélection plus difficile à organiser manuellement [2].

L'intelligence artificielle désigne notamment des systèmes capables de produire des prédictions, recommandations ou décisions pour des objectifs définis par des humains [3]. Dans le cas du recrutement, cette capacité ne retire ni la responsabilité ni le jugement professionnel du recruteur. Les travaux sur les outils algorithmiques de ressources humaines soulignent à la fois leur intérêt pour traiter de nombreux dossiers et les risques de biais, d'opacité ou de mauvaise interprétation qui justifient une supervision humaine [2], [4].

Une recherche par mots-clés est utile pour repérer des termes explicitement demandés, mais elle ne traite pas toujours les formulations proches, les synonymes ou les compétences décrites indirectement. La représentation contextuelle des textes a précisément été développée pour mieux exploiter le contexte linguistique [5]. Des représentations de phrases comparables par similarité cosinus permettent, par exemple, de rapprocher deux textes sur une base sémantique plutôt que de compter seulement des mots identiques [6]. Pour la présélection, l'enjeu n'est donc pas de remplacer le recruteur, mais de fournir un résultat reproductible, décomposé et intelligible : ce qui concorde, ce qui manque et ce qui a conduit à une recommandation.

1.3 Étude de l'existant

1.3.1 Processus manuel de présélection

Dans un processus manuel, le recruteur lit l'offre, définit ses critères, ouvre les CV, repère les expériences et compétences pertinentes, puis conserve ou écarte des candidatures avant les étapes suivantes. Cette séquence laisse place à une appréciation qualitative indispensable, notamment pour interpréter un parcours atypique ou contextualiser une expérience.

Elle devient cependant coûteuse à répéter lorsque les documents sont nombreux et de formats disparates. Elle rend également plus difficile la traçabilité du raisonnement appliqué à chaque dossier. La littérature consacrée aux décisions algorithmiques en ressources humaines rappelle que la recherche d'efficacité ne constitue pas, à elle seule, une garantie d'équité : la définition des critères, la qualité des données et le contrôle humain restent déterminants [4]. JobTest ne cherche donc pas à transformer une préconisation en décision d'embauche ; il structure l'examen initial et rend visibles ses éléments de calcul.

1.3.2 Approches lexicales et ATS traditionnels

Les systèmes de suivi de candidatures (*Applicant Tracking Systems*, ou ATS) organisent habituellement les offres, les profils candidats, les étapes de pipeline et les évaluations. Dans ce type d'environnement, la recherche peut s'appuyer sur des mots, des étiquettes, des filtres ou des champs structurés. Cette approche est efficace quand l'exigence est explicitement formulée et que la terminologie est stable. Elle devient moins robuste lorsqu'une compétence est exprimée par une paraphrase, lorsque le CV est multilingue ou lorsqu'une expérience pertinente ne reprend pas le vocabulaire exact de l'offre.

L'analyse textuelle par termes demeure ainsi un signal, non une preuve complète d'adéquation. Les revues sur le recrutement assisté indiquent que les outils de sélection vont aujourd'hui de la recherche de mots à des traitements cherchant des correspondances plus contextuelles ; elles invitent également à distinguer le filtrage technique de la validité d'une décision de sélection [4]. La comparaison sémantique et l'analyse explicite des compétences constituent donc des compléments pertinents, à condition que leurs résultats restent contrôlables.

1.3.3 Solutions numériques étudiées

Trois solutions ont été retenues pour une comparaison ciblée, à partir de leur documentation officielle consultée le 13 septembre 2026. LinkedIn Recruiter propose une recherche assistée par IA à partir d'une demande en langage naturel ; selon l'offre et la langue, elle peut produire des recommandations et une synthèse expliquant l'adéquation affichée sur la carte d'un candidat [7], [8]. La documentation étudiée ne permet pas de conclure à la présence d'une analyse détaillée des lacunes de compétences ni d'un export de rapport de correspondance CV/offre.

Workable centralise les profils, les CV et le pipeline. Sa vue de candidature permet notamment une recherche par mots-clés ou recherche avancée, l'affichage des compétences et le tri par meilleure évaluation ou meilleur appariement fondé sur les évaluations IA [9]. Sa documentation de sécurité mentionne des dispositifs de protection des données, dont le chiffrement en transit, des mécanismes liés aux droits des candidats et des contrôles de conformité [10]. Ces informations décrivent la plateforme Workable, pas le niveau de protection d'une installation particulière.

Greenhouse Recruiting documente une fonction *Talent Matching* qui compare les CV à une calibration définie et pondérée pour le poste. Il propose aussi des *scorecards* pour évaluer les candidats selon des critères préparés, ainsi que des rapports candidats exportables au format tableur [11], [12], [13]. Les sources consultées ne documentent pas un diagnostic autonome de lacunes de

compétences comparable à celui visé par JobTest.

1.3.4 Synthèse comparative

Le tableau 1.1 met en regard des capacités directement documentées. Il ne mesure ni la qualité réelle de chaque solution, ni leur couverture complète : la mention *non documenté* signifie seulement qu'aucune preuve explicite n'a été trouvée dans les pages officielles consultées. Les informations sur JobTest proviennent du code, des rapports techniques du dépôt et du déploiement vérifié du projet.

Critère	**LinkedIn Recruiter**	**Workable**	**Greenhouse Recruiting**	**JobTest**
Correspondance CV/offre	Recherche IA et recommandations à partir d'une demande ; synthèse de fit selon disponibilité	Tri *best match* issu d'évaluations IA ; CV dans le pipeline	Talent Matching : CV comparé à une calibration pondérée	Analyse hybride : lexical, sémantique et compétences
Éléments explicatifs	Synthèse IA de fit pour certains accès ; granularité non documentée	Évaluation et score visibles ; décomposition d'IA non documentée	Scorecards structurées et critères d'entretien	Scores composantes, compétences, écarts et recommandations
Analyse des écarts de compétences	Non documenté	Non documenté	Non documenté	Compétences correspondantes, manquantes et critiques
Classement et export	Résultats de recherche ; export de rapport d'appariement non documenté	Tri de candidats ; export de rapport d'appariement non documenté	Rapport candidat exportable en XLS	Classement multi-CV, historique et rapport PDF
Contrôle et données	Modèle de déploiement/confidentialité non documenté dans les sources consultées	Documentation sécurité et confidentialité disponible	Modèle de déploiement/confidentialité non documenté dans les sources consultées	Persistance limitée au recruteur ; déploiement PFE sur VM Azure

Tableau 1.1: Comparaison fonctionnelle fondée sur les documentations officielles consultées [8], [9], [10], [11], [12], [13]

1.4 Critique de l'existant

Les plateformes étudiées couvrent des besoins opérationnels importants : recherche, pipeline, évaluation structurée, gestion de candidats et, selon les produits, fonctions d'IA. Elles ne sont donc pas des alternatives faibles à écarter ; elles répondent à des contextes organisationnels plus larges que le périmètre du PFE. La comparaison révèle surtout un besoin de cohérence entre l'analyse du texte, le contrôle de la règle de décision et l'explication fournie au recruteur.

JobTest adresse ce besoin par une chaîne unique : une analyse hybride associe un signal lexical, un signal sémantique et une analyse des compétences ; un moteur de décision central applique la même politique à toutes les interfaces ; les composantes, les compétences manquantes et des recommandations sont restituées avec le résultat. Les évaluations enregistrent également des

métadonnées de reproductibilité, notamment la configuration de décision et les poids utilisés. Une analyse qualitative facultative, exécutée localement via Ollama, intervient après le score statistique et ne peut ni le modifier ni annuler la recommandation.

Cette proposition comporte des limites qu'il faut exposer dès maintenant. L'évaluation hors ligne repose sur des données synthétiques ; elle ne constitue donc pas une preuve de performance sur des décisions d'embauche réelles. L'inférence sémantique et la petite couche LLM locale sont exécutées sur CPU, ce qui introduit une latence perceptible pour l'analyse approfondie. Le déploiement Azure vérifié est adapté à une démonstration de PFE : il s'agit d'une instance unique, publiée en HTTP, sans dispositif de haute disponibilité. Enfin, le petit modèle local peut produire une analyse superficielle ou inexacte ; son résultat est strictement consultatif. Ces limites renforcent la nécessité d'un recruteur décisionnaire.

1.5 Problématique

La problématique du projet peut être formulée ainsi :

> *Comment fournir au recruteur, à partir d'un CV et d'une offre d'emploi, une évaluation hybride de la proximité sémantique et des compétences qui soit explicable, reproductible et persistante, tout en préservant une analyse qualitative consciente de ses limites et sans déléguer la décision d'embauche à l'outil ?*

Cette question associe quatre exigences. Il faut traiter le contenu hétérogène des documents, rendre la recommandation vérifiable par ses composantes, conserver le contexte nécessaire à la reproductibilité et encadrer l'usage d'une analyse générative par une approche respectueuse des données et du rôle du recruteur.

1.6 Solution proposée : JobTest

JobTest est une plateforme web d'aide au recrutement destinée au recruteur ou spécialiste RH. Elle permet de créer et gérer des offres, de saisir le texte d'un CV ou de téléverser un document PDF ou DOCX, puis de lancer une évaluation contre une offre sélectionnée. Le résultat comprend une recommandation `HIRE`, `CONSIDER` ou `REJECT`, un score global, une décomposition des composantes et une présentation des compétences détectées, correspondantes ou absentes.

La plateforme complète l'analyse individuelle par la gestion des candidats et de l'historique,

un classement de plusieurs CV pour une même offre, ainsi qu'un rapport PDF. Elle met aussi à disposition une comparaison de modèles et un *AI Quality Center* qui expose un benchmark hors ligne avec sa provenance. Ces éléments distinguent une aide au recrutement instrumentée d'une simple recherche textuelle.

Une analyse approfondie optionnelle est disponible via un modèle local exécuté par Ollama. Elle produit une lecture qualitative, des points d'attention, des recommandations et des questions d'entretien. Elle est appelée après la couche statistique ; elle est donc additionnelle. La décision et le score officiels restent issus du moteur déterministe de la plateforme. Les données métier sont isolées par recruteur, et les résultats d'analyse sont conservés afin de pouvoir les retrouver dans le workflow.

1.7 Objectifs du projet

1.7.1 Objectif général

L'objectif général est de concevoir et développer une plateforme d'aide à la présélection qui aide un recruteur à comparer des CV à une offre de façon structurée, explicable et reproductible, sans automatiser la décision d'embauche.

1.7.2 Objectifs spécifiques

Les objectifs spécifiques retenus sont les suivants :

- mettre en oeuvre un workflow de recrutement comprenant l'authentification, la gestion des offres et la gestion des candidats ;
- accepter un CV sous forme de texte saisi ou de document PDF/DOCX ;
- associer des signaux lexicaux, sémantiques et orientés compétences dans une même évaluation ;
- appliquer une politique de décision cohérente pour les recommandations `HIRE`, `CONSIDER` et `REJECT` ;
- exposer les composantes du score, les compétences correspondantes, les écarts et des recommandations d'amélioration ;
- enregistrer les évaluations, leurs paramètres utiles et l'historique associé au recruteur ;

- générer un rapport PDF et proposer un classement de plusieurs candidatures pour une offre ;
- comparer les méthodes disponibles et afficher les résultats d'une évaluation hors ligne avec leur provenance ;
- intégrer des mesures de sécurité applicative, notamment l'authentification et le cloisonnement des données par recruteur ;
- déployer une version démontrable de la solution avec le frontend, l'API, la base de données et la couche d'analyse locale.

1.8 Méthodologie de conduite du projet

1.8.1 Justification du modèle en cascade

Le modèle en cascade organise un projet logiciel en séquences identifiables, depuis l'expression du besoin jusqu'à la validation. Dans son article fondateur, Royce insiste sur la documentation et sur la nécessité de retours contrôlés lorsque les résultats d'une phase révèlent un problème [14]. Il ne doit donc pas être interprété comme une interdiction absolue de corriger l'implémentation.

Pour JobTest, ce cadre convient à un PFE dont les livrables attendus peuvent être stabilisés : besoins, conception, réalisation, tests et déploiement. Il fournit un fil de lecture clair pour le mémoire et la démonstration. Des corrections techniques et des retours de validation ont néanmoins été réalisés à l'intérieur des phases d'implémentation et de validation, notamment sur la cohérence de la décision, l'analyse des compétences, la qualité du benchmark et la sécurisation des accès. Le modèle en cascade décrit ici la progression principale ; il ne masque pas ces boucles de correction.

1.8.2 Phases appliquées à JobTest

La figure 1.2 présente la déclinaison de cette démarche pour JobTest. Chaque étape produit un résultat qui alimente la suivante, tandis que la phase de validation peut conduire à corriger le livrable concerné.

Le recueil et la formalisation des besoins ont permis de préciser le rôle du recruteur, les entrées CV/offre et les attentes d'explication. L'analyse a ensuite structuré les cas d'usage, les données manipulées et les contraintes de confidentialité. La conception a défini la séparation entre interface web, API, moteur d'appariement, décision, persistance et analyse qualitative optionnelle.

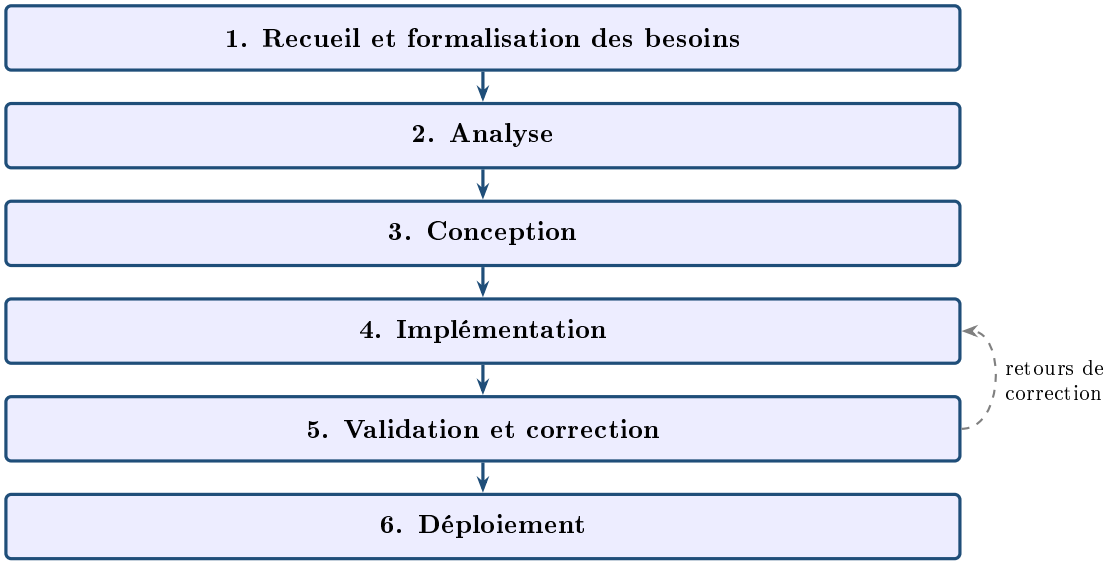


Figure 1.2: Déroulement en cascade adapté au projet JobTest

L'implémentation a produit l'application web, les services de traitement des CV, la gestion des offres et des candidats, l'export PDF et les écrans d'évaluation. La validation a porté sur les parcours fonctionnels, le classement, les rapports, les résultats d'évaluation et les corrections du moteur. Enfin, le déploiement a rassemblé Next.js, FastAPI, PostgreSQL, Ollama et Nginx dans une configuration Docker Compose sur une machine virtuelle Azure. Cette dernière étape reste une livraison de démonstration, avec les limites d'exploitation déjà indiquées.

Conclusion

Ce chapitre a présenté CyberCloud Solutions, le cadre individuel du stage et les contraintes du problème de présélection. L'étude de l'existant a montré l'intérêt des ATS et de l'assistance par IA, tout en mettant en évidence l'importance de l'explicabilité, de la supervision humaine et de la prudence face aux résultats automatisés. JobTest répond à ce besoin par une analyse hybride, une décision cohérente et une restitution orientée recruteur, tout en assumant les limites de ses données, de ses modèles et de son déploiement. Le chapitre suivant, *Analyse et spécification des besoins*, détaillera les acteurs, les besoins fonctionnels et non fonctionnels ainsi que la spécification du système.

Chapter 2

Analyse et spécification des besoins

Plan

1 Périmètre fonctionnel de JobTest 13

2 Identification des acteurs et parties prenantes 13

3 Spécification des besoins fonctionnels 15

4 Spécification des besoins non fonctionnels 19

5 Règles de gestion 22

6 Diagramme global des cas d'utilisation 22

7 Raffinement des cas d'utilisation principaux 23

8 Descriptions textuelles des cas d'utilisation 29

9 Matrice de traçabilité 34

10 Contraintes et hypothèses 35

Introduction

L'analyse des besoins transforme le périmètre présenté au chapitre précédent en attentes vérifiables, cas d'utilisation et règles de gestion. Elle s'appuie ici sur le comportement du dépôt JobTest : interface Next.js, routes FastAPI effectivement sollicitées, persistance et moteur de décision. L'objectif n'est pas de décrire une plateforme RH générique, mais de fixer le contrat fonctionnel de l'application démontrable. Après la délimitation du système et l'identification de ses parties prenantes, le chapitre spécifie les besoins, les contraintes de qualité, les règles de gestion, les cas d'utilisation et leur traçabilité.

2.1 Périmètre fonctionnel de JobTest

JobTest est une application d'aide à la présélection destinée au recruteur. Elle organise des offres et des dossiers candidats, puis fournit une évaluation hybride entre un CV et une offre. Le résultat associe une recommandation officielle, ses composantes, les compétences détectées ou manquantes et des éléments d'explication. L'analyse qualitative produite par le LLM local complète cette restitution sans participer à la décision officielle.

Le périmètre couvert comprend l'inscription et la session du recruteur, la gestion des offres et des candidats, l'évaluation d'un texte ou d'un document PDF/DOCX, la consultation des résultats et de l'historique, le classement multi-CV, l'export PDF, la comparaison ponctuelle des méthodes, le centre de qualité hors ligne et le réglage de seuils pour la session. Les données opérationnelles et les résultats persistés sont isolés par recruteur.

Sont hors périmètre une décision d'embauche autonome, un portail candidat, la conduite des entretiens et un apprentissage sur des résultats réels d'embauche. JobTest ne remplace donc pas l'appréciation du recruteur ni les étapes humaines de sélection. Son benchmark repose sur des données synthétiques. Enfin, le déploiement Azure validé est une démonstration PFE sur une instance sans haute disponibilité permanente ; il ne constitue pas une offre de production 24/7.

2.2 Identification des acteurs et parties prenantes

2.2.1 Visiteur non authentifié

Avant l'authentification, le visiteur peut créer un compte recruteur ou se connecter. Aucun parcours public de consultation d'offres, de dépôt de candidature, de réinitialisation de mot de passe, de

vérification d'e-mail ou de connexion sociale n'a été retenu, car il n'est pas exposé par l'application actuelle.

2.2.2 Recruteur authentifié

Le recruteur est l'acteur opérationnel principal. Une fois la session JWT établie, il consulte le tableau de bord et gère ses propres offres, candidats et résultats. Il peut évaluer un CV, consulter le résultat explicable, demander l'analyse approfondie facultative, exporter un rapport, classer plusieurs CV, consulter l'historique, comparer les méthodes, visualiser la qualité hors ligne et modifier les seuils actifs de sa session. Les contrôles d'appartenance empêchent l'accès aux objets d'un autre recruteur.

2.2.3 Candidat en tant qu'entité métier

Le candidat est le sujet d'une évaluation et une entité persistée : identité facultative, coordonnées, statut de pipeline, CV et résultats associés. Il n'est pas un acteur logiciel interactif. La version étudiée ne fournit ni compte candidat ni écran candidat ni soumission publique de CV. Cette séparation évite de confondre l'objet métier évalué avec l'utilisateur autorisé à utiliser l'application.

2.2.4 Partie prenante technique

Une personne chargée de maintenance ou d'assurance qualité peut exécuter le benchmark hors ligne, exporter ses figures ou intervenir sur l'environnement. Cette partie prenante est technique et secondaire : aucun portail administrateur, rôle d'administration distinct ni parcours d'authentification administrateur n'est prouvé dans l'interface opérationnelle. Elle ne figure donc pas comme acteur du diagramme global.

Le tableau 2.1 récapitule ces acteurs et situe chacun d'eux par rapport au parcours métier.

Élément	**Type**	**Rôle**	**Accès au système**
Visiteur non authentifié	Acteur	Crée un compte ou amorce une session	Écran d'accès, sans données métier
Recruteur authentifié	Acteur	Gère et évalue ses dossiers	Toutes les fonctions métier après JWT
Candidat	Entité métier	Porte le CV, le pipeline et les résultats	Aucun accès interactif
Maintenance / QA	Partie prenante technique	Produit ou contrôle le benchmark hors ligne	Hors parcours métier courant

Tableau 2.1: Acteurs et parties prenantes de JobTest

2.3 Spécification des besoins fonctionnels

Les besoins sont présentés par domaine afin de décrire le parcours réellement disponible, sans confondre les fonctions du recruteur avec une décision autonome de recrutement.

Le tableau 2.2 énumère ces besoins avec leur identifiant, l'acteur concerné et le critère qui permet d'en constater la réalisation.

Tableau 2.2: Besoins fonctionnels de JobTest

Identifiant	Besoin fonctionnel	Acteur	Priorité	Critère d'acceptation
BF-AUTH-01	Créer un compte recruteur avec nom, e-mail et mot de passe.	Visiteur	Essentiel	Un compte de rôle `recruiter` est créé ; un e-mail déjà utilisé est refusé.
BF-AUTH-02	S'authentifier et récupérer l'utilisateur courant.	Visiteur	Essentiel	Les identifiants valides donnent un JWT bearer puis le profil `/auth/me`.
BF-AUTH-03	Clore la session côté interface.	Recruteur	Essentiel	Le jeton local est supprimé lors de la déconnexion.
BF-JOB-01	Créer et lister ses offres.	Recruteur	Essentiel	Titre et description sont exigés ; un code `JOB-YYYY-NNN` est attribué.
BF-JOB-02	Modifier une offre possédée et son statut.	Recruteur	Essentiel	Les statuts admis sont `OPEN`, `ON_HOLD` et `CLOSED`.
BF-JOB-03	Basculer rapidement l'ouverture d'une offre.	Recruteur	Compl.	L'action dédiée alterne `OPEN` et `CLOSED`.
BF-JOB-04	Supprimer une offre non évaluée.	Recruteur	Essentiel	La suppression est refusée si des évaluations sont liées ; l'offre peut alors être clôturée.
BF-CAND-01	Consulter ses candidats.	Recruteur	Essentiel	La liste est filtrée par propriétaire et expose activité, meilleur score et statut.

Identifiant	Besoin fonctionnel	Acteur	Priorité	Critère d'acceptation
BF-CAND-02	Mettre à jour identité, contact et pipeline.	Recruteur	Essentiel	Nom obligatoire ; statuts admis : `NEW`, `UNDER_REVIEW`, `SHORTLISTED`, `INTERVIEW`, `HIRED`, `REJECTED`.
BF-CAND-03	Supprimer un candidat sans résultat lié.	Recruteur	Essentiel	Une suppression avec évaluation est refusée ; le recruteur peut modifier le statut du pipeline.
BF-EVAL-01	Évaluer du texte CV contre une offre possédée.	Recruteur	Essentiel	Le CV contient au moins cinq mots ; méthode, score, décision et explication sont retournés et persistés.
BF-EVAL-02	Évaluer un PDF ou DOCX.	Recruteur	Essentiel	Seuls PDF et DOCX sont acceptés ; un texte lisible est extrait avant l'évaluation.
BF-EVAL-03	Afficher le résultat explicable.	Recruteur	Essentiel	Verdict `HIRE`/`CONSIDER`/`REJECT`, composantes, compétences, écarts, recommandations et données radar sont restitués.
BF-LLM-01	Demander séparément une analyse approfondie.	Recruteur	Compl.	La requête cible un `MatchResult` possédé ; son état peut être `ok`, désactivé, indisponible ou en erreur.
BF-REPORT-01	Générer et télécharger un rapport PDF depuis un résultat.	Recruteur	Compl.	Le navigateur reçoit un document PDF à partir du résultat et de son explication.

Identifiant	Besoin fonctionnel	Acteur	Priorité	Critère d'acceptation
BF-RANK-01	Classer plusieurs CV pour une offre.	Recruteur	Compl.	Des fichiers PDF/DOCX sont évalués pour une offre sélectionnée, puis ordonnés par score décroissant.
BF-RANK-02	Exporter le classement courant.	Recruteur	Compl.	L'interface produit un CSV côté navigateur ; il ne crée pas un nouvel enregistrement métier.
BF-HIST-01	Consulter l'historique et le détail d'une évaluation.	Recruteur	Essentiel	Les résultats propres au recruteur sont listés, filtrables et consultables par identifiant.
BF-QUALITY-01	Comparer les méthodes en direct.	Recruteur	Compl.	*AI Insights* compare les méthodes sur un CV/offre fourni sans persister de `MatchResult`.
BF-QUALITY-02	Consulter le benchmark hors ligne.	Recruteur	Compl.	*AI Quality Center* affiche des données et figures file-backed avec leur disponibilité.
BF-SETTINGS-01	Configurer les seuils de décision de la session.	Recruteur	Compl.	Les seuils Strong Fit et Potential Fit sont validés, transmis aux évaluations ultérieures et conservés dans celles-ci.

2.3.1 Authentification du recruteur

- Créer un compte recruteur avec nom, e-mail et mot de passe, en refusant un e-mail déjà utilisé.
- Se connecter, consulter son profil courant et fermer la session côté interface.

L'inscription crée uniquement un recruteur. La connexion utilise le formulaire OAuth2 de l'API, qui retourne un jeton bearer. L'interface conserve ce jeton pour appeler `/auth/me` et les routes protégées ; la déconnexion l'efface côté client. Aucun besoin de réinitialisation de mot de passe ou

d'activation n'est spécifié car ces parcours ne sont pas présents.

2.3.2 Gestion des offres et des candidats

- Créer, consulter, modifier, clôturer ou rouvrir ses propres offres ; le code est attribué automatiquement.
- Supprimer une offre seulement lorsqu'aucune évaluation ne lui est liée ; une offre évaluée peut être clôturée.
- Consulter les candidats du recruteur, mettre à jour leurs informations et leur statut de pipeline, puis supprimer un dossier sans résultat associé.

Les offres sont créées avec un titre, une description, des compétences requises facultatives et un statut. Le code est calculé automatiquement. Toutes les opérations vérifient le recruteur propriétaire. Une offre liée à des évaluations ne peut pas être supprimée, afin de préserver la cohérence de l'historique. L'évaluation crée ou réutilise le dossier candidat lorsque les informations disponibles le permettent ; la page candidats sert ensuite à la consultation et à la mise à jour. Les fonctions exposées dans l'interface courante sont la modification et la suppression conditionnelle. L'API historique comporte une anonymisation, mais elle n'est pas présentée comme un parcours du frontend Next.js faute d'action correspondante vérifiée.

2.3.3 Évaluation et restitution

- Évaluer un texte CV ou un document PDF/DOCX par rapport à une offre possédée.
- Consulter le verdict, le score, les composantes, les compétences, les écarts et les recommandations enregistrés.
- Demander séparément une analyse approfondie locale ; son indisponibilité ne modifie pas le verdict officiel.
- Générer et télécharger le rapport PDF d'une évaluation.

Le recruteur sélectionne une offre qu'il possède, saisit éventuellement identité et e-mail, puis transmet du texte ou un fichier. La couche statistique lance l'appariement choisi, le *DecisionEngine* applique les seuils actifs et le service persiste un résultat. Le résultat visible comprend le score global, les sous-scores, la décision, l'explication, les compétences et les écarts. L'analyse approfondie est une seconde demande sur une évaluation déjà persistée. Si Ollama est désactivé, indisponible ou

en erreur, l'état est retourné sans bloquer ni altérer le verdict statistique. Seule une enveloppe avec état `ok` est sauvegardée. Le rapport PDF, enfin, réutilise les données déjà calculées et n'ouvre pas un nouveau calcul de correspondance.

2.3.4 Historique, classement et réglages

- Consulter l'historique et le détail des évaluations appartenant au recruteur.
- Classer plusieurs CV pour une offre, puis exporter le classement courant au format CSV.
- Comparer ponctuellement les méthodes, consulter les résultats du benchmark hors ligne et régler les seuils actifs de la session.

Le classement accepte plusieurs fichiers pour une seule offre et lance l'évaluation de chaque fichier avant de les trier par score décroissant ; l'export CSV est généré par le frontend. L'historique expose les résultats persistés du recruteur et leur détail. *AI Insights* exécute une comparaison directe non persistante, tandis que *AI Quality Center* présente un benchmark hors ligne : il ne doit pas être interprété comme une mesure d'exactitude de production en temps réel. Les seuils Strong Fit et Potential Fit sont configurés dans le navigateur et accompagnent les évaluations ultérieures de cette session ; ils doivent respecter `potential < strong`, et les résultats antérieurs gardent les seuils enregistrés.

2.4 Spécification des besoins non fonctionnels

Les qualités attendues sont structurées à partir du modèle de qualité produit ISO/IEC 25010, utilisé comme grille pour spécifier et évaluer des caractéristiques de logiciel [15]. Les indicateurs suivants expriment des critères vérifiables sur la version étudiée, sans prétendre démontrer une certification.

Le détail figure dans le tableau 2.3, où chaque exigence est accompagnée de l'élément observable qui permet d'en juger.

Tableau 2.3: Besoins non fonctionnels

Identifiant	**Catégorie**	**Exigence**	**Indicateur ou critère de vérification**
BNF-SEC-01	Sécurité	Les opérations métier exigent un utilisateur authentifié et respectent l'appartenance recruteur.	Les routes `/ui` appellent `get_current_user` et filtrent par `recruiter_id`.
BNF-SEC-02	Sécurité	Les mots de passe ne sont pas persistés en clair et la session a une durée limitée.	Le compte stocke `hashed_password` ; la création de JWT applique l'expiration configurée.
BNF-SEC-03	Sécurité	Les services internes ne sont pas directement publiés dans la topologie Docker validée.	Seul Nginx expose HTTP ; API, base et Ollama restent sur le réseau Compose.
BNF-PRIV-01	Confidentialité	Les dossiers et résultats sont isolés par recruteur.	Les listes, détails et vérifications d'objets incluent l'identifiant du recruteur.
BNF-PRIV-02	Confidentialité	La suppression est encadrée pour préserver l'historique ; l'analyse LLM est locale au service Ollama.	Les gardes 409 empêchent la suppression avec résultat ; le LLM est joint sur réseau interne.
BNF-EXPL-01	Explicabilité	Le verdict affiché et persisté provient de la même politique de décision.	`DecisionEngine` produit la décision ; les composantes, écarts et recommandations sont affichés avec le résultat.
BNF-REPRO-01	Reproductibilité	Une évaluation conserve les paramètres utiles à son interprétation.	`MatchResult` porte méthode, version, poids, seuils et métadonnées de décision lorsqu'ils sont fournis.

Identifiant	**Catégorie**	**Exigence**	**Indicateur ou critère de vérification**
BNF-PERF-01	Performance	La couche statistique reste le flux de verdict ; la couche LLM ne le bloque pas.	La Deep Analysis est appelée après l'affichage de Layer 1. Son exécution CPU sur la VM Azure a été observée autour de 54 secondes.
BNF-RES-01	Fiabilité	L'indisponibilité d'Ollama ne corrompt ni ne retarde le verdict Layer 1.	La route renvoie un état `disabled`, `unavailable` ou `error` et ne modifie le résultat officiel.
BNF-ERG-01	Ergonomie	L'interface doit fournir retours de chargement, d'absence et d'erreur, avec une mise en page réactive.	Les composants d'états et les vues responsive sont présents ; les animations consultent `prefers-reduced-motion`.
BNF-PORT-01	Portabilité	La démonstration doit pouvoir être exécutée avec des services séparés et configurables.	Docker Compose assemble Next.js, FastAPI, PostgreSQL, Ollama et Nginx ; les paramètres viennent de l'environnement.
BNF-MAINT-01	Maintenabilité	Les responsabilités d'interface, API, appariement, décision et persistance restent séparées.	Les routes `/ui`, `matching.service`, `decisioning` et les modèles ORM sont des modules distincts.

La sécurité et la confidentialité décrites sont des exigences vérifiées localement, non une affirmation de sécurité complète : aucune conformité légale ni audit externe n'est déduite. La limite de latence de la couche LLM et le caractère démonstratif du déploiement doivent rester visibles lors de l'utilisation de la plateforme.

2.5 Règles de gestion

Le tableau 2.4 énonce les règles que toute évolution ultérieure de la plateforme devra continuer de respecter.

Tableau 2.4: Règles de gestion

ID	Règle de gestion	Justification / effet
RG-01	Un recruteur accède uniquement à ses offres, candidats et évaluations.	Cloisonne les données métier et évite l'accès inter-recruteur.
RG-02	Une offre possédant des évaluations ne peut pas être supprimée définitivement.	Préserve les résultats ; l'offre doit être clôturée.
RG-03	Un candidat possédant des évaluations ne peut pas être supprimé définitivement.	Préserve l'historique ; le statut de pipeline reste modifiable.
RG-04	Les bandes de décision utilisent les seuils Strong Fit et Potential Fit actifs.	La recommandation est cohérente avec la configuration transmise.
RG-05	Le seuil Potential Fit doit être strictement inférieur au seuil Strong Fit.	Évite des bandes de décision ambiguës ou inversées.
RG-06	La couche LLM ne modifie jamais le score ni la décision officielle Layer 1.	L'analyse approfondie demeure consultative et additive.
RG-07	La comparaison *AI Insights* ne persiste pas de `MatchResult`.	Une comparaison exploratoire ne pollue pas l'historique métier.
RG-08	Une Deep Analysis est persistée uniquement si son état est `ok`.	Les états d'indisponibilité ou d'erreur ne sont pas assimilés à une analyse valide.
RG-09	Les résultats historiques conservent leurs métadonnées de score et de seuil.	Les modifications futures de session ne réécrivent pas les évaluations passées.
RG-10	Les téléversements d'évaluation acceptent seulement PDF et DOCX ; le texte extrait doit être exploitable.	Les formats non pris en charge ou sans texte lisible sont rejetés.

2.6 Diagramme global des cas d'utilisation

La notation UML est employée pour séparer les acteurs externes de la frontière du système et représenter les objectifs utilisateur ; UML 2.5.1 est la spécification normative retenue pour ce

vocabulaire [16]. La figure 2.1 ne représente donc ni PostgreSQL, ni Ollama, ni FastAPI ou Next.js comme acteurs : ce sont des composants internes.

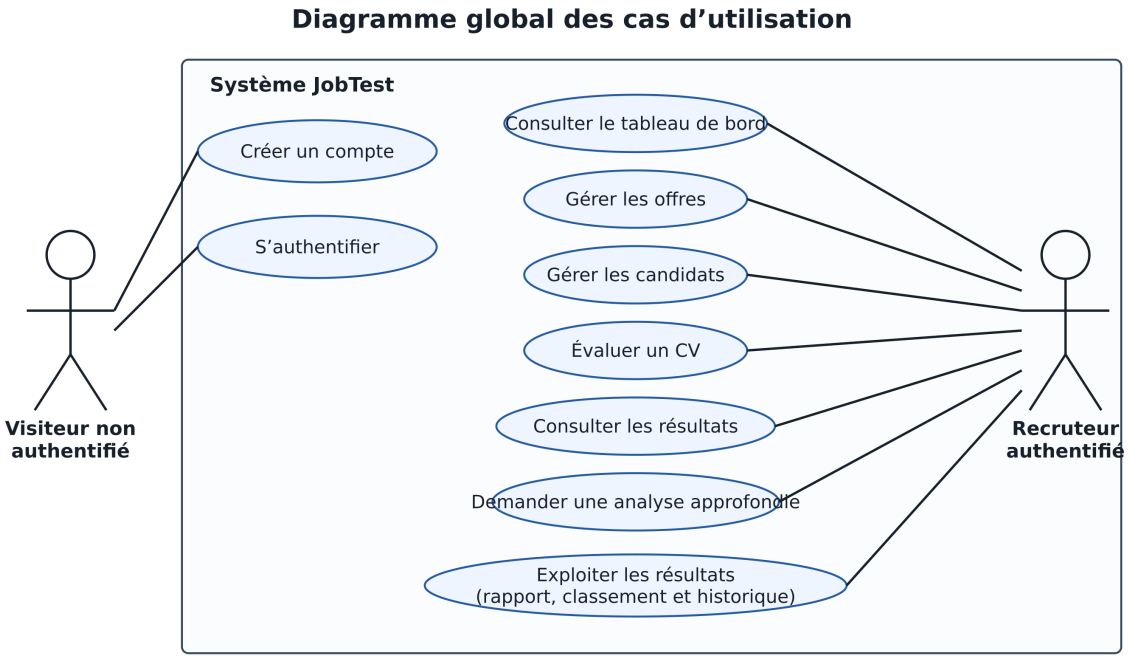


Figure 2.1: Diagramme global des cas d'utilisation de JobTest

Le visiteur reste limité à la création de compte et à l'authentification. Le recruteur porte tous les cas opérationnels. L'analyse approfondie est une extension facultative de la consultation du résultat ; l'export PDF dépend de l'existence d'un résultat. L'authentification est une précondition aux autres cas et non un lien *include* répété dans tout le diagramme.

2.7 Raffinement des cas d'utilisation principaux

Les diagrammes suivants détaillent les huit parcours significatifs sans transformer les opérations internes en cas d'utilisation. Les relations *include* expriment une sous-action nécessaire au but représenté ; la relation *extend* est réservée à l'analyse approfondie facultative.

La série s'ouvre avec la figure 2.2, consacrée à l'authentification et à la gestion de session.

La production du rapport PDF, déclenchée depuis un résultat existant, est représentée par la figure 2.7.

Le classement de plusieurs candidatures sur une même offre fait l'objet de la figure 2.8.

La figure 2.9 termine cette série par la consultation de l'historique des évaluations.

Les figures 2.5 et 2.6 isolent explicitement le flux obligatoire d'évaluation du complément

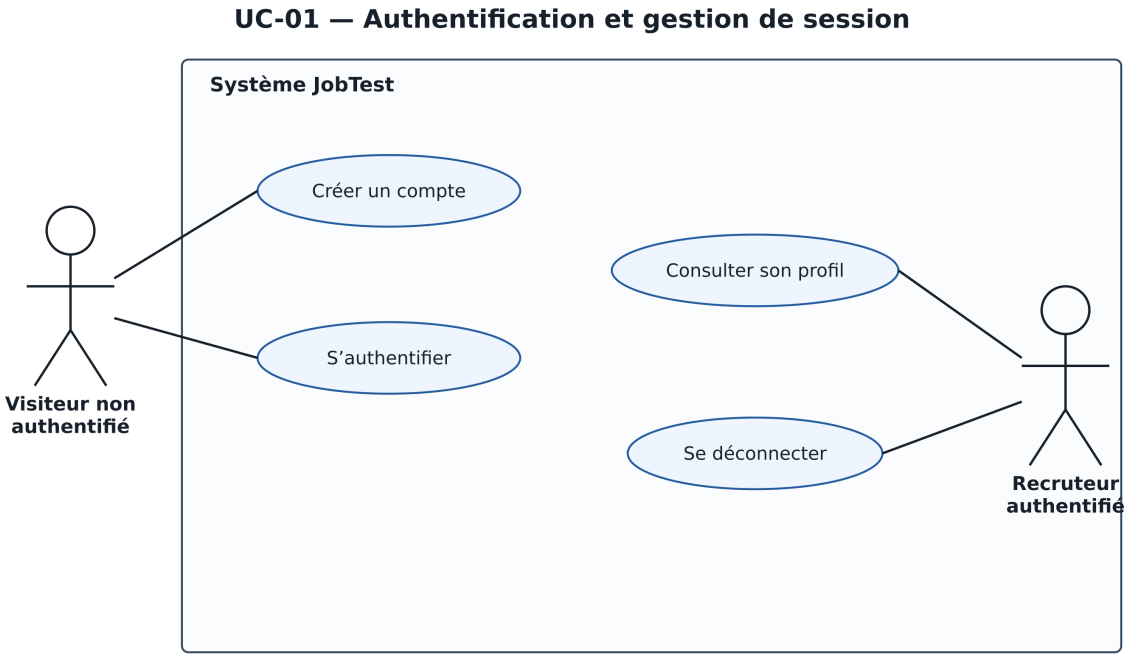


Figure 2.2: UC-01 : authentification et session

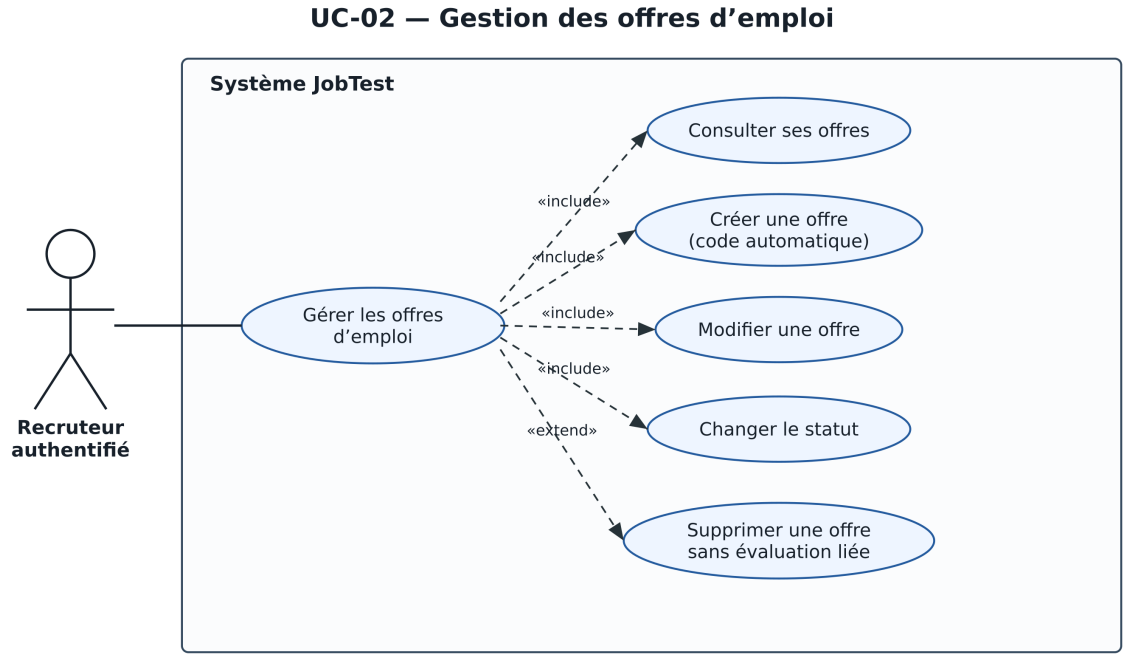


Figure 2.3: UC-02 : gestion des offres d'emploi

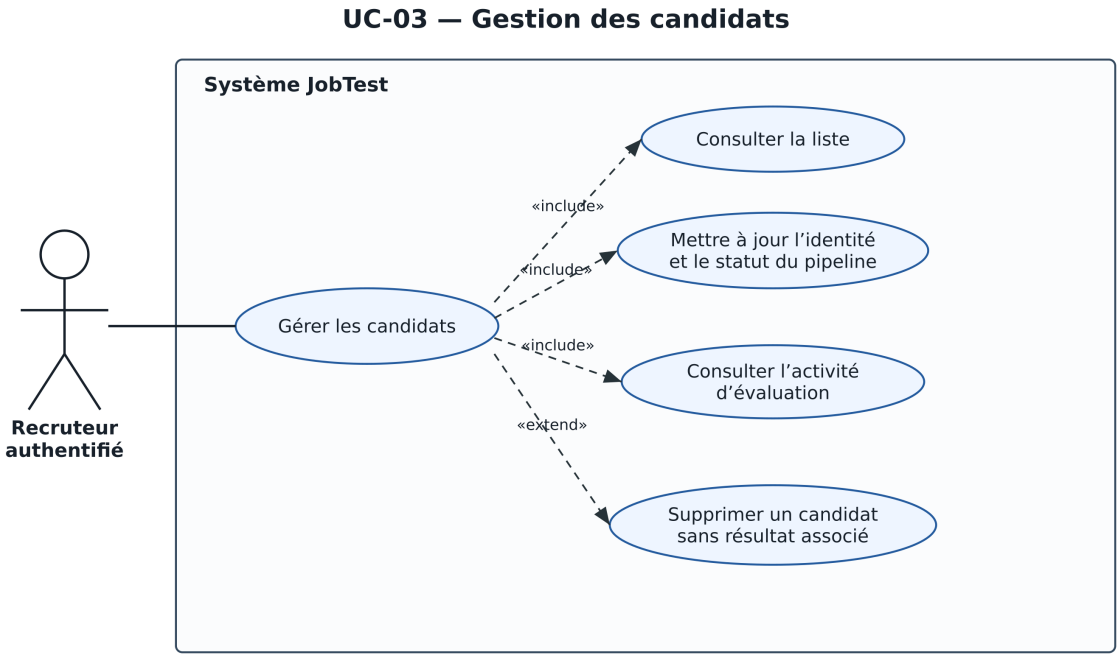


Figure 2.4: UC-03 : gestion des candidats

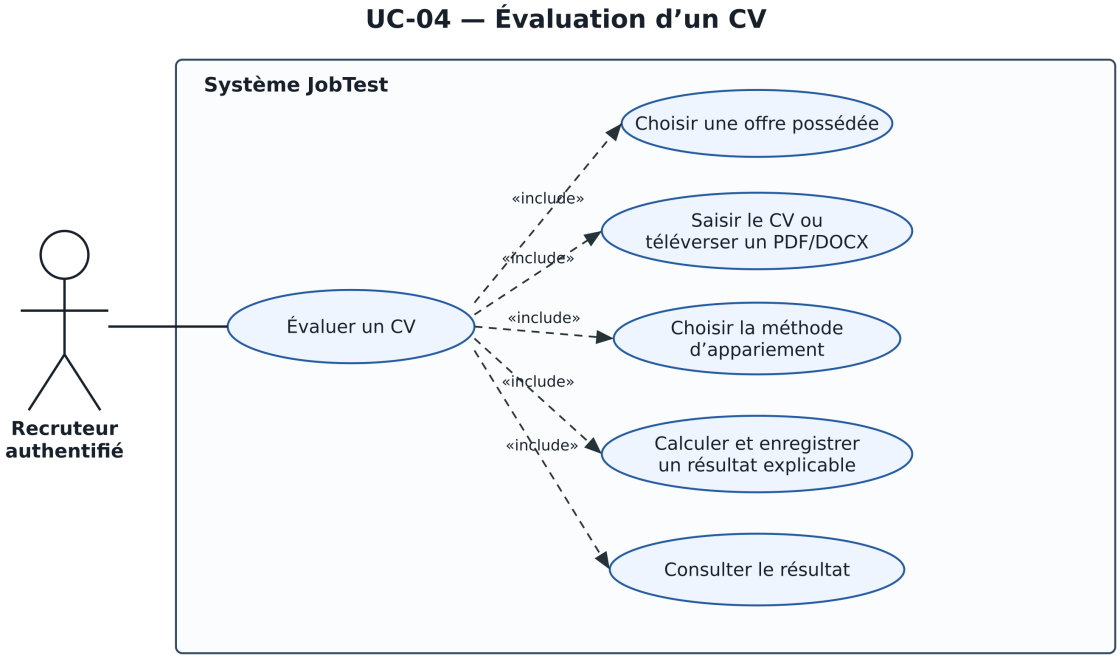


Figure 2.5: UC-04 : évaluation d'un CV

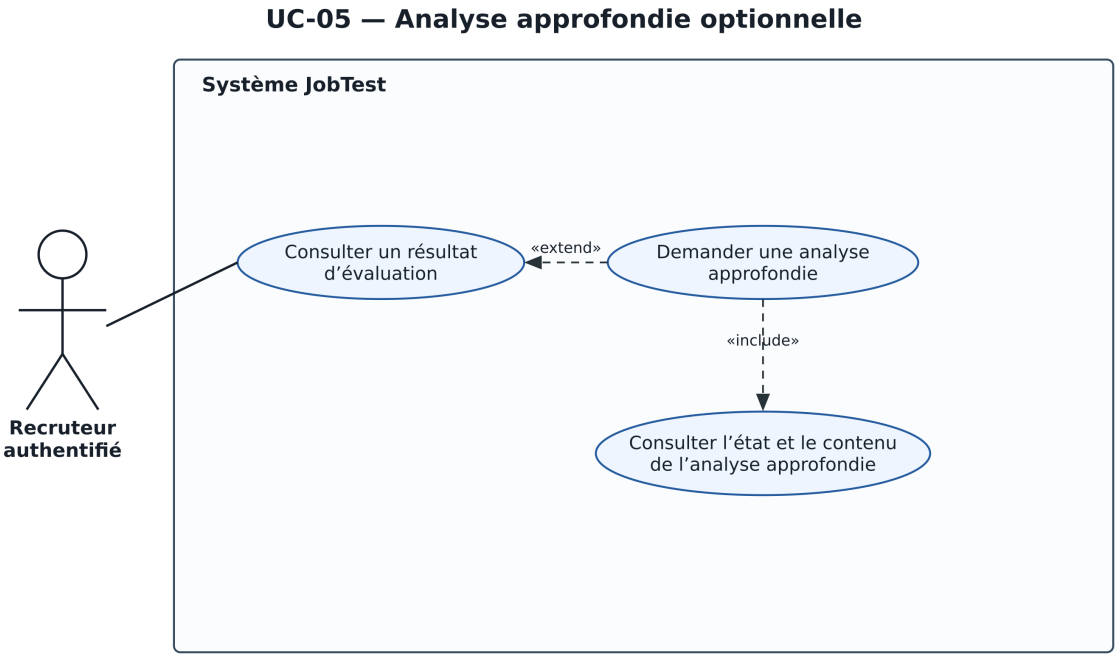


Figure 2.6: UC-05 : analyse approfondie optionnelle

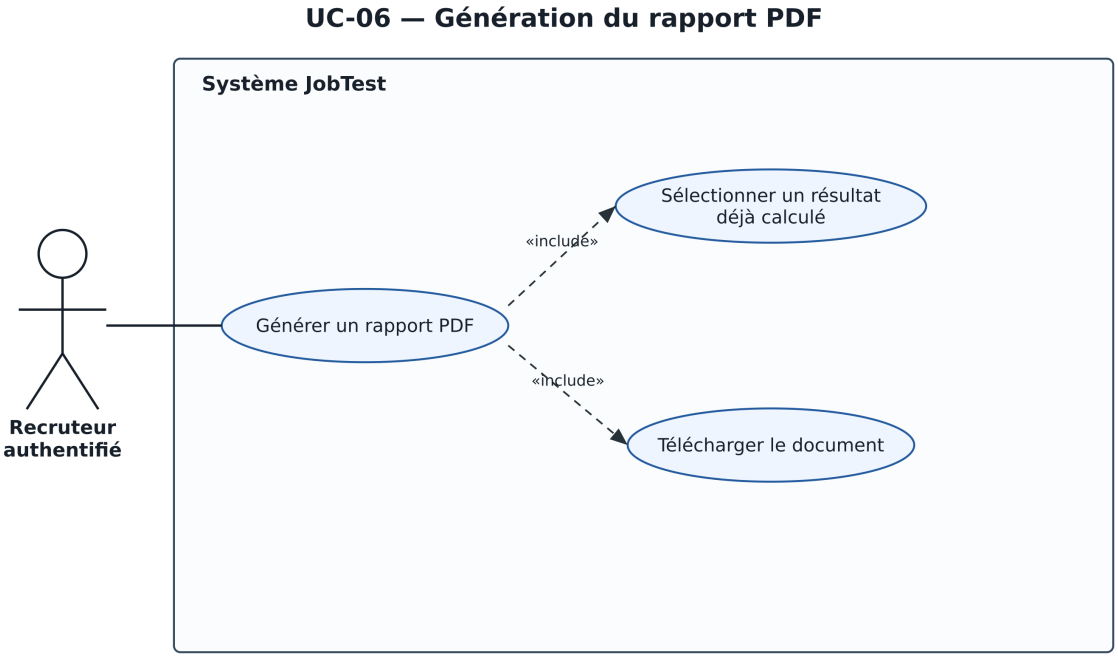


Figure 2.7: UC-06 : génération du rapport PDF

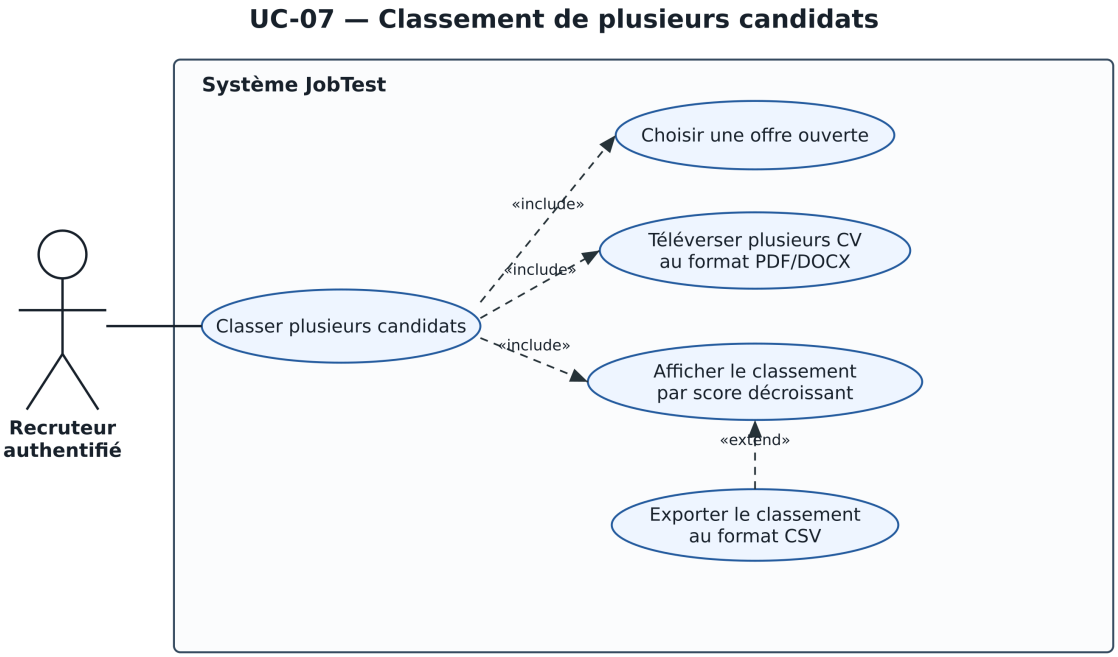


Figure 2.8: UC-07 : classement de plusieurs candidats

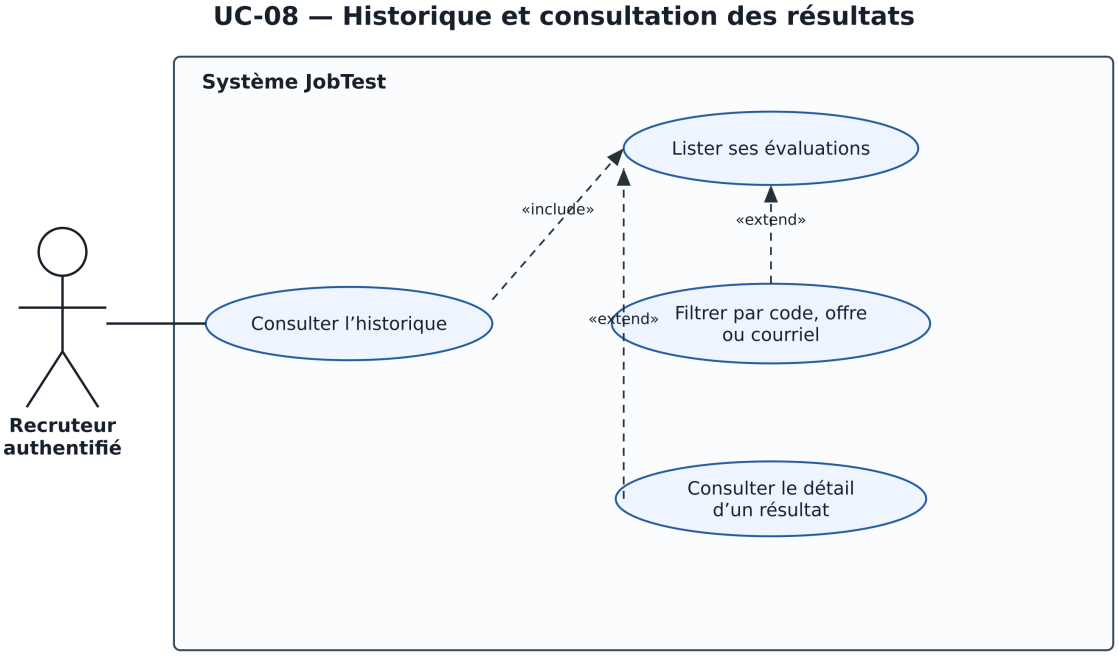


Figure 2.9: UC-08 : historique et résultats

LLM. Les figures 2.3 et 2.4 regroupent les opérations CRUD afin de conserver une vue centrée sur le but du recruteur plutôt que sur chaque bouton d'interface.

2.8 Descriptions textuelles des cas d'utilisation

Les huit cas retenus sont maintenant décrits champ par champ, dans le même ordre que les diagrammes précédents.

Le tableau 2.5 commence par le cas d'authentification.

Tableau 2.5: UC-01 : s'authentifier

hline **Champ**	**Description**
hline Identifiant	UC-01
Titre	S'authentifier
Acteur principal	Visiteur non authentifié
Objectif	Ouvrir une session recruteur donnant accès aux données possédées.
Déclencheur	Le visiteur valide le formulaire de connexion.
Préconditions	Un compte recruteur existe.
Postconditions	Le JWT est stocké côté client et le profil courant est récupérable.
Scénario nominal	1. Le visiteur saisit e-mail et mot de passe. 2. Le système vérifie les identifiants. 3. Le système émet le JWT bearer. 4. L'interface charge `/auth/me` puis redirige vers l'espace opérationnel.
Scénarios alternatifs	Un nouveau visiteur choisit la création de compte, puis revient à la connexion.
Exceptions	Identifiants invalides, compte absent ou API indisponible : message d'erreur, aucune session créée.
Règles associées	RG-01.
hline	

La gestion des offres est décrite dans le tableau 2.6.

Tableau 2.6: UC-02 : gérer une offre d'emploi

hline **Champ**	**Description**
hline Identifiant	UC-02
Titre	Gérer une offre d'emploi
Acteur principal	Recruteur authentifié
Objectif	Créer, consulter, modifier, clôturer ou supprimer une offre possédée.

Déclencheur	Le recruteur ouvre la page des offres ou son formulaire.
Préconditions	Session valide.
Postconditions	L'offre est créée ou mise à jour ; l'historique associé reste cohérent.
Scénario nominal	1. Le recruteur saisit titre, description, compétences et statut. 2. Le système valide titre et description. 3. À la création, il attribue un code `JOB-YYYY-NNN`. 4. Il enregistre l'offre sous le recruteur courant. 5. L'interface actualise la liste.
Scénarios alternatifs	Le recruteur modifie une offre ou utilise l'action de bascule `OPEN`/`CLOSED`.
Exceptions	Offre non possédée ou champs obligatoires absents : opération refusée. Suppression avec évaluations : réponse 409 et proposition de clôture.
Règles associées	RG-01, RG-02.
hline	

Le tableau 2.7 poursuit avec le suivi des candidats.

Tableau 2.7: UC-03 : gérer un candidat

hline **Champ**	**Description**
hline Identifiant	UC-03
Titre	Gérer un candidat
Acteur principal	Recruteur authentifié
Objectif	Mettre à jour et suivre les dossiers candidats issus des évaluations.
Déclencheur	Le recruteur consulte la page candidats et choisit un dossier.
Préconditions	Session valide ; candidat possédé par le recruteur.
Postconditions	Les coordonnées ou le statut de pipeline sont mis à jour, ou le dossier non évalué est supprimé.
Scénario nominal	1. Le système liste les candidats du recruteur. 2. Le recruteur ouvre l'édition. 3. Il modifie nom, e-mail, téléphone ou statut. 4. Le système valide puis enregistre. 5. L'interface affiche les données actualisées.
Scénarios alternatifs	Le recruteur consulte l'activité et le meilleur score du dossier.
Exceptions	Nom vide, e-mail déjà utilisé ou candidat hors périmètre : mise à jour refusée. Candidat évalué : suppression bloquée.
Règles associées	RG-01, RG-03.
hline	

Vient ensuite l'évaluation d'un CV, dont le tableau 2.8 détaille le déroulement et les exceptions.

Tableau 2.8: UC-04 : évaluer un CV

hline **Champ**	**Description**
hline Identifiant	UC-04
Titre	Évaluer un CV
Acteur principal	Recruteur authentifié
Objectif	Obtenir et conserver une évaluation explicable d'un CV contre une offre.
Déclencheur	Le recruteur lance l'évaluation depuis la page d'analyse.
Préconditions	Session valide ; offre possédée ; texte de CV ou fichier PDF/DOCX disponible ; seuils valides.
Postconditions	Un `MatchResult` est persisté avec score, décision, explication et métadonnées.
Scénario nominal	1. Le recruteur choisit une offre. 2. Il fournit identité facultative et texte CV, ou un PDF/DOCX. 3. Le système valide l'entrée et extrait le texte si nécessaire. 4. Il exécute l'appariement statistique choisi. 5. Le *DecisionEngine* détermine le verdict. 6. Le système persiste le résultat. 7. L'interface affiche score, sous-scores, compétences, écarts et recommandations.
Scénarios alternatifs	Le recruteur choisit l'une des méthodes admises : hybride, embedding, TF-IDF ou compétences.
Exceptions	Offre manquante ; texte inférieur à cinq mots ; format non admis ; document sans texte lisible ; seuils invalides ; erreur de persistance : aucune évaluation valide n'est affichée comme réussie.
Règles associées	RG-01, RG-04, RG-05, RG-09, RG-10.
hline	

Le tableau 2.9 traite la demande d'analyse approfondie et son caractère facultatif.

Tableau 2.9: UC-05 : demander une analyse approfondie

hline **Champ**	**Description**
hline Identifiant	UC-05
Titre	Demander une analyse approfondie
Acteur principal	Recruteur authentifié

Objectif	Compléter un résultat persistant par une lecture qualitative locale.
Déclencheur	Le résultat Layer 1 est déjà affiché et le recruteur demande l'analyse.
Préconditions	Session valide ; `MatchResult` possédé par le recruteur.
Postconditions	Une enveloppe LLM est affichée ; elle est sauvegardée seulement si son état est `ok`.
Scénario nominal	1. Le recruteur demande l'analyse. 2. Le système reconstruit le contexte depuis le résultat existant. 3. Il sollicite Ollama. 4. Il vérifie la réponse. 5. Il affiche la synthèse et persiste l'enveloppe réussie.
Scénarios alternatifs	Une analyse déjà disponible peut être relue dans le résultat conservé.
Exceptions	LLM désactivé, indisponible, sortie malformée ou incohérente : l'état est retourné sans modifier score ni verdict.
Règles associées	RG-01, RG-06, RG-08.
hline	

La génération du rapport est reprise par le tableau 2.10.

Tableau 2.10: UC-06 : générer un rapport PDF

hline **Champ**	**Description**
hline Identifiant	UC-06
Titre	Générer un rapport PDF
Acteur principal	Recruteur authentifié
Objectif	Télécharger une restitution portable d'une évaluation calculée.
Déclencheur	Le recruteur active l'export depuis le résultat.
Préconditions	Un résultat et son explication sont disponibles.
Postconditions	Le navigateur reçoit le PDF ; aucun nouveau score n'est calculé.
Scénario nominal	1. Le recruteur sélectionne l'export. 2. L'interface transmet le résultat, l'explication, les recommandations et les libellés disponibles. 3. Le système produit le document. 4. Le navigateur déclenche le téléchargement.
Scénarios alternatifs	Le recruteur revient au résultat sans exporter.
Exceptions	Données de résultat incomplètes ou échec de génération : l'interface signale l'échec.
Règles associées	RG-01, RG-09.

hline	

Le tableau 2.11 formalise le classement de plusieurs candidats pour une offre unique.

Tableau 2.11: UC-07 : classer plusieurs candidats

hline **Champ**	**Description**
hline Identifiant	UC-07
Titre	Classer plusieurs candidats
Acteur principal	Recruteur authentifié
Objectif	Comparer plusieurs CV contre une seule offre.
Déclencheur	Le recruteur sélectionne une offre et plusieurs fichiers.
Préconditions	Session valide ; offre ouverte disponible ; fichiers PDF/DOCX lisibles.
Postconditions	Chaque évaluation réussie est persistée et l'interface affiche le classement décroissant.
Scénario nominal	1. Le recruteur choisit une offre. 2. Il ajoute plusieurs CV PDF/DOCX. 3. L'interface soumet chaque fichier au flux d'évaluation. 4. Elle collecte les résultats réussis. 5. Elle les trie par score décroissant. 6. Elle permet l'export CSV du classement courant.
Scénarios alternatifs	Le recruteur retire un fichier avant le lancement.
Exceptions	Aucun poste, fichier non lisible ou évaluation en échec : le fichier concerné est signalé ; les autres résultats valides restent consultables.
Règles associées	RG-01, RG-04, RG-05, RG-10.
hline	

La consultation de l'historique clôt la série avec le tableau 2.12.

Tableau 2.12: UC-08 : consulter l'historique

hline **Champ**	**Description**
hline Identifiant	UC-08
Titre	Consulter l'historique et les résultats
Acteur principal	Recruteur authentifié
Objectif	Retrouver les évaluations persistées et leur détail.
Déclencheur	Le recruteur ouvre la page historique.

Préconditions	Session valide.
Postconditions	Une liste filtrée ou le détail d'un résultat possédé est affiché.
Scénario nominal	1. Le système liste les évaluations du recruteur par date décroissante. 2. Le recruteur filtre par code, offre ou e-mail. 3. Il sélectionne une ligne. 4. Le système vérifie l'appartenance et retourne le détail. 5. L'interface affiche les métadonnées et données associées.
Scénarios alternatifs	Le recruteur consulte l'historique sans filtre ou revient à la liste.
Exceptions	Identifiant absent ou résultat d'un autre recruteur : accès refusé ; erreur réseau : état d'erreur affiché.
Règles associées	RG-01, RG-09.
hline	

2.9 Matrice de traçabilité

La matrice relie les besoins aux cas d'utilisation et à des groupes de routes ou composants observés. Elle ne remplace pas une documentation API exhaustive ; elle constitue une preuve de correspondance entre l'analyse et la version implémentée.

Le tableau 2.13 en présente le contenu.

Tableau 2.13: Matrice de traçabilité des besoins

Besoin	**Cas d'utilisation**	**Composant ou route vérifiée**	**Critère de validation**
BF-AUTH-01–03	UC-01	`/auth/register`, `/auth/login`, `/auth/me`, client API	Compte créé, jeton reçu, profil chargé et jeton effacé à la déconnexion.
BF-JOB-01–04	UC-02	`/ui/job-offers`, page `/jobs`	Création codée, mise à jour, statuts et garde 409 vérifiables.
BF-CAND-01–03	UC-03	`/ui/candidates`, page `/candidates`	Liste propriétaire, édition de pipeline et suppression conditionnelle.
BF-EVAL-01–03	UC-04	`/ui/evaluate`, `/ui/evaluate/file`, `evaluate_and_persist`	Résultat persistant et explication après texte ou fichier valide.

Besoin	**Cas d'utilisation**	**Composant ou route vérifiée**	**Critère de validation**
BF-LLM-01	UC-05	`/ui/evaluate/deep-analysis`, `llm_reasoning`	États LLM rendus sans changement du verdict.
BF-REPORT-01	UC-06	`/report/pdf`, client `reportPdf`	Réponse PDF téléchargeable depuis le résultat.
BF-RANK-01–02	UC-07	page `/leaderboard`, `evaluateFile`	Tri décroissant des résultats et CSV côté client.
BF-HIST-01	UC-08	`/ui/matches`, `/ui/matches/{id}`, page `/history`	Liste filtrée et détail soumis à l'appartenance.
BF-QUALITY-01–02	UC global	`/ui/compare`, `/ui/quality`	Comparaison sans persistance et benchmark file-backed distingués.
BF-SETTINGS-01	UC global	`decision-config.tsx`, page `/settings`	Seuils validés, envoyés aux nouvelles évaluations et mémorisés avec elles.

2.10 Contraintes et hypothèses

Le tableau 2.14 rassemble ces contraintes et hypothèses ainsi que leur effet sur l'analyse.

Nature	**Contrainte ou hypothèse**	**Effet sur l'analyse**
Fonctionnelle	Aucun portail candidat ni décision autonome d'embauche.	Le recruteur reste l'acteur opérationnel et décideur final.
Fonctionnelle	Les paramètres de seuil sont locaux à la session du navigateur.	Ils ne constituent pas une politique serveur globale et n'écrasent pas l'historique.
Technique	Textes de CV et d'offres possibles en français ou anglais.	La qualité dépend du contenu extrait et des modèles disponibles.
Technique	PDF/DOCX seulement ; extraction parfois impossible ou incomplète.	Les fichiers non lisibles sont rejetés avant évaluation.
Technique	Ollama local, CPU/RAM limités et latence notable.	La Deep Analysis est optionnelle, séparée du verdict et susceptible d'indisponibilité.
Évaluation	Benchmark synthétique, sans résultats réels d'embauche.	Les indicateurs hors ligne ne prouvent pas une performance RH réelle.
Déploiement	VM Azure PFE, HTTP, instance unique et pas de haute disponibilité.	La solution est démontrable mais non qualifiée comme hébergement permanent.
Historique	Discontinuité de score pour des résultats antérieurs à une correction de pondérations.	La comparaison temporelle doit tenir compte de la méthode et des métadonnées conservées.

Tableau 2.14: Contraintes et hypothèses spécifiques à JobTest

Conclusion

Ce chapitre a fixé les limites de JobTest, identifié le recruteur comme acteur fonctionnel principal et distingué le candidat comme entité métier. Les besoins couvrent le parcours implémenté, depuis la session et les données de recrutement jusqu'à l'évaluation explicable, son historique et ses compléments facultatifs. Les règles de gestion maintiennent la propriété des données, la cohérence de l'historique et la séparation stricte entre décision statistique et analyse LLM. Les cas d'utilisation et la matrice de traçabilité constituent la base de *Chapitre 3 — Conception fonctionnelle et architecturale*, qui ne sera entamé qu'après validation de ce chapitre.

Chapter 3

Conception fonctionnelle et architecturale

Plan

Introduction 39

1 Démarche de conception 39

2 Vue fonctionnelle globale 39

3 Architecture logique de JobTest 39

4 Architecture applicative et responsabilités 41

5 Conception statique – diagramme de classes global 41

6 Conception dynamique – diagrammes de séquence 42

7 Conception des données 46

8 Principes de conception retenus 47

Conclusion 48

Introduction

Le chapitre précédent a défini le périmètre de JobTest et les besoins du recruteur. Le présent chapitre traduit ces besoins en une conception vérifiable dans l'application : responsabilités, données, interactions et déploiement logique. La conception sépare volontairement le calcul déterministe de correspondance de l'analyse générative. Ainsi, le recruteur conserve la décision finale ; le système restitue un résultat de correspondance et des éléments d'interprétation, sans automatiser l'embauche.

3.1 Démarche de conception

Dans le cycle en cascade retenu, la conception établit le passage des exigences du chapitre 2 vers des composants implémentables, avant leur réalisation et leur validation. Elle fixe les responsabilités et les échanges qui doivent demeurer stables lorsque l'interface ou l'infrastructure évoluent.

La notation UML 2.5.1 est employée de manière ciblée : le diagramme de classes représente les objets persistants et leurs liens, tandis que les diagrammes de séquence montrent l'ordre des appels et les alternatives importantes. La spécification UML de l'OMG sert de référence de notation [16]. Le diagramme global de cas d'utilisation reste celui de la figure 2.1; il n'est donc pas reproduit ici.

3.2 Vue fonctionnelle globale

La vue complémentaire de la figure 3.1 organise les fonctions par enchaînement plutôt que par acteur. L'authentification ouvre une session de recruteur. Celui-ci gère ses offres et ses candidats, lance l'évaluation puis consulte un résultat persistant. Le résultat constitue le point d'entrée du rapport PDF, de l'historique et de l'analyse approfondie. Cette dernière est une couche complémentaire : elle reçoit des informations déjà calculées et ne revient jamais modifier le score ni le verdict déterministe.

3.3 Architecture logique de JobTest

La solution déployée est un monolithe modulaire conteneurisé, et non un ensemble de microservices métier autonomes. Le navigateur ne joint publiquement que Nginx sur le port 80. Nginx relaie le préfixe `/api/v1/` vers FastAPI et les autres chemins vers Next.js. Les ports 3000, 8000, 5432 et 11434 sont internes au réseau Docker : ils ne sont pas publiés sur Internet.

La figure 3.2 représente cette organisation ainsi que les frontières réseau qui la délimitent.

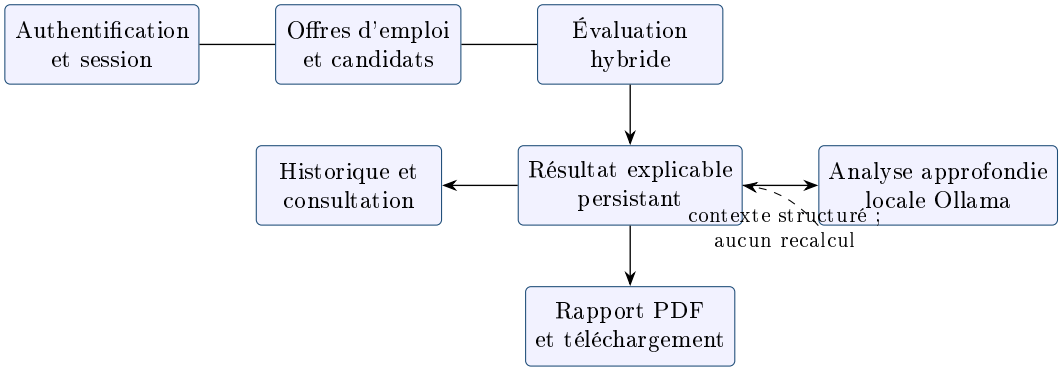


Figure 3.1: Vue de synthèse fonctionnelle de JobTest

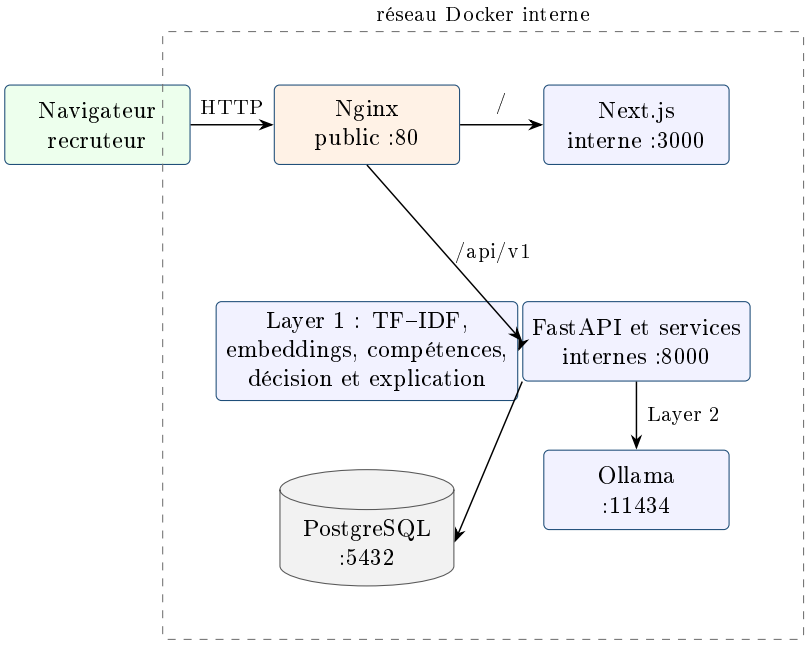


Figure 3.2: Architecture logique et frontières réseau de JobTest

La couche de présentation est une application Next.js. Son client API conserve le jeton bearer dans le stockage local du navigateur et utilise le même origin `/api/v1`. La couche FastAPI porte l'authentification, les routes HTTP, les contrôles d'appartenance et l'orchestration des services. Elle appelle en processus la Layer 1, composée des modèles TF–IDF, embeddings et compétences, du moteur de décision et du générateur d'explications.

La Layer 2 appelle Ollama à l'intérieur du réseau Docker avec un contexte structuré issu de l'évaluation déjà persistée. PostgreSQL est la persistance du déploiement; l'application conserve néanmoins SQLite comme valeur par défaut de développement à travers `DATABASE_URL`. Le chapitre 6 détaillera les choix d'exploitation Azure, sans les confondre avec cette vue logique.

3.4 Architecture applicative et responsabilités

Le tableau 3.1 relie les responsabilités de conception aux modules réellement présents. Les routes `/ui` sont un routeur de parité ajoutant une façade HTTP pour l'interface Next.js : elles réutilisent les mêmes services de calcul, modèles SQLAlchemy et moteur de décision.

Tableau 3.1: Modules applicatifs et responsabilités

Couche ou module	**Éléments vérifiés**	**Responsabilité**
Interface utilisateur	`web/app`, composants d'analyse, `web/lib/api/client.ts`	Écrans Next.js, saisie des offres et CV, restitution des résultats, appel des routes et téléchargement du PDF.
Exposition HTTP	`src/api/routers/auth.py`, `src/api/routers/ui.py`, `src/api/main.py`	JWT, validation Pydantic, contrôle du recruteur connecté, routes métiers et transformation des réponses JSON.
Services métier	`src/matching/service.py`, `src/decisioning/decision_engine.py`	Orchestration de l'évaluation, décision canonique, persistance atomique et métadonnées de reproductibilité.
Extraction et modèles	`src/parsing/parser.py`, `src/models`, `src/fusion/hybrid_scorer.py`	Extraction de texte PDF/DOCX, scores TF–IDF, sémantique et compétences, puis agrégation hybride.
Explicabilité	`src/explainability/explainer.py`	Composantes de score, analyse des écarts, compétences et recommandations cohérentes avec le résultat déterministe.
Analyse approfondie	`src/ai/llm_reasoning.py`, `DeepAnalysis.tsx`	Construction du contexte, appel Ollama local, validation JSON, contrôle de cohérence et restitution d'un état contrôlé.
Rapport PDF	`src/reporting/pdf_generator.py`, `/report/pdf`	Composition ReportLab du rapport à partir du résultat, de l'explication et des recommandations déjà calculés.
Persistance	`src/models_db.py`, `src/database.py`	Modèles SQLAlchemy, relations, transactions et adaptation SQLite/PostgreSQL par configuration.

3.5 Conception statique – diagramme de classes global

La figure 3.3 privilégie les concepts métier et les attributs nécessaires à la compréhension. Les noms entre parenthèses correspondent aux classes techniques SQLAlchemy. Une analyse LLM n'est pas

une table distincte : son enveloppe JSON est conservée dans `MatchResult.llm_analysis_json` uniquement lorsqu'elle est valide. De même, le rapport PDF est généré à la demande et ne possède pas d'entité persistée propre.

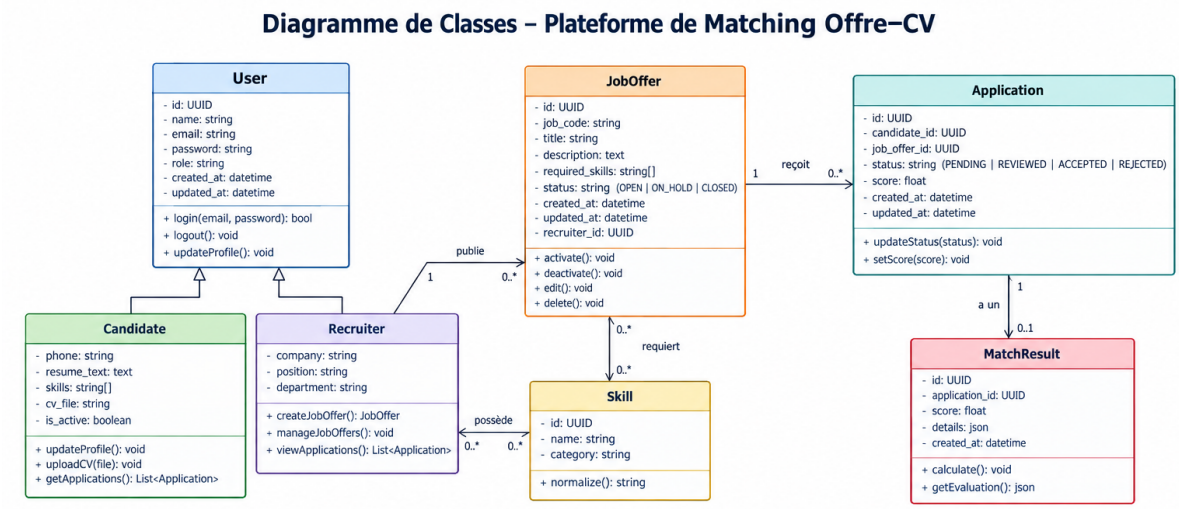


Tableau des classes

Objet (Classe)	Rôle / Entité	Attributs principaux	Relations
User	Utilisateur du système (parent)	id, name, email, password, role, created_at, updated_at	Généralise → Candidate, Recruiter
Candidate	Candidat qui postule aux offres	id, phone, resume_text, skills, cv_file, is_active	1 — * Application | * — * Skill
Recruiter	Recruteur qui publie des offres	id, company, position, department	1 — * JobOffer
JobOffer	Offre d'emploi	id, job_code, title, description, required_skills, status, created_at, updated_at, recruiter_id	1 — * Application | * — * Skill
Application	Candidature d'un candidat sur une offre	id, candidate_id, job_offer_id, status, score, created_at, updated_at	* — 1 Candidate | * — 1 JobOffer |1 — 0..1 MatchResult
MatchResult	Résultat de matching (similarité)	id, application_id, score, details, created_at	1 — 0..1 Application
Skill	Compétence (métier / technique)	id, name, category	* — * Candidate | * — * JobOffer

Figure 3.3: Diagramme de classes métier simplifié de JobTest

Un utilisateur recruteur possède des offres et des candidats. Un candidat peut regrouper plusieurs CV et plusieurs résultats de correspondance; chaque résultat se rattache au CV et à l'offre qui ont servi à son calcul. Les compétences sont normalisées dans `Skill` et reliées aux CV et aux offres par les tables d'association `cv_skills` et `job_skills`. Les chemins actuels d'évaluation conservent également le texte et les informations d'explication dans le résultat pour en permettre la consultation ultérieure.

3.6 Conception dynamique – diagrammes de séquence

3.6.1 Authentification du recruteur

La connexion utilise le formulaire OAuth2 de FastAPI. Après vérification du mot de passe haché, l'API émet un JWT bearer. L'interface le stocke localement puis appelle `/auth/me` afin de charger le profil de session.

La figure 3.4 détaille cet échange, depuis la soumission du formulaire jusqu'au chargement du profil.

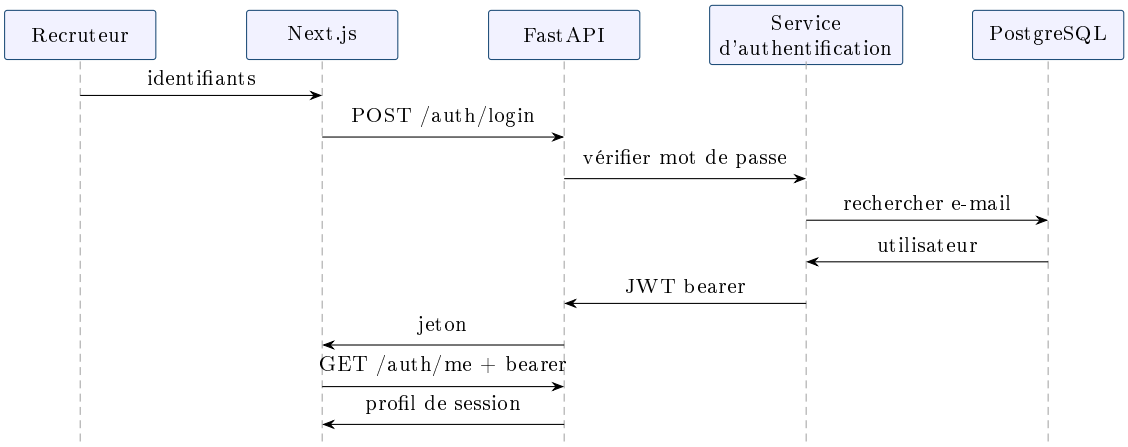


Figure 3.4: Séquence d'authentification du recruteur

3.6.2 Création d'une offre d'emploi

L'écran des offres transmet le titre, la description, les compétences requises et l'état demandé. La route `/ui/job-offers` vérifie la présence du titre et de la description, associe l'offre au recruteur connecté et construit le code `JOB-AAAA-NNN` avant l'écriture en base.

La figure 3.5 en retrace le déroulement.

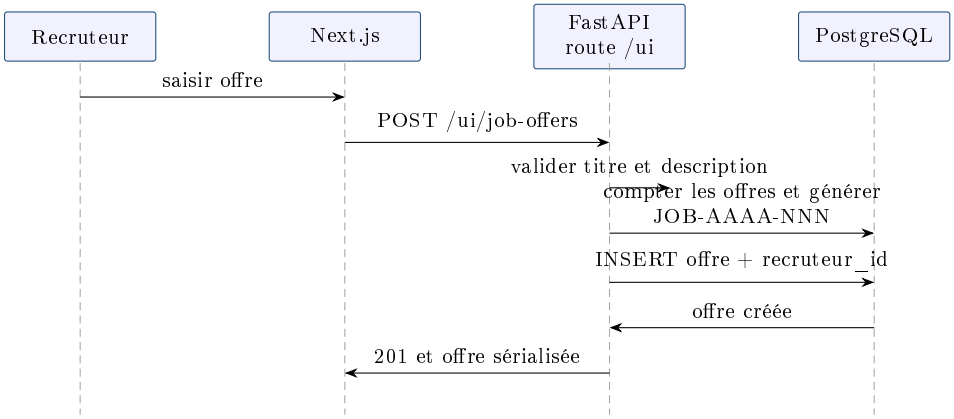


Figure 3.5: Séquence de création d'une offre d'emploi

3.6.3 Évaluation d'un CV par rapport à une offre

La figure 3.6 représente le chemin canonique. Le CV est fourni comme texte ou par téléversement; dans ce dernier cas, le routeur extrait le texte du PDF ou du DOCX avant l'évaluation. L'offre doit appartenir au recruteur. Le service exécute les trois composantes, les agrège, applique les seuils et

persiste un `MatchResult`.

Pour le modèle hybride par défaut, les scores élémentaires sont compris entre 0 et 1 et la formule effective est :

$$S_h = 0{,}2S_{\text{TF-IDF}} + 0{,}4S_{\text{emb}} + 0{,}4S_{\text{comp}}.$$

Le pourcentage affiché vaut $100S_h$. Avec la configuration par défaut, le `DecisionEngine` produit `HIRE` si le score est supérieur ou égal à 75, `CONSIDER` s'il est compris entre 55 et strictement inférieur à 75, et `REJECT` en dessous de 55. Les seuils de session peuvent être transmis par l'interface, mais la configuration utilisée est enregistrée avec le résultat. Ce verdict est une recommandation algorithmique et non une décision de recrutement.

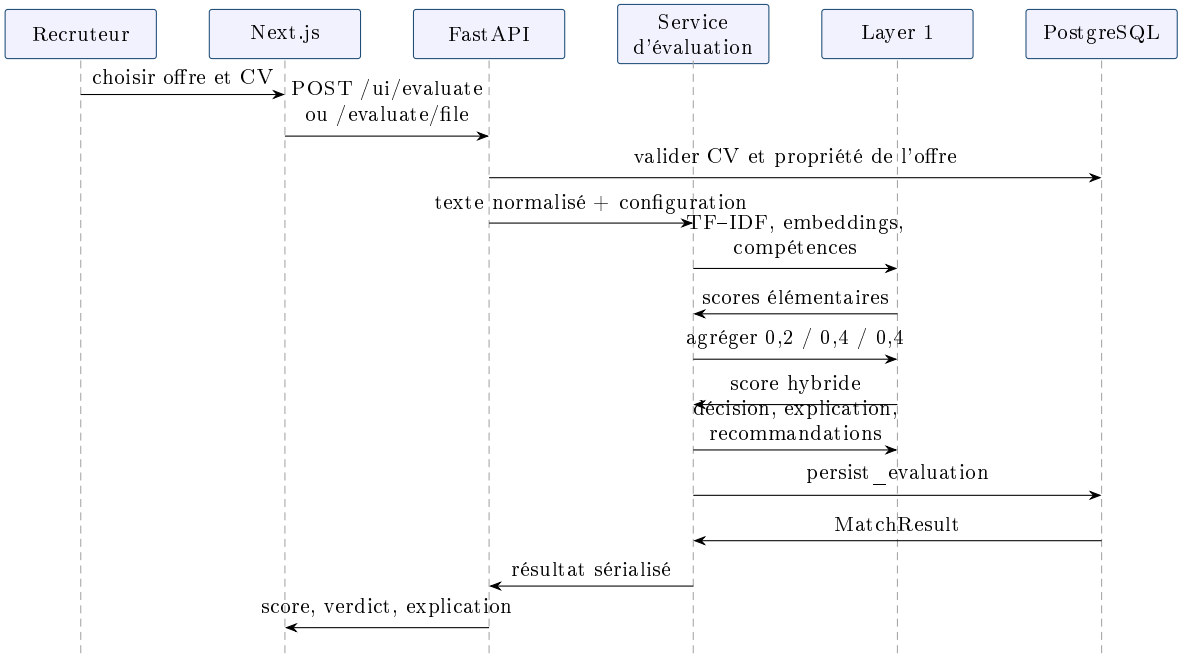


Figure 3.6: Séquence centrale d'évaluation et de persistance d'un CV

3.6.4 Analyse approfondie avec Ollama

Après le rendu de l'évaluation, le composant `DeepAnalysis` déclenche automatiquement un appel séparé pour l'identifiant du résultat; il peut aussi être relancé depuis l'interface. L'API reconstruit le contexte à partir du `MatchResult` sans recalculer la Layer 1. Ollama reçoit ce contexte structuré et doit fournir un JSON conforme. Une analyse réussie est ajoutée au résultat; sinon l'interface reçoit un état `disabled`, `unavailable`, `error` ou `inconsistent`. Dans tous les cas, le score et la décision de la Layer 1 restent inchangés.

L'enchaînement correspondant apparaît sur la figure 3.7, où l'on voit que l'appel part d'un résultat déjà enregistré.

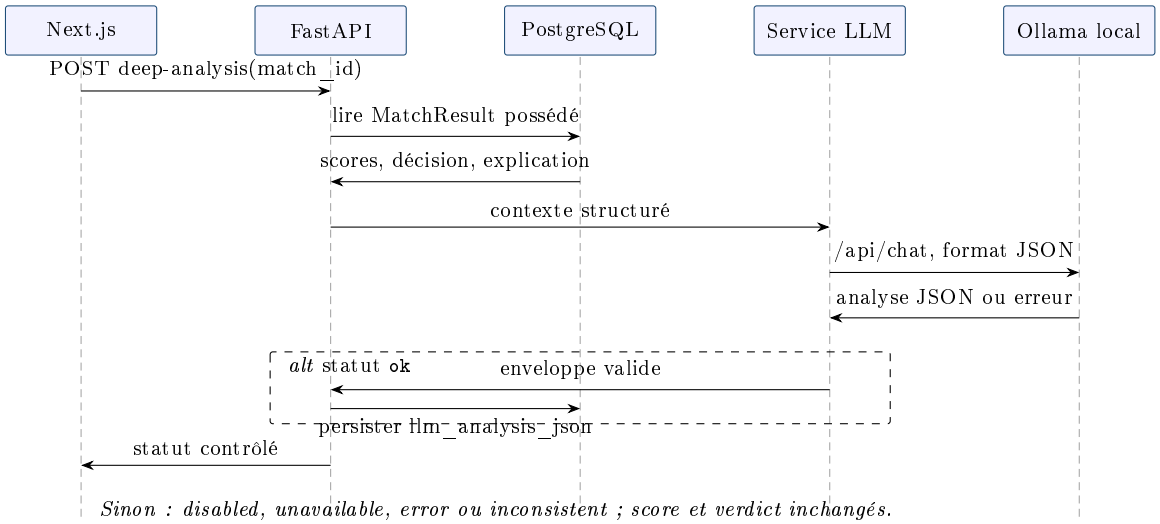


Figure 3.7: Séquence d'analyse approfondie, additive à la Layer 1

3.6.5 Génération du rapport PDF

Le téléchargement demande au serveur un document à partir du résultat, de l'explication et des recommandations déjà affichés. FastAPI délègue la mise en page au générateur ReportLab et retourne les octets PDF avec une en-tête de pièce jointe. Cette opération ne crée pas de nouvelle évaluation et ne modifie pas le résultat de correspondance.

La figure 3.8 montre la séquence suivie jusqu'au téléchargement du document.

3.7 Conception des données

Le modèle relationnel est porté par SQLAlchemy. `users.email`, `skills.name` et `job_offers.job_code` sont déclarés uniques. Les clés étrangères relient candidats et offres au recruteur, les CV aux

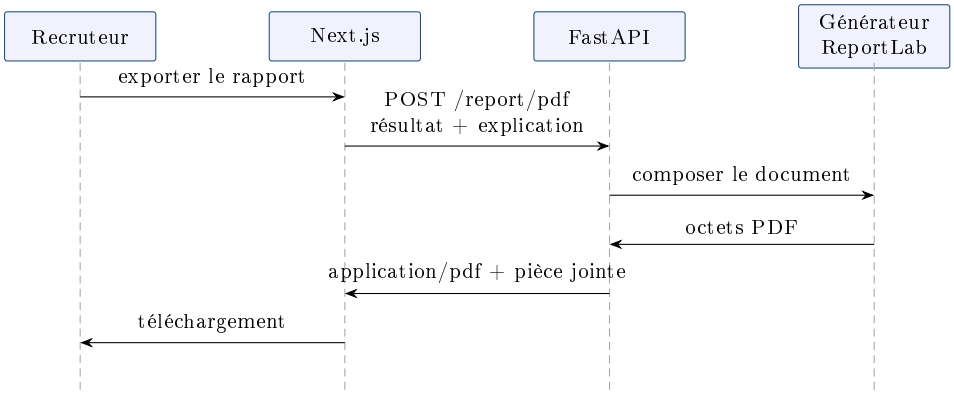


Figure 3.8: Séquence de génération et téléchargement d'un rapport PDF

candidats, puis les résultats à leur candidat, CV et offre. Les suppressions en cascade sont configurées entre candidat et CV, ainsi qu'entre candidat ou offre et résultats de correspondance.

Le résultat de correspondance est l'élément de traçabilité principal : il conserve les scores hybride, sémantique, lexical et compétences, la décision, les seuils utilisés, la version du modèle, la méthode et les poids. L'explication et les recommandations sont sérialisées dans `explanation_json`; l'analyse approfondie valide est ajoutée dans `llm_analysis_json`. Les comparaisons de modèles et le benchmark de qualité ne créent pas de résultats persistants.

En développement, la valeur par défaut de `DATABASE_URL` pointe vers SQLite et l'application active les clés étrangères SQLite. Dans le déploiement conteneurisé, cette variable cible PostgreSQL 16. Les mêmes modèles SQLAlchemy sont employés; la différence porte sur le moteur de base de données, non sur le modèle métier présenté.

3.8 Principes de conception retenus

- **Séparation des responsabilités :** Next.js présente et collecte, FastAPI orchestre, les services calculent, SQLAlchemy persiste et Ollama interprète qualitativement.
- **Traçabilité et reproductibilité :** le résultat conserve les composantes, poids, version, méthode et seuils actifs lors de son calcul.
- **Isolation :** la pile Docker place PostgreSQL, FastAPI, Next.js et Ollama sur un réseau interne; seul Nginx expose le port 80.
- **Dégradation contrôlée :** l'indisponibilité, l'erreur ou l'incohérence de la Layer 2 est représentée par un état explicite, sans bloquer l'évaluation déterministe.

- **Responsabilité humaine :** la décision `HIRE`/`CONSIDER`/`REJECT` facilite l'examen d'un dossier; le recruteur demeure responsable de la décision finale.

Conclusion

La conception de JobTest articule une interface Next.js, une API FastAPI, une persistance relationnelle et deux couches d'intelligence aux rôles distincts. Les diagrammes ont montré la conservation d'un résultat traçable, depuis l'authentification jusqu'au rapport et à l'analyse approfondie. Le chapitre suivant détaillera la conception et l'évaluation du moteur intelligent, notamment ses composantes de correspondance et ses résultats de validation.

Chapter 4

Conception et évaluation du moteur intelligent

Plan

Introduction 50

1 Objectifs et principes de conception 50

2 Prétraitement et structuration des documents 50

3 Similarité lexicale TF–IDF 51

4 Similarité sémantique par embeddings 51

5 Rapprochement structuré des compétences 51

6 Fusion hybride et décision 52

7 Explicabilité et couche d'analyse locale 52

8 Jeu de données et protocole expérimental 54

9 Résultats et analyse comparative 55

10 Corrections du moteur et du protocole en cours de projet 60

11 Limites, risques et perspectives 62

Conclusion 63

Introduction

Le moteur de rapprochement de JobTest relie un CV et une offre à une recommandation exploitable par un recruteur. Il ne prédit ni une embauche ni une décision autonome : il mesure une compatibilité textuelle et métier, puis applique des règles explicites. Ce chapitre décrit les composants réellement implémentés et leur évaluation reproductible sur le corpus synthétique du projet.

4.1 Objectifs et principes de conception

Le moteur doit accepter des CV hétérogènes, confronter vocabulaire et sens général, signaler les compétences manquantes et rendre le résultat vérifiable. Le choix retenu est une fusion tardive : chaque composant conserve son score avant agrégation. Deux couches sont séparées. La couche 1 produit le score, la décision et les justifications déterministes. La couche 2, optionnelle, enrichit seulement l'affichage par une analyse locale ; elle ne modifie ni score ni décision.

La figure 4.1 résume cet enchaînement et la place qu'y occupe la couche optionnelle.

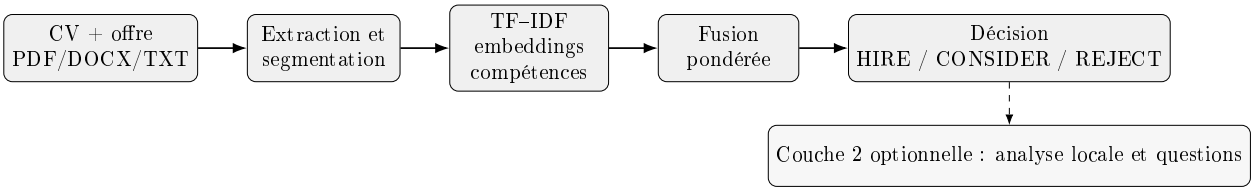


Figure 4.1: Pipeline de rapprochement mis en œuvre dans JobTest. La couche 2 est additive et non décisionnelle.

4.2 Prétraitement et structuration des documents

L'extraction privilégie `pdfplumber` pour les PDF et bascule sur `PyPDF2` en cas d'indisponibilité ; les fichiers DOCX sont lus avec `python-docx`, avec un repli XML. Le texte est nettoyé des URL, courriels, numéros de téléphone, retours superflus et artefacts de césure. La normalisation disponible applique, lorsque les modèles spaCy sont installés, tokenisation, lemmatisation et filtrage des mots-outils ; un repli par expressions régulières et lemmatisation légère maintient le traitement sinon.

Le CV est ensuite segmenté par intitulés français et anglais : profil, expérience, compétences, formation, langues, certifications et projets. Cette structure n'est pas une interprétation humaine du dossier ; elle permet notamment de pondérer l'expérience distinctement des compétences.

4.3 Similarité lexicale TF–IDF

Le premier composant vectorise CV et offre avec `TfidfVectorizer`. La configuration implémentée utilise des n-grammes de 1 à 3 termes, au plus 10 000 caractéristiques et une fréquence de terme sous-linéaire. TF–IDF valorise les termes discriminants ; la similarité cosinus compare les vecteurs issus des deux textes [17], [18] :

$$\mathrm{tfidf}(t,d) = \mathrm{tf}(t,d)\log\left(\frac{N+1}{\mathrm{df}(t)+1}\right)+1, \qquad S_{\mathrm{TFIDF}} = \frac{\vec{v}_{CV}\cdot\vec{v}_{offre}}{\|\vec{v}_{CV}\|\|\vec{v}_{offre}\|}.$$

Cette composante est lisible lorsque les mêmes technologies sont citées, mais elle reste sensible aux synonymes et aux reformulations.

4.4 Similarité sémantique par embeddings

Le deuxième composant encode les textes avec Sentence-Transformers lorsqu'il est disponible. Le modèle anglais configuré par défaut est `all-mpnet-base-v2`. Le modèle multilingue est identifié dans la configuration par :

`paraphrase-multilingual-mpnet-base-v2`

`HashingVectorizer` constitue le repli local en cas de chargement impossible. Les représentations de phrases et leur comparaison cosinus s'appuient sur le principe de Sentence-BERT [6].

Les similitudes anglaise et multilingue sont moyennées. Une similitude par sections reçoit les poids expérience 0,40 ; compétences 0,30 ; résumé 0,15 ; formation 0,10 ; langues 0,05. Le composant retourne donc :

$$S_{\mathrm{emb}} = 0{,}7\,(0{,}5s_{en} + 0{,}5s_{multi}) + 0{,}3s_{sections}.$$

La pondération rend l'expérience et les compétences plus influentes sans prétendre représenter une adéquation humaine complète.

4.5 Rapprochement structuré des compétences

Le troisième composant exploite le dictionnaire versionné du projet. Il détecte les compétences techniques et comportementales, les certifications, les variantes et des n-grammes normalisés. Les termes de nécessité, tels que *required*, *must* ou *obligatoire*, signalent des compétences critiques.

Soient H et S les recouvrements techniques et comportementaux. La base répartit les poids

0,70 et 0,30 entre les catégories présentes. Elle reçoit un bonus de domaine de 0,05, un bonus de certification plafonné à 0,10, puis une pénalité plafonnée à 0,20 pour les compétences critiques absentes :

$$S_{\text{skill}} = \text{clip}\left(S_{base} + 0{,}05 I_{domaine} + \min(0{,}10, 0{,}03 n_{cert}) - 0{,}20 \frac{n_{critiques\ absentes}}{n_{critiques}}, 0, 1\right).$$

Il restitue aussi les compétences trouvées, manquantes et critiques : ces éléments constituent la justification métier principale.

4.6 Fusion hybride et décision

La configuration documentée n'apprend pas les poids de fusion ; elle les fixe ainsi :

$$S_{hybride} = 0{,}2 S_{\text{TFIDF}} + 0{,}4 S_{\text{emb}} + 0{,}4 S_{\text{skill}}.$$

Elle donne le même poids à la proximité sémantique et à la couverture explicite des compétences. Pour l'exemple didactique $(0{,}60, 0{,}70, 0{,}80)$, le score est 0,72, soit 72,0 %.

Le `DecisionEngine` applique ensuite les règles du Tableau 4.1. Elles sont distinctes des seuils expérimentaux calibrés sur la validation lors du benchmark.

Tableau 4.1: Seuils de recommandation de la couche 1

Intervalle	Recommandation	Interprétation opérationnelle
$[75, 100]$	`HIRE`	adéquation forte à examiner en priorité
$[55, 75[$	`CONSIDER`	profil à examiner avec réserves explicites
$[0, 55[$	`REJECT`	adéquation insuffisante selon les signaux disponibles

La confiance présentée par le moteur est une distance technique à la frontière de seuil la plus proche. Elle n'est pas une probabilité calibrée d'embauche.

4.7 Explicabilité et couche d'analyse locale

La couche 1 conserve les sous-scores, poids, décision, compétences trouvées ou absentes, compétences critiques et expérience détectée. L'interface peut donc formuler une justification contrôlable, par exemple l'absence de Docker et Kubernetes parmi les prérequis.

Après ce résultat, l'interface peut demander une analyse approfondie. Le service transmet alors le texte source et ce contexte déterministe à un modèle local configuré par défaut comme

`qwen2.5:1.5b`, exécuté via Ollama. La réponse structurée propose trajectoire, alertes, justification, recommandations et questions d'entretien. Les états `disabled`, `unavailable`, `error` et `inconsistent` sont contrôlés sans bloquer la couche 1 ; une incohérence générative ne révise donc jamais la recommandation.

4.7.1 Construction du contexte et format de réponse

L'appel au modèle local n'est pas une requête libre. Le service assemble un contexte à partir du résultat déjà persistant : pourcentage global, sous-scores des trois composantes, décision retenue et libellé de sa bande, compétences trouvées, compétences absentes et compétences critiques manquantes. Le modèle reçoit donc le verdict comme une donnée d'entrée à commenter, et non comme une question à trancher. Le texte du CV et celui de l'offre accompagnent ce contexte pour que l'analyse puisse s'appuyer sur le contenu réel du dossier.

La réponse attendue est un objet structuré comportant cinq parties : une lecture de la trajectoire professionnelle, les points d'attention, une justification du positionnement, des recommandations et des questions d'entretien. Cette contrainte de format sert deux objectifs. Elle rend la réponse exploitable par l'interface sans analyse de texte libre, et elle interdit au modèle de produire un score concurrent : aucun champ ne prévoit de note chiffrée ni de verdict. Une réponse qui ne se conforme pas au schéma attendu est rejetée plutôt que corrigée, et l'état correspondant est remonté à l'interface.

4.7.2 Garde-fou de cohérence et tentative d'injection

La contrainte de format ne suffit pas. Un petit modèle exécuté sur processeur peut respecter la structure demandée tout en énonçant une appréciation qui contredit la décision statistique : décrire comme un excellent profil un dossier que le moteur a classé en `REJECT` avec des compétences critiques absentes. Une telle réponse n'est pas seulement inexacte, elle est trompeuse, puisqu'elle apparaît à côté du verdict qu'elle dément.

Le service applique donc un contrôle de cohérence sur les cas de faible adéquation. Le texte produit est examiné à la recherche d'affirmations positives incompatibles avec le verdict, en tenant compte des négations pour éviter de sanctionner une formulation telle que « ce n'est pas un profil adapté ». Lorsqu'une contradiction est détectée, le service relance une fois le modèle avec un rappel explicite du verdict à respecter. Si la seconde réponse contredit encore la décision, l'analyse est abandonnée et l'état `inconsistent` est retourné : le système préfère ne rien afficher plutôt qu'afficher

un commentaire qui affaiblirait la recommandation. C'est un comportement de défaillance fermée, cohérent avec le principe selon lequel la couche 2 ne peut jamais réviser la couche 1.

Ce garde-fou joue également un rôle de sécurité. Le texte d'un CV est fourni par un tiers et se retrouve intégré au message envoyé au modèle ; un candidat averti pourrait y insérer des instructions destinées à l'influencer, par exemple une consigne affirmant qu'il s'agit d'un profil parfait et qu'aucune réserve ne doit être émise. L'audit de sécurité du projet a retenu ce scénario comme un risque réel. Deux propriétés le contiennent. D'une part, le score et la décision sont calculés par le moteur déterministe avant tout appel au modèle : une injection réussie ne peut pas modifier le verdict, seulement le commentaire qui l'accompagne. D'autre part, c'est précisément sur les dossiers faiblement classés qu'une injection aurait un intérêt, et c'est là que le contrôle de cohérence est actif : un texte trop favorable sur un dossier rejeté conduit au rejet de l'analyse. La protection n'est pas absolue, mais la surface exposée se limite à du texte consultatif, jamais à la recommandation elle-même.

4.8 Jeu de données et protocole expérimental

L'évaluation utilise `cv_job_dataset.csv`, généré localement avec la graine 42. Il contient 12 000 paires CV–offre synthétiques, plusieurs domaines et niveaux d'expérience, des textes français et anglais, des abréviations, fautes de frappe, compétences obsolètes et offres contradictoires. Les étiquettes positives regroupent les cas de forte et de transférable adéquation générés ; elles ne sont pas des décisions de recrutement observées.

La séparation est stratifiée par étiquette et domaine, puis vérifiée par groupe de paire CV–offre normalisée : 8 400 paires pour l'apprentissage, 1 800 pour la validation et 1 800 pour le test final. Les seuils de chaque méthode sont sélectionnés sur validation en maximisant le F1, puis les métriques sont calculées sur le test isolé. Cette précaution réduit la fuite de paires identiques, sans remplacer une validation externe réelle.

Les métriques sont l'exactitude, la précision, le rappel, le F1, l'AUC-ROC, l'aire précision–rappel, le kappa de Cohen et la matrice de confusion, selon les définitions de scikit-learn [19]. Le F1 équilibre précision et rappel :

$$F1 = 2\frac{\text{precision} \times \text{rappel}}{\text{precision} + \text{rappel}}.$$

4.9 Résultats et analyse comparative

Le Tableau 4.2 reproduit les résultats enregistrés sur les 1 800 paires de test. Les seuils affichés sont des points de fonctionnement expérimentaux et non les seuils `HIRE`/`CONSIDER`/`REJECT` de l'interface.

Tableau 4.2: Comparaison sur le jeu de test isolé

Méthode	Seuil	Exact.	Préc.	Rappel	F1	AUC-ROC
TF–IDF	0,04	0,93	0,93	0,97	0,95	0,98
Embeddings	0,54	0,78	0,81	0,89	0,85	0,81
Compétences	0,00	0,89	0,89	0,97	0,93	0,95
Hybride	0,26	0,91	0,94	0,93	0,93	0,96

La figure 4.2 met ces deux indicateurs en regard pour les quatre méthodes.

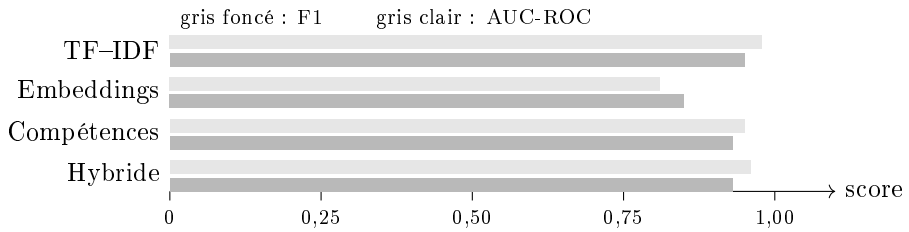


Figure 4.2: F1 et AUC-ROC des quatre méthodes sur le test isolé. Données : `evaluation_results.json`.

Les visualisations suivantes complètent le tableau et la comparaison synthétique. Elles illustrent les résultats obtenus sur le jeu de données expérimental ; elles ne constituent pas une validation sur des décisions de recrutement réelles.

La figure 4.3 montre comment se répartissent les erreurs de chaque méthode.

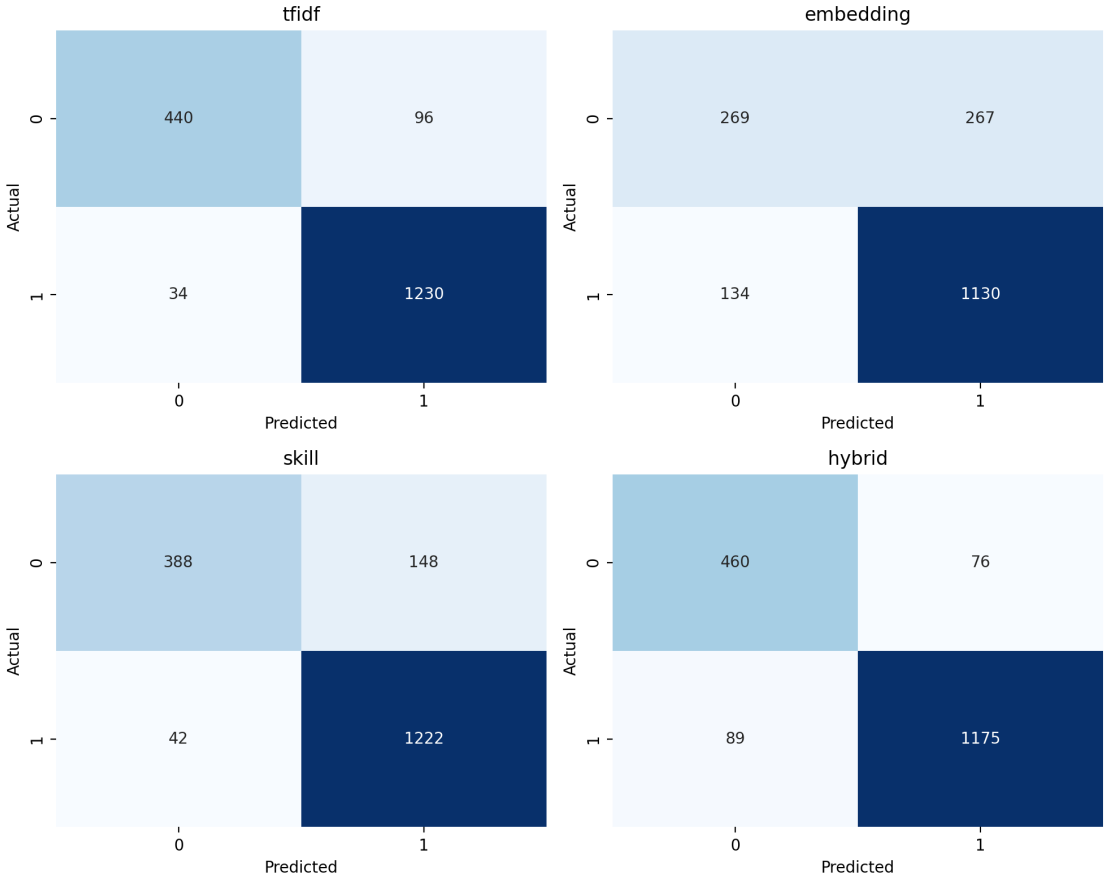


Figure 4.3: Matrices de confusion des méthodes de rapprochement sur le jeu de test

La figure 4.4 décompose les scores obtenus selon le domaine métier des paires.

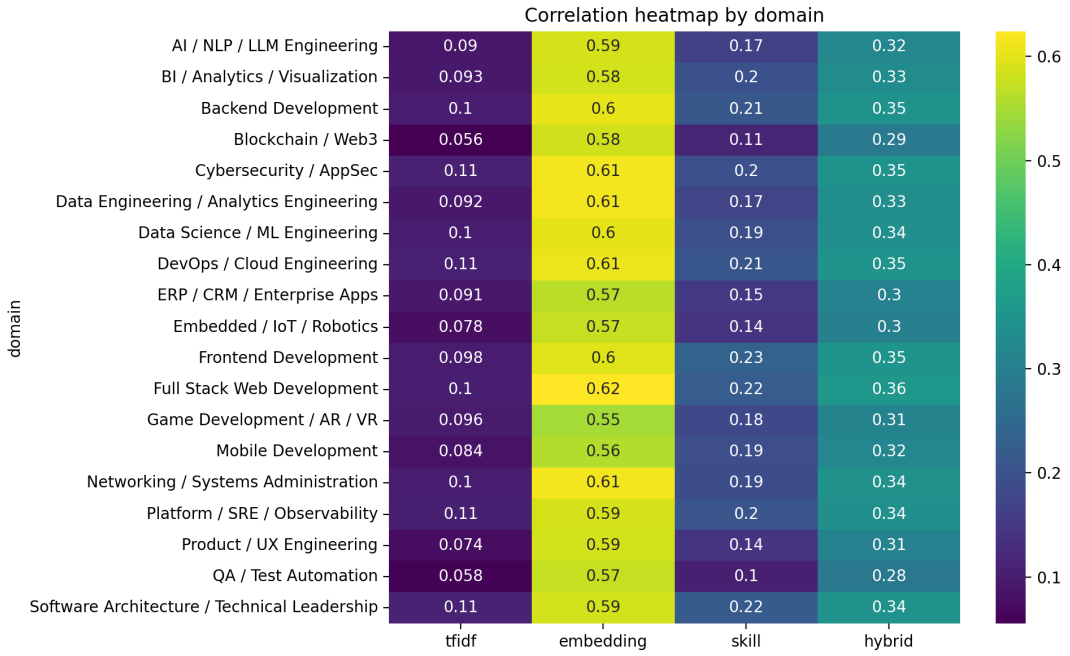


Figure 4.4: Carte thermique des scores de rapprochement par domaine

La figure 4.5 confronte directement le signal sémantique et le signal issu des compétences.

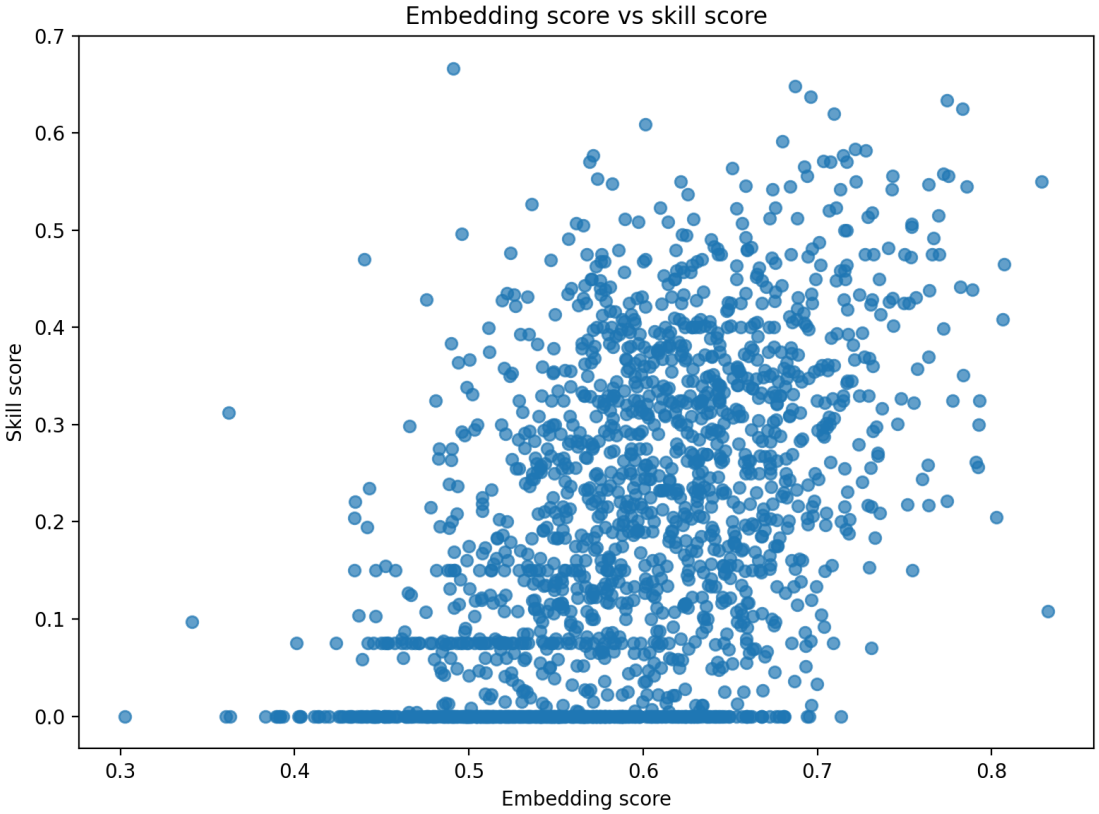


Figure 4.5: Comparaison des signaux sémantique et de compétences

La figure 4.6 reprend enfin les valeurs de F1 sous forme graphique.

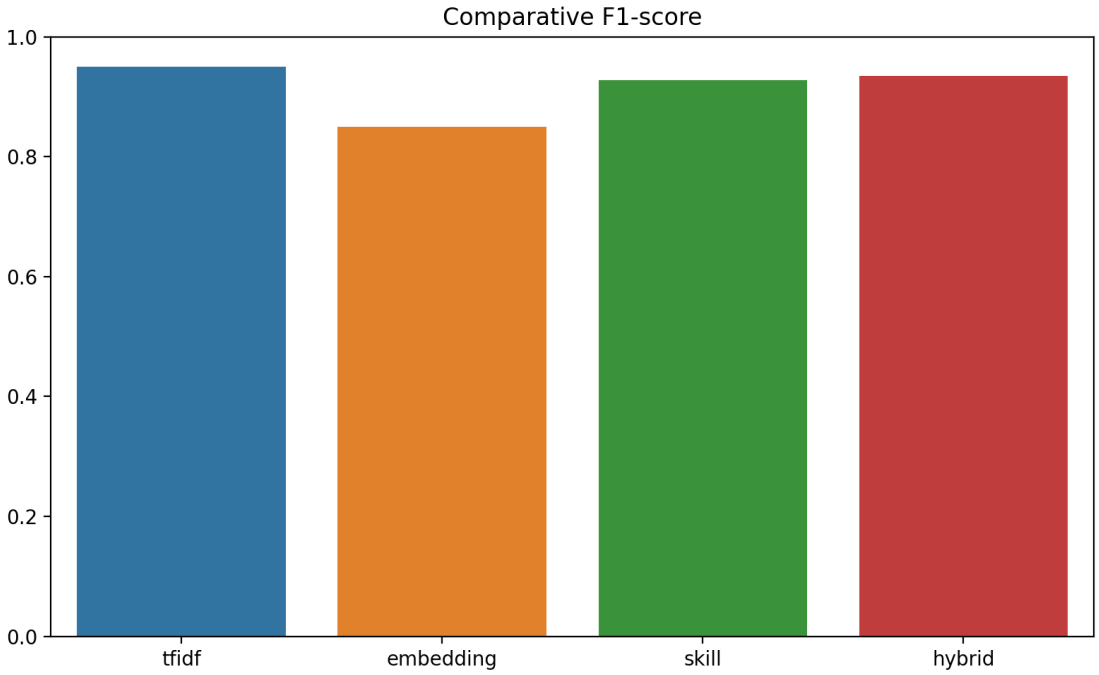


Figure 4.6: Scores F1 des méthodes évaluées

TF–IDF obtient le meilleur F1 dans ce protocole, tandis que la fusion atteint $F1 = 0{,}93$ et $AUC = 0{,}96$. Cela ne démontre pas une supériorité générale du lexical : le générateur construit CV et offres à partir d'un vocabulaire contrôlé et partagé, ce qui favorise le recouvrement des termes. La fusion est retenue pour associer ce signal à la sémantique et à une justification par compétences. Le temps moyen mesuré est 55,85 ms par paire pour la fusion, contre 921,60 ms pour les embeddings seuls dans cet environnement.

Deux lectures distinctes du même score

Les chiffres du Tableau 4.2 répondent à une question de qualité statistique : le score hybride sépare-t-il correctement les paires étiquetées positives des autres ? Pour y répondre, le seuil est choisi sur la validation de façon à maximiser le F1, ce qui place le point de fonctionnement à 0,2636 pour la fusion. Ce nombre est un instrument de mesure. Il ne décrit pas le comportement de l'application, dont la politique de décision repose sur deux bornes métier fixées indépendamment à 55 % et 75 % (Tableau 4.1). Confondre les deux conduirait à attribuer au produit une performance qui n'a pas été mesurée à ses propres bornes ; le Tableau 4.3 explicite donc l'écart plutôt que de le laisser implicite.

Tableau 4.3: Le même score hybride lu au point de calibration et aux bornes du produit (1 800 paires de test)

Lecture	Seuil	Exact.	Préc.	Rappel	F1	Positifs
Calibration (qualité du modèle)	0,2636	0,91	0,94	0,93	0,93	1 251
Politique produit – `CONSIDER`	0,55	0,31	1,00	0,02	0,04	23
Politique produit – `HIRE`	0,75	0,30	—	0,00	0,00	0

L'explication de cet écart tient à la plage effectivement occupée par le score sur ce corpus. Sur les 1 800 paires de test, la fusion produit des valeurs comprises entre 0,122 et 0,609, avec une médiane à 0,329 et un quatre-vingt-dixième centile à 0,459. La borne `CONSIDER` n'est franchie que par 23 paires et la borne `HIRE` ne l'est jamais. Autrement dit, le corpus ne peuple pas la partie haute de l'échelle : appliquer 55 % et 75 % à ces données revient à mesurer la politique métier sur une population qui ne contient presque aucun cas censé la déclencher, et le F1 alors obtenu ne renseigne ni sur le moteur ni sur le produit.

Cette compression du score n'est pas un défaut du moteur mais une propriété du jeu de données. Le générateur assemble des CV et des offres de longueur homogène à partir d'un vocabulaire contrôlé ; même une paire fortement compatible partage rarement assez de contenu pour que les trois composantes soient simultanément élevées, alors que la fusion exige précisément cette convergence pour dépasser 0,75. La précision de 1,00 relevée à 0,55 va dans le même sens : les rares paires qui atteignent cette zone sont toutes des positifs. Le signal est donc correctement ordonné, mais son amplitude est plus étroite sur des documents synthétiques que sur les CV réels observés pendant la démonstration, où une évaluation a produit un verdict `CONSIDER`.

Il faut en tirer deux conséquences honnêtes. D'abord, les bornes 55 % et 75 % sont un choix de politique de recrutement, réglable par le recruteur dans l'interface et enregistré avec chaque évaluation ; elles ne sont pas issues du benchmark et ne doivent pas être présentées comme validées par lui. Ensuite, leur calibration ne pourra être menée que sur des candidatures réelles : c'est la distribution des scores sur des dossiers authentiques, et non sur un corpus généré, qui dira si ces bornes placent la frontière au bon endroit. Tant que cette étape n'est pas faite, le Tableau 4.2 doit être lu comme une mesure de la capacité de discrimination du moteur, et non comme une prévision du taux de `HIRE` que produira la plateforme.

Sur le déploiement Azure CPU, un essai de couche 2 a produit les cinq sections attendues en 54 290,6 ms. Cette mesure ponctuelle décrit la démonstration, non une garantie de latence ou une

évaluation de qualité générative.

4.10 Corrections du moteur et du protocole en cours de projet

Les résultats présentés ci-dessus ne sont pas le produit d'un premier essai. Deux défauts ont été identifiés puis corrigés pendant le développement, l'un dans la pondération du moteur, l'autre dans la méthode d'évaluation elle-même. Les rapporter sert la reproductibilité : ils expliquent pourquoi les chiffres du dépôt ont changé au cours du projet et pourquoi les valeurs retenues sont celles de ce mémoire.

4.10.1 Sensibilité à la formulation et rétablissement des poids documentés

Le point de départ est une observation gênante faite sur le modèle de production alors en service. Ce dernier ne portait pas les poids documentés mais la combinaison `tfidf 0,9 / embedding 0,0 / skill 0,1`, héritée d'une recherche sur grille qui maximisait l'AUC sur une partition de validation. Un poids nul sur la composante sémantique signifie, concrètement, que le moteur ne disposait plus d'aucun moyen de reconnaître qu'une même idée est exprimée avec d'autres mots.

Pour mesurer l'effet réel de cette configuration, une expérience contrôlée a été construite : deux CV décrivant le même profil, avec des compétences rigoureusement identiques, ont été confrontés à la même offre. Le premier reprenait la formulation de l'annonce, le second exprimait le même contenu autrement. Les compétences obtenant le score maximal dans les deux cas, toute différence de résultat ne pouvait provenir que du vocabulaire. Le Tableau 4.4 donne les valeurs obtenues avant puis après correction.

Tableau 4.4: Effet de la reformulation à compétences identiques, avant et après le rétablissement des poids

Pondération du moteur	CV calqué sur l'offre	CV reformulé	Écart
0,9 / 0,0 / 0,1 (état initial)	99,2 %	73,6 %	25,6 points
0,2 / 0,4 / 0,4 (documentée)	94,4 %	84,8 %	9,6 points

Un écart de 25,6 points imputable à la seule rédaction du CV constitue une faille exploitable : il suffisait de recopier les termes de l'annonce pour gagner une vingtaine de points, ce qui est exactement le comportement que la présence d'une composante sémantique doit décourager. Le modèle de production a donc été réexporté avec les poids fixes documentés, sans optimisation, et son point de fonctionnement recalculé sur la validation seule, ce qui l'a fait passer de 0,04 à 0,2636.

L'écart résiduel de 9,6 points est cohérent avec ce que l'on attend d'un moteur qui conserve une part lexicale assumée : la formulation compte encore, mais elle ne domine plus la décision.

Cette correction a une conséquence qu'il serait incorrect de passer sous silence. Les évaluations enregistrées avant le 9 septembre 2026 ont été calculées avec l'ancienne pondération ; leurs scores ne sont pas sur la même échelle que ceux produits depuis. Ces enregistrements ont été conservés tels quels plutôt que recalculés, et c'est précisément pour permettre ce genre de lecture que chaque `MatchResult` conserve la méthode, les poids et les seuils employés au moment du calcul.

4.10.2 Intégrité du protocole d'évaluation

Le second défaut portait sur la mesure et non sur le moteur. La version initiale du banc d'essai appliquait un seuil unique de 0,60 à toutes les méthodes, sans tenir compte du fait que leurs scores n'occupent pas la même plage. TF–IDF et la composante compétences produisent des valeurs concentrées dans le bas de l'intervalle : jugées à 0,60, elles ne classaient presque rien en positif. Il en résultait une contradiction visible, une AUC élevée accompagnée d'un F1 quasi nul, symptôme classique d'un seuil mal placé plutôt que d'un modèle inopérant. Le Tableau 4.5 montre l'ampleur de la correction apportée par une calibration propre à chaque méthode.

Tableau 4.5: Effet de la calibration par méthode sur les métriques publiées (exactitude / F1 / AUC-ROC)

Méthode	Seuil unique 0,60	Seuil calibré sur validation
TF–IDF	29,8 % / 0,000 / 0,979	92,8 % / 0,950 / 0,979
Embeddings	51,2 % / 0,479 / 0,801	77,7 % / 0,849 / 0,811
Compétences	30,2 % / 0,011 / 0,947	89,4 % / 0,928 / 0,947
Hybride	92,8 % / 0,950 / 0,983	90,8 % / 0,934 / 0,964

La stabilité des AUC entre les deux colonnes est instructive : la capacité de classement des méthodes n'avait pas changé, seule la lecture qu'on en faisait était faussée. Le cas de la fusion mérite une remarque inverse : ses chiffres ont légèrement baissé. C'est attendu et souhaitable, car l'ancienne mesure bénéficiait de poids choisis par optimisation sur les mêmes données que celles servant à la juger, alors que la nouvelle fixe les poids avant toute évaluation.

Trois garde-fous ont été ajoutés à cette occasion. Le partitionnement regroupe désormais les paires CV–offre par empreinte du texte normalisé et vérifie par assertion que les trois sous-ensembles n'ont aucun groupe en commun ; l'inspection du corpus a montré que les 12 000 paires étaient déjà

distinctes, mais l'absence de contrôle laissait passer ce risque sans le signaler. Les poids de fusion sont fixés avant l'évaluation et ne sont jamais ajustés sur le jeu de test. Enfin, une alerte automatique se déclenche lorsqu'une méthode présente à la fois une AUC supérieure ou égale à 0,80 et un F1 inférieur ou égal à 0,05, afin que la pathologie corrigée ici ne puisse pas réapparaître silencieusement. Ces contrôles sont couverts par des tests du dépôt, ce qui les rend vérifiables indépendamment de ce mémoire.

4.11 Limites, risques et perspectives

Les résultats sont internes à un corpus synthétique et à des étiquettes générées par règles. Ils ne mesurent ni la qualité d'une embauche, ni les effets sur les candidats, ni l'équité entre groupes protégés. Le dictionnaire peut omettre une technologie ou mal qualifier un intitulé ambigu ; les embeddings peuvent rapprocher des contenus trop généraux ; le modèle local peut produire une justification plausible mais erronée. JobTest doit donc rester une aide à la priorisation, avec révision humaine, traçabilité des critères et possibilité de correction.

Les suites nécessaires sont une validation avec des recruteurs sur des données autorisées et minimisées, la mesure de l'accord inter-évaluateurs, des tests de biais et de robustesse par sous-groupes, ainsi qu'une calibration prospective des seuils. L'évaluation du modèle de langage doit être séparée de celle du score : fidélité aux faits, utilité des questions et taux d'incohérences sont des axes distincts.

Le tableau 4.6 récapitule le rôle de chaque couche et le garde-fou qui lui est associé.

Tableau 4.6: Synthèse des couches et de leur statut

Couche	Rôle	Statut et garde-fou
Couche 1	score hybride, décision et explications	implémentée ; décision déterministe par seuils
Couche 2	analyse locale et questions d'entretien	optionnelle ; ne change jamais la couche 1
Évaluation	benchmark train/validation/test synthétique	disponible ; non représentatif d'une embauche réelle

Conclusion

Le moteur intelligent combine signaux lexicaux, sémantiques et structurés dans une recommandation contrôlable. Son évaluation est encourageante sur le corpus disponible, sans autoriser une généralisation à la sélection réelle. Le chapitre suivant présentera la réalisation de la plateforme et l'intégration de ces services dans l'expérience utilisateur.

Chapter 5

Réalisation et présentation de la plateforme

Plan

1 Environnement matériel et logiciel 65

2 Choix des technologies 66

3 Organisation du code source 69

4 Réalisation du backend 69

5 Réalisation du frontend 70

6 Intégration frontend–backend 71

7 Présentation des interfaces 71

8 Parcours fonctionnel de démonstration 82

9 Difficultés de réalisation et solutions apportées 83

Introduction

Ce chapitre décrit le passage de la conception à une plateforme utilisable par un recruteur. L'enjeu n'est pas de reprendre l'architecture logique ni les calculs présentés précédemment, mais de montrer comment ils sont rendus accessibles par une application Web cohérente : saisie des données, exécution d'une évaluation, restitution expliquée et conservation des résultats. Les éléments décrits ci-après proviennent de la version courante du dépôt JobTest ; l'ancienne interface Streamlit est donc considérée comme un prototype et non comme l'interface finale.

5.1 Environnement matériel et logiciel

5.1.1 Environnement matériel

Le développement, les validations locales et la rédaction de ce mémoire ont été réalisés sur l'ordinateur personnel ci-dessous. Cet environnement local est distinct de la machine virtuelle Azure décrite au chapitre 6 ; aucune mesure de performance matérielle n'est déduite de ces caractéristiques.

Le tableau 5.1 en donne les caractéristiques.

Tableau 5.1: Environnement matériel local de réalisation

Composant	**Caractéristique**
Ordinateur	HP
Processeur	Intel Core i3, 11e génération
Mémoire vive	8 Go de RAM
Processeur graphique	Intel UHD Graphics
Stockage total	Environ 1 300 Go

5.1.2 Environnement logiciel

Le développement associe un service Python et une application Web Next.js. L'environnement local s'appuie sur Windows, Python, Node.js et `npm`. L'API est exécutée avec Uvicorn ; le navigateur sert à vérifier les parcours de l'interface. Git est utilisé pour le suivi des sources. Les versions expressément verrouillées par le projet sont indiquées ; aucune version non vérifiable de système, d'éditeur ou de navigateur n'est inférée.

Le tableau 5.2 récapitule ces outils et le rôle tenu par chacun.

Tableau 5.2: Environnement de réalisation effectivement utilisé

Élément	**Outil ou technologie**	**Rôle dans le projet**
Système local	Windows	Développement et exécution locale des services.
Backend	Python, FastAPI et Uvicorn	Exposition de l'API, services métier et exécution ASGI.
Frontend	Next.js 14.2.35, React 18, TypeScript	Pages App Router, composants et interactions du recruteur.
Styles et animation	Tailwind CSS, Framer Motion	Système visuel, états de l'interface et transitions.
Données locales	SQLite et SQLAlchemy	Persistance locale des comptes, offres, candidats, CV et évaluations.
Outils IA	scikit-learn, Sentence-Transformers, spaCy, Ollama	Moteur de rapprochement et analyse qualitative optionnelle.
Documents	ReportLab	Construction du rapport PDF d'évaluation.
Conteneurs	Docker Compose, Nginx, PostgreSQL	Environnement de déploiement ; détaillé au chapitre 6.

5.2 Choix des technologies

Les choix présentés dans cette section sont justifiés par les responsabilités effectivement portées par chaque partie de JobTest.

5.2.1 Technologies frontend

Le répertoire `web/` contient le frontend final. Next.js organise les écrans dans `app/`, selon l'App Router, et React porte les composants réutilisables. TypeScript définit les contrats manipulés par le client API et évite que les données de réponse soient traitées comme des objets non structurés. Les routes principales correspondent notamment à la connexion, à la vue d'ensemble, aux offres, aux

candidats, à l'analyse, à l'historique, au classement et au centre de qualité.

Tailwind CSS est employé pour composer les styles de l'interface et les variantes des composants communs, tels que boutons, cartes, tableaux, badges, champs et états vides. Framer Motion est utilisé pour les transitions de pages, l'apparition du résultat et les animations de score. Le projet référence également Three.js, `@react-three/fiber` et `@react-three/drei` ; leur emploi est concret dans les éléments graphiques `AmbientBackground`, `ScoreTorus` et `NeuralField`. Ces éléments complètent la lecture visuelle du score sans porter la décision elle-même.

5.2.2 Technologies backend

FastAPI regroupe l'API sous le préfixe `/api/v1`. L'application installe les routeurs d'authentification et le routeur `/api/v1/ui`, destiné aux besoins du frontend actuel. Les modèles Pydantic définissent et valident les corps des requêtes, par exemple une offre, une évaluation ou une demande de rapport. La séparation rend explicites les entrées HTTP avant l'appel aux services de rapprochement.

SQLAlchemy représente les entités *User*, *Candidate*, *CV*, *JobOffer*, *MatchResult* et *Skill*. Les relations lient le candidat à ses CV et ses résultats, et le recruteur à ses propres candidats et offres. Les contrôles de propriété appliqués par l'API empêchent qu'un identifiant d'offre ou de candidat appartenant à un autre recruteur soit utilisé dans une évaluation.

Le parsing des CV repose sur `pdfplumber` avec repli `PyPDF2` pour les PDF, et sur `python-docx` avec repli XML pour les documents Word. ReportLab génère le PDF à partir des données déjà calculées : en-tête, identité affichée, score, radar, compétences, écarts, recommandations et aide à la décision. La génération reste ainsi découplée de l'affichage Next.js.

L'authentification s'appuie sur des jetons JWT transmis dans l'en-tête `Authorization: Bearer`. Le client conserve le jeton dans le stockage local du navigateur et l'ajoute aux appels protégés. Les mots de passe ne sont pas retournés par les endpoints ; seul le hachage est conservé dans le modèle utilisateur.

5.2.3 Technologies d'intelligence artificielle

Le premier niveau d'analyse mobilise les composants décrits au chapitre 4 : TF–IDF de scikit-learn, similarité sémantique avec Sentence-Transformers, extraction et rapprochement de compétences, puis fusion hybride. spaCy intervient lorsqu'un modèle linguistique est disponible, avec une stratégie de repli dans le prétraitement. Le chargement des services est différé jusqu'à la première requête nécessitant le moteur, ce qui permet à l'API de démarrer sans calculer immédiatement les modèles.

La seconde couche qualitative est appelée après l'affichage du résultat principal. Le runtime local Ollama est configuré avec `qwen2.5:1.5b`. Son indisponibilité n'empêche pas l'évaluation statistique : la réponse expose alors un état contrôlé tel que *unavailable*, *disabled* ou *error*. Cette distinction est importante pour l'utilisateur : l'analyse approfondie enrichit la lecture, mais ne modifie ni le score ni le verdict du premier niveau.

5.2.4 Technologies de données et d'infrastructure

SQLite constitue le choix par défaut en développement local grâce à une base fichier et à l'intégration SQLAlchemy. Pour l'environnement conteneurisé, le fichier `docker-compose.yml` définit PostgreSQL 16 comme service de données. Docker Compose réunit également l'API, l'application Web, Ollama et Nginx. Nginx est le point d'entrée HTTP ; il relaie les chemins `/api/v1/` vers FastAPI et les autres chemins vers Next.js. La procédure complète de sécurisation, de conteneurisation et de déploiement sur machine virtuelle Azure est réservée au chapitre 6.

Le tableau 5.3 synthétise ces choix et l'usage qui en est réellement fait.

Tableau 5.3: Synthèse des choix techniques et de leur usage

Domaine	**Technologie**	**Utilisation réelle dans JobTest**
Interface	Next.js, React, TypeScript	Écrans App Router, composants et client HTTP.
Présentation	Tailwind CSS, Framer Motion, Three.js	Styles, transitions et représentations visuelles du score.
API	FastAPI, Pydantic	Endpoints, validation et sérialisation JSON.
Persistance	SQLAlchemy, SQLite / PostgreSQL	Entités métier et stockage selon l'environnement.
Rapprochement	scikit-learn, Sentence-Transformers, spaCy	Signaux lexical, sémantique et compétences.
Analyse qualitative	Ollama, Qwen	Analyse approfondie non bloquante.
Export	ReportLab	Rapport PDF téléchargeable.

5.3 Organisation du code source

L'organisation privilégie la séparation entre interface, API, règles de rapprochement et accès aux données. L'arborescence simplifiée suivante ne montre que les dossiers participant directement au fonctionnement de la version étudiée.

```
web/                pages Next.js, composants, client API et styles
src/api/            application FastAPI et routeurs auth / UI
src/matching/       service d'évaluation et persistance du résultat
src/models/         TF-IDF, embeddings, compétences et chargement
src/fusion/         score hybride
src/decisioning/    règles de verdict
src/explainability/ explication et recommandations structurées
src/ai/             prompts et analyse qualitative Ollama
src/parsing/        extraction PDF/DOCX et nettoyage
src/reporting/      génération ReportLab
src/models_db.py    modèles SQLAlchemy
deploy/nginx/       configuration de relais HTTP
```

Le frontend ne recopie pas les règles du moteur. Son fichier `lib/api/client.ts` centralise les appels HTTP et les types associés. Côté API, le routeur de parité UI traduit les requêtes de l'interface vers le même service métier que les endpoints techniques. Cette organisation limite les divergences entre un calcul réalisé depuis un écran et un calcul réalisé par API.

5.4 Réalisation du backend

L'initialisation FastAPI crée les tables lorsqu'elles sont absentes et applique une vérification additive de schéma. Les routeurs couvrent l'inscription, la connexion, la lecture de l'utilisateur courant, les offres, les candidats, les résultats, l'évaluation, l'analyse approfondie, la comparaison des modèles, les indicateurs et le rapport PDF. Un middleware ajoute un identifiant de requête, mesure le temps de traitement et applique une limitation de fréquence en mémoire.

Le routeur UI fournit les opérations nécessaires aux pages : lecture et création d'offres, modification de leur statut, consultation et modification des candidats, lecture d'historique, évaluation textuelle ou par fichier, et lecture des indicateurs. Lors d'une évaluation, l'API vérifie l'offre

sélectionnée, le contenu minimal du CV et l'appartenance des données au recruteur connecté. Le service `evaluate_and_persist` calcule le résultat, crée ou retrouve le candidat selon les données fournies, conserve le CV et enregistre les sous-scores, le verdict, les seuils actifs et l'explication JSON.

La route de rapport reçoit les objets déjà calculés et renvoie un flux `application/pdf` avec un en-tête de téléchargement. Ce choix évite de recalculer le matching lors de l'export. De manière analogue, la route d'analyse approfondie reçoit l'identifiant d'un résultat persistant, reconstruit son contexte et renvoie une enveloppe d'état sans faire échouer le parcours lorsque le LLM n'est pas accessible.

5.5 Réalisation du frontend

La zone authentifiée est enveloppée par un layout commun : barre latérale, barre supérieure, transition de page et zone centrale. Au chargement, le layout vérifie la présence du jeton puis appelle `/auth/me`. En cas d'échec, il redirige vers la connexion. Cette vérification protège l'expérience utilisateur au niveau de l'interface, tandis que l'API vérifie de nouveau le jeton pour chaque ressource protégée.

Les pages utilisent des composants génériques pour maintenir une restitution homogène. `Table` porte les listes, `Modal` les opérations de modification ou de confirmation, `Field` les saisies, et `DataStates` les états de chargement ou d'absence de données. Les pages d'offres et de candidats affichent leurs données sous forme de tableau et appellent le client centralisé pour actualiser leur état après une modification. Les erreurs réseau ou fonctionnelles sont converties en notifications visibles ; le bouton de l'évaluation affiche également un état de travail pendant le calcul.

La page d'analyse regroupe la sélection d'une offre ouverte, le choix de méthode, l'identité facultative du candidat et deux modes d'entrée du CV : collage de texte ou téléversement PDF/DOCX. Après la réponse JSON, elle révèle le score, le verdict, les composantes, le radar de compétences, les forces, lacunes et recommandations. L'export PDF transmet les données du résultat vers le backend. Le composant `DeepAnalysis` déclenche séparément la couche qualitative et présente son état sans masquer la restitution du premier niveau.

5.6 Intégration frontend–backend

Le parcours d'une évaluation relie deux responsabilités complémentaires. Le navigateur assure la collecte et la restitution ; le serveur possède les règles métier et les données persistantes.

1. Le recruteur choisit une offre et saisit ou téléverse le CV dans l'écran *Candidate Analysis*.
2. Le frontend contrôle les informations requises et construit une requête vers `/api/v1/ui/evaluate` ou `/api/v1/ui/evaluate/file`.
3. Le client API ajoute le jeton bearer, puis FastAPI valide le schéma Pydantic, l'utilisateur et la propriété de l'offre.
4. Le service métier extrait éventuellement le document, exécute le moteur de niveau 1 et produit explication, recommandations et décision.
5. SQLAlchemy conserve le candidat, le CV et le résultat lorsque le parcours est persistant.
6. La réponse JSON est affichée par les composants de score ; l'analyse approfondie est demandée indépendamment après cette réponse.
7. Sur demande, le frontend envoie les éléments nécessaires à `/api/v1/report/pdf` et télécharge le document retourné.

En développement, le client pointe vers l'API locale configurée par `NEXT_PUBLIC_API_BASE`. Dans le déploiement, le même chemin relatif `/api/v1` est servi derrière Nginx, ce qui évite d'exposer au navigateur les adresses internes des conteneurs. Les réponses d'erreur HTTP sont transformées en messages d'interface ; l'état particulier de la Layer 2 est, lui, affiché comme une disponibilité de service et non comme un échec de l'évaluation principale.

5.7 Présentation des interfaces

Les écrans suivants sont ceux définis par le frontend Next.js final. La capture de la figure 5.1 a été obtenue sur l'instance locale lancée pour cette réalisation ; les valeurs préremplies correspondent au jeu de démonstration de la page. Aucune donnée de candidat réel n'a été utilisée.

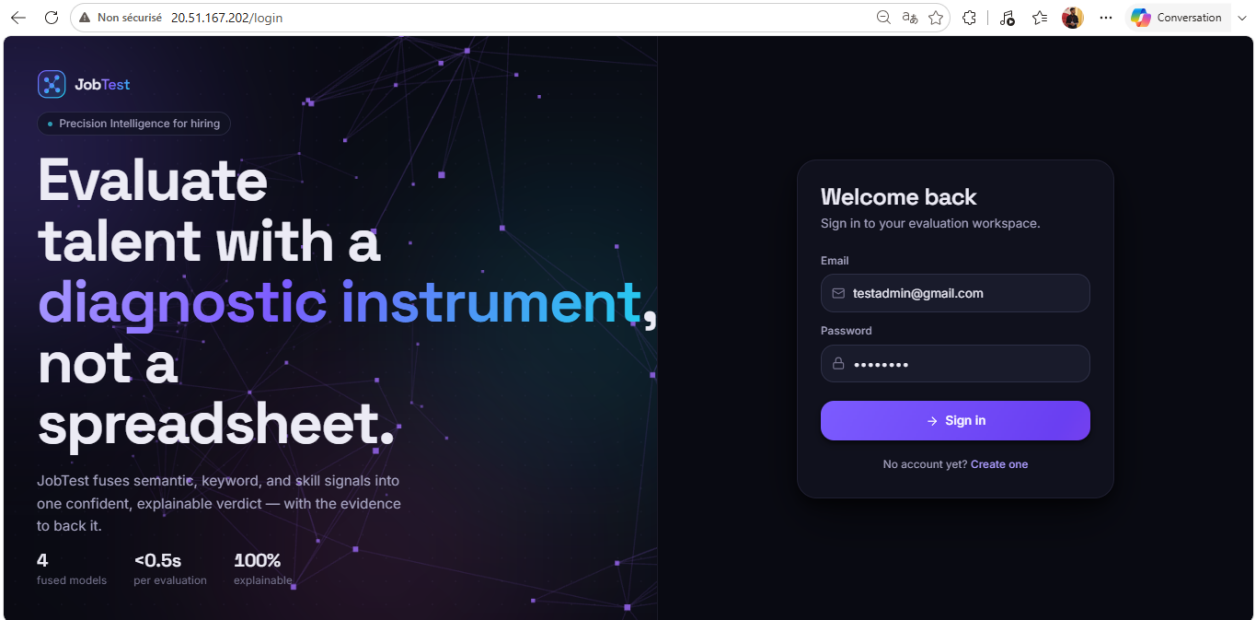


Figure 5.1: Écran réel de connexion de JobTest, sur l'instance locale de démonstration

5.7.1 Connexion et identité visuelle

La page de connexion comporte les champs courriel et mot de passe, un lien de création de compte et une notification d'erreur en cas d'échec. La partie visuelle présente l'identité JobTest et le rôle d'aide au recrutement. Après authentification, l'utilisateur est dirigé vers la vue d'ensemble. L'inscription est présente sur ce même écran ; elle demande le nom, le courriel, le mot de passe et sa confirmation.

La figure 5.2 montre le formulaire correspondant.

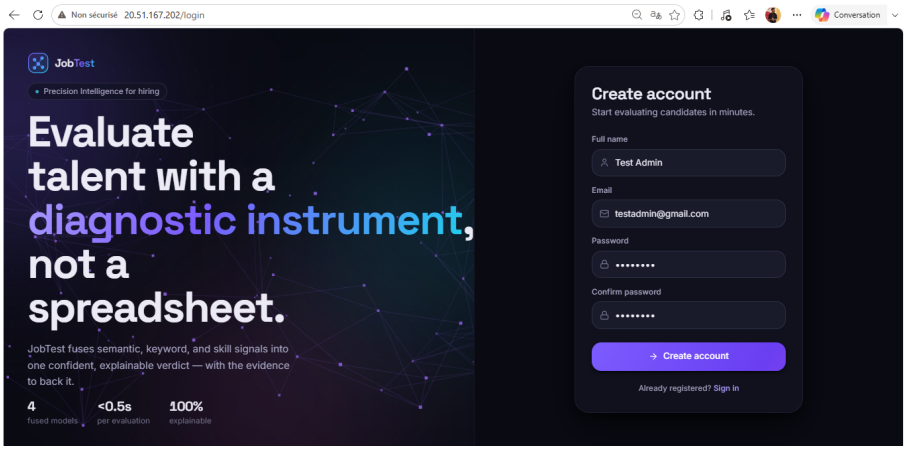


Figure 5.2: Écran d'inscription du recruteur

5.7.2 Consultation sur mobile

Les interfaces conservent leur lisibilité sur un écran mobile. Les captures suivantes illustrent l'adaptation de la consultation à un format compact, sans modifier les règles métier ni les résultats fournis par l'API.

La figure 5.3 donne un premier aperçu de cette adaptation.

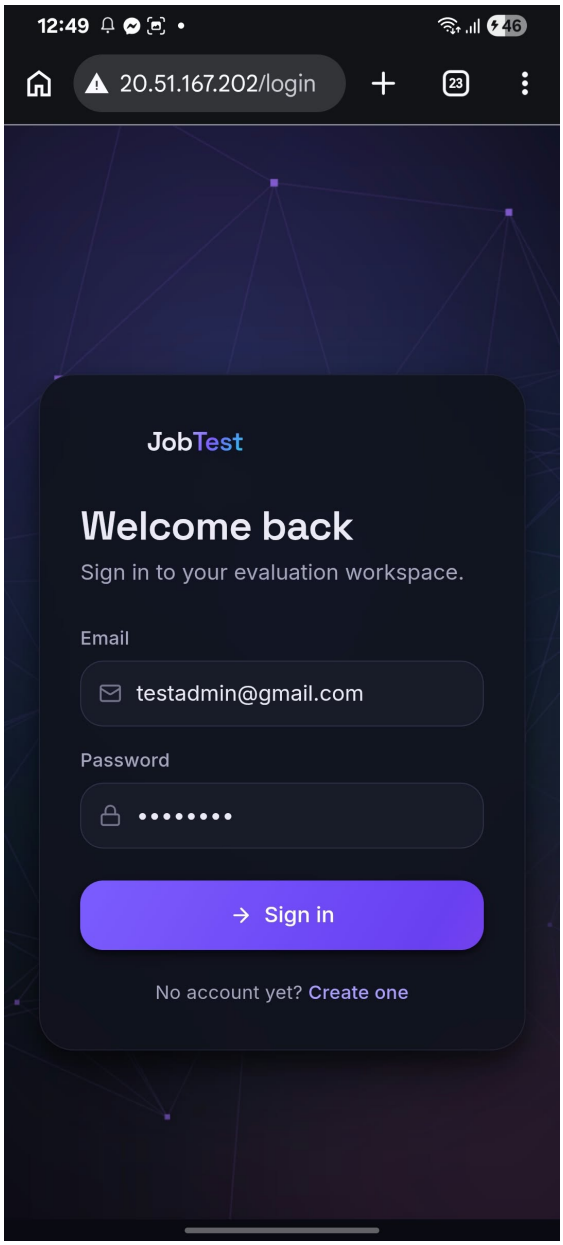


Figure 5.3: Interface JobTest adaptée à la consultation sur mobile

La figure 5.4 complète cette illustration sur un autre écran.

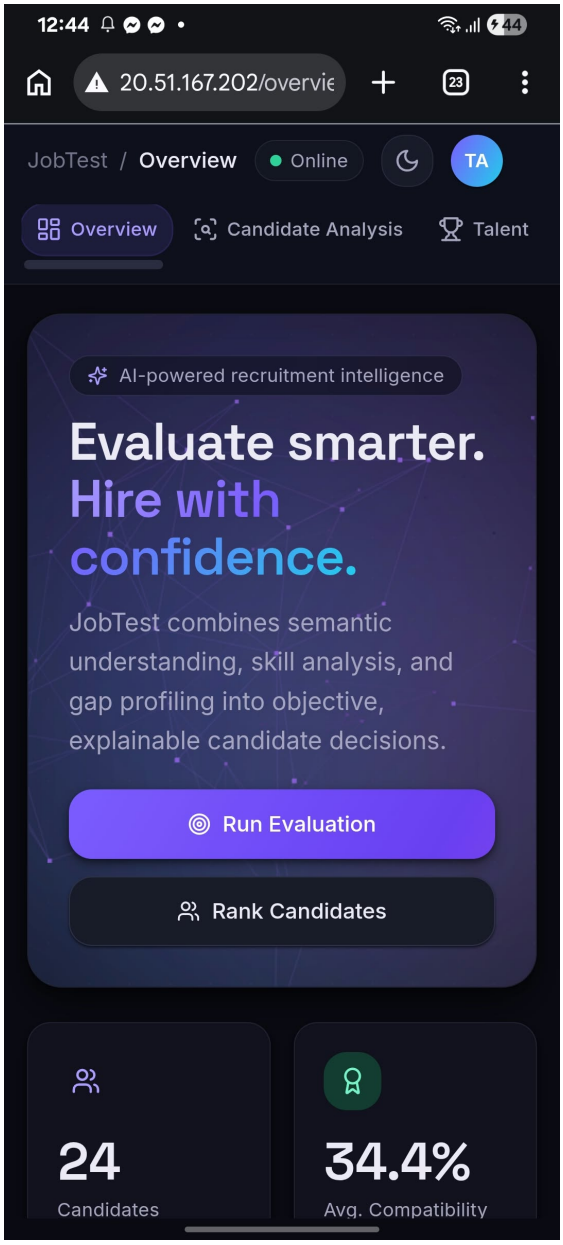


Figure 5.4: Deuxième vue de l'interface JobTest sur mobile

5.7.3 Vue d'ensemble, offres et candidats

La vue *Overview* synthétise le nombre de candidats, le nombre d'offres, le nombre d'évaluations, le score moyen ainsi que la part des verdicts *HIRE*. Elle complète ces indicateurs par les résultats récents. Ces valeurs sont calculées côté API à partir des objets appartenant au recruteur connecté, et ne sont donc pas des statistiques globales fictives.

La page *Job Offers* permet de créer, consulter, modifier, fermer ou rouvrir une offre. Chaque offre possède un code, un titre, une description, des compétences demandées et un statut parmi *OPEN*, *ON_HOLD* et *CLOSED*. La page *Candidates* distingue les candidats des comptes recruteurs.

Elle affiche les informations métier disponibles, l'état du parcours et les évaluations associées ; la modification et la suppression sont des actions explicitement proposées par l'interface.

Cette vue est reproduite sur la figure 5.5.

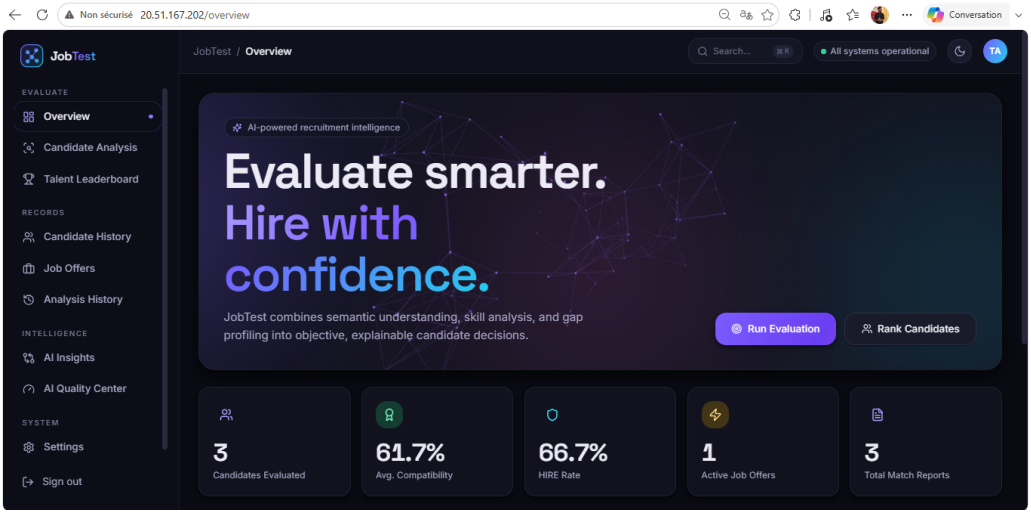


Figure 5.5: Vue d'ensemble de l'interface JobTest

La figure 5.6 illustre la recherche transversale accessible depuis la barre supérieure.

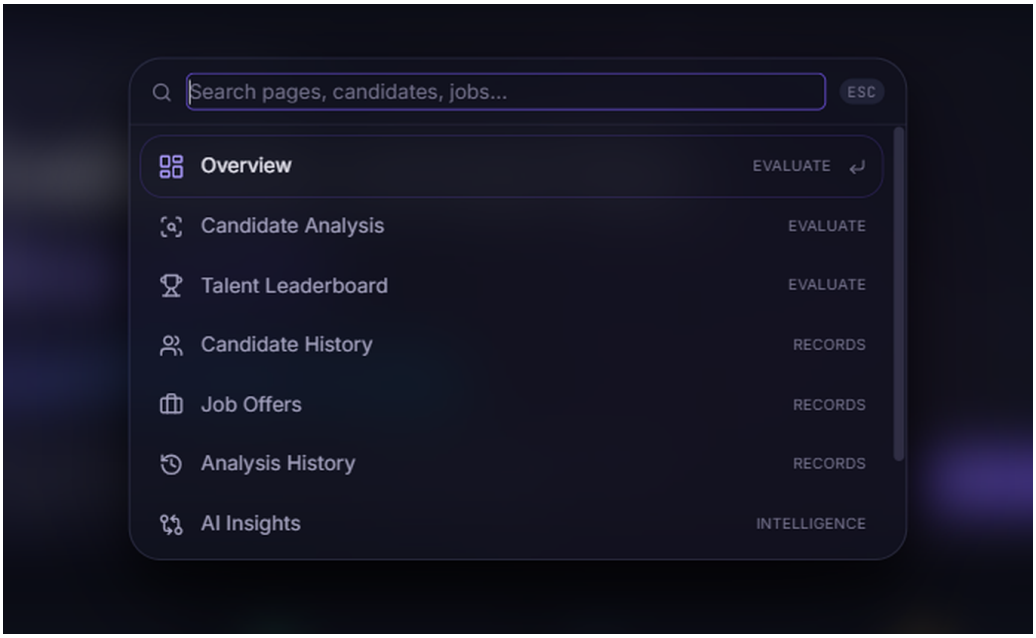


Figure 5.6: Fonction de recherche dans l'interface JobTest

La liste des offres apparaît sur la figure 5.7.

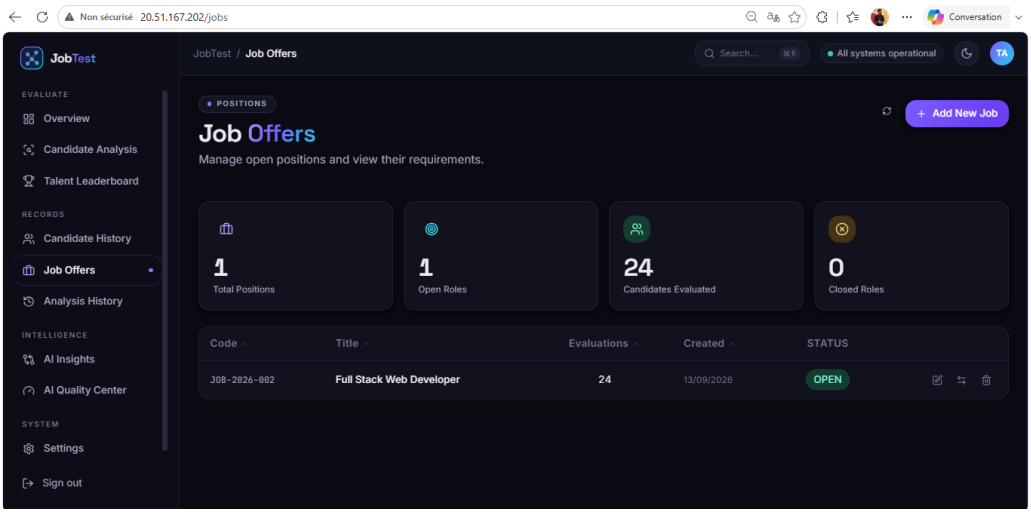


Figure 5.7: Consultation des offres d'emploi

Le formulaire de création est présenté par la figure 5.8.

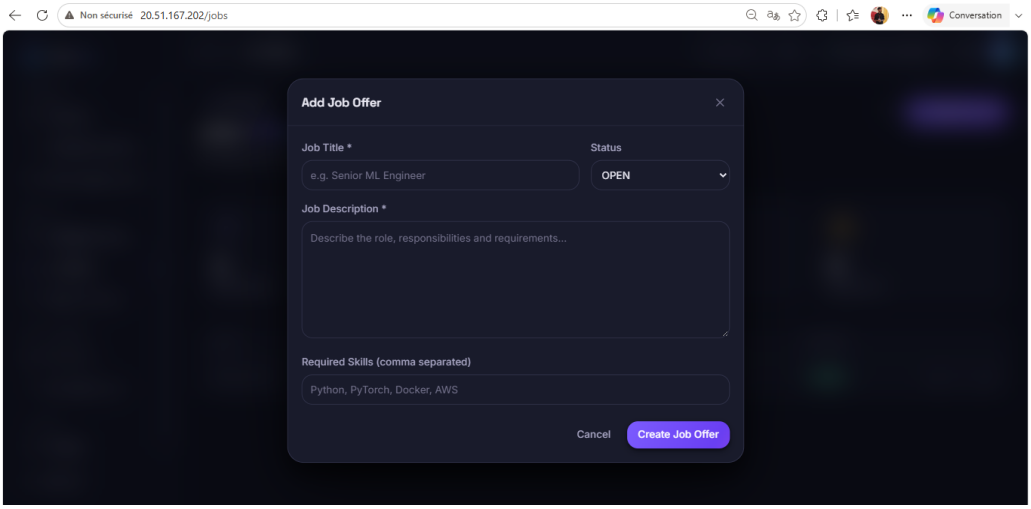


Figure 5.8: Formulaire de création d'une offre d'emploi

La figure 5.9 montre le suivi des candidats et leur état de pipeline.

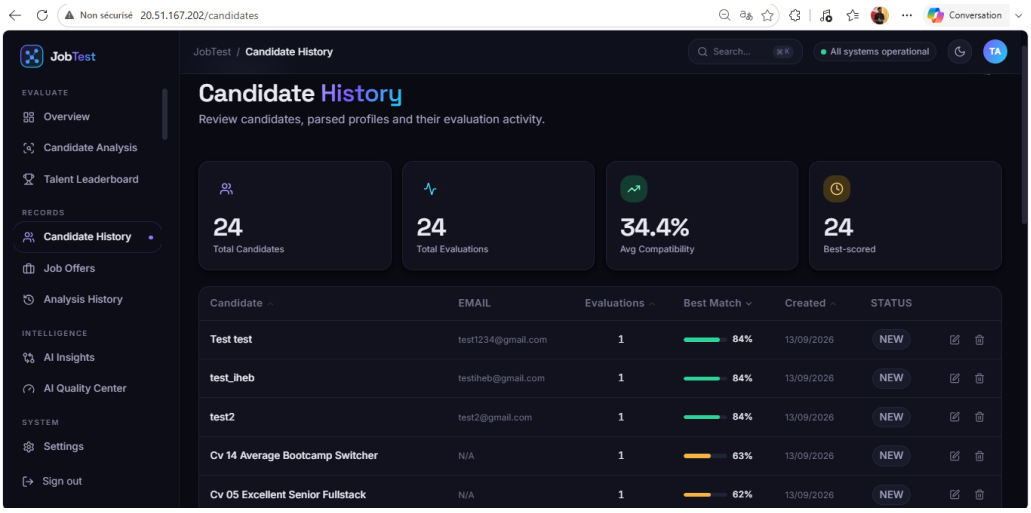


Figure 5.9: Suivi des candidats

L'historique des évaluations correspondantes est visible sur la figure 5.10.

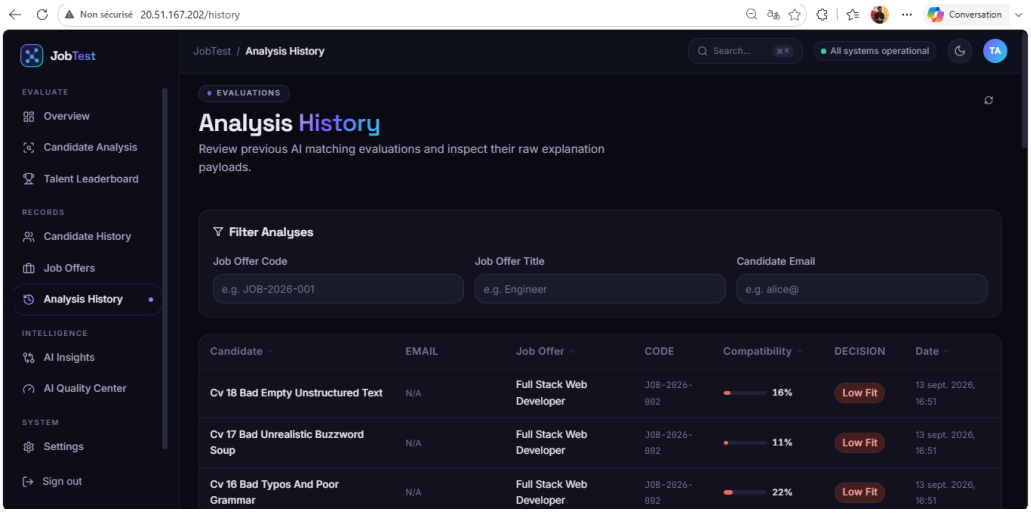


Figure 5.10: Historique des évaluations

5.7.4 Évaluation, résultat et analyse approfondie

L'écran *Candidate Analysis* constitue le point de départ du parcours principal. Il présente les compétences demandées de l'offre choisie, la politique de décision active et le bouton de lancement. Pendant le traitement, l'interface rend visibles les étapes de parsing, d'extraction de compétences, de calcul sémantique, de fusion et de verdict. Cette représentation décrit la progression technique sans présenter le système comme décideur d'embauche.

Le résultat affiche le score global, le verdict recommandé, les sous-scores, le radar et les éléments d'explication. Les compétences correspondantes et manquantes, les forces, les écarts et les actions recommandées sont issus du résultat calculé. Le verdict reste une recommandation

algorithmique que le recruteur interprète avec son propre jugement. Le composant d'analyse approfondie peut ensuite montrer une synthèse, une trajectoire, des signaux à examiner, des recommandations et des questions d'entretien lorsque le runtime LLM répond ; sinon, son indisponibilité est affichée sans altérer le score principal.

La figure 5.11 montre l'écran de saisie qui ouvre ce parcours.

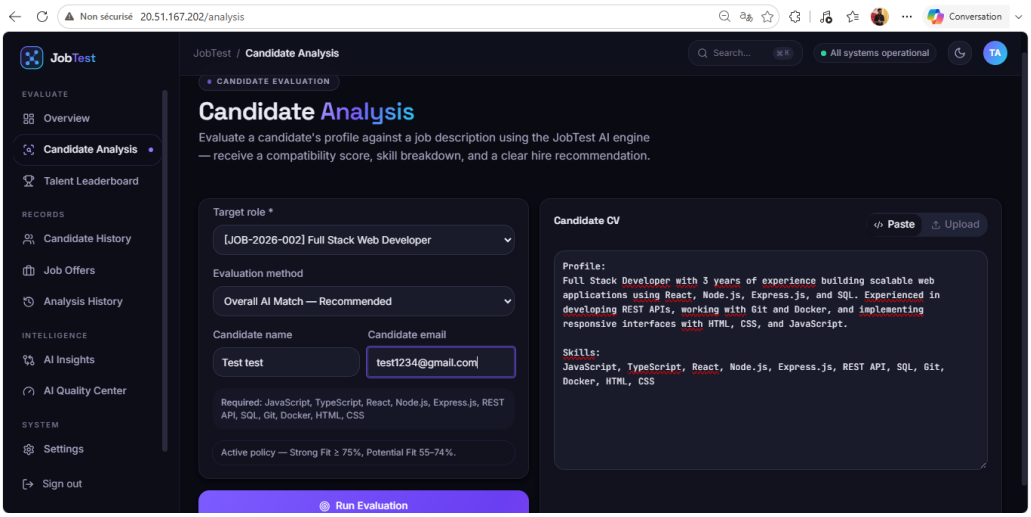


Figure 5.11: Saisie des éléments de l'évaluation d'un candidat

La progression du calcul puis le score global apparaissent sur la figure 5.12.

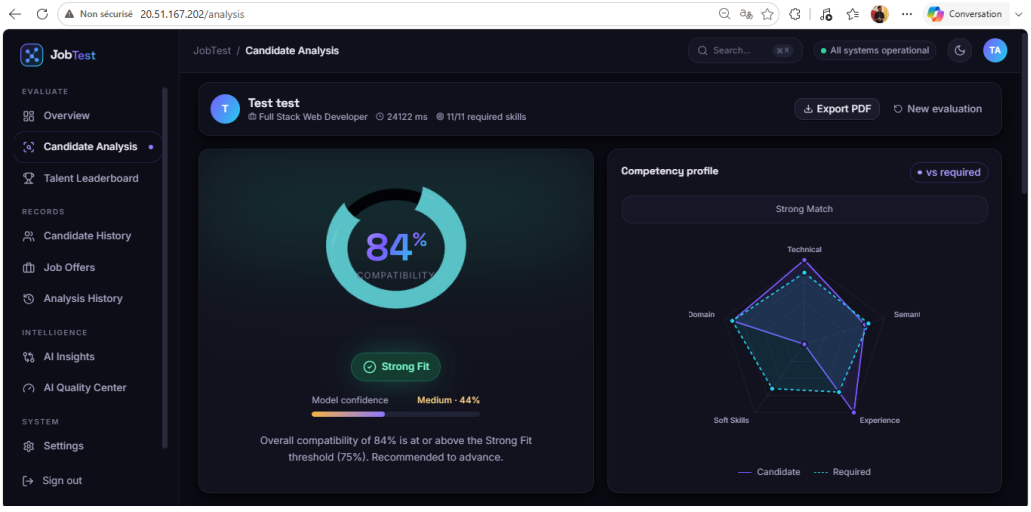


Figure 5.12: Progression du calcul et score global de l'évaluation

Le détail des composantes et de l'explication est reproduit par la figure 5.13.

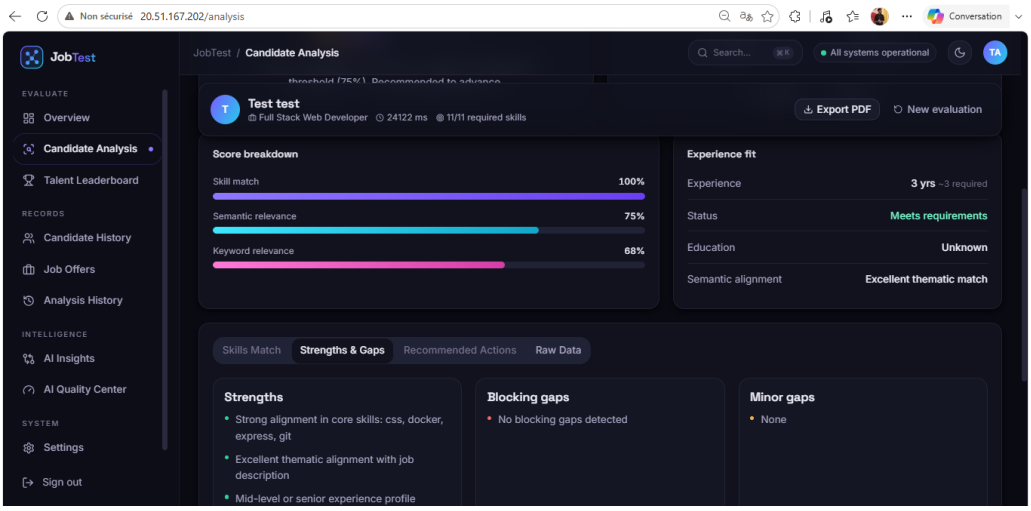


Figure 5.13: Composantes et explication du résultat d'évaluation

La figure 5.14 en donne la lecture graphique complémentaire.

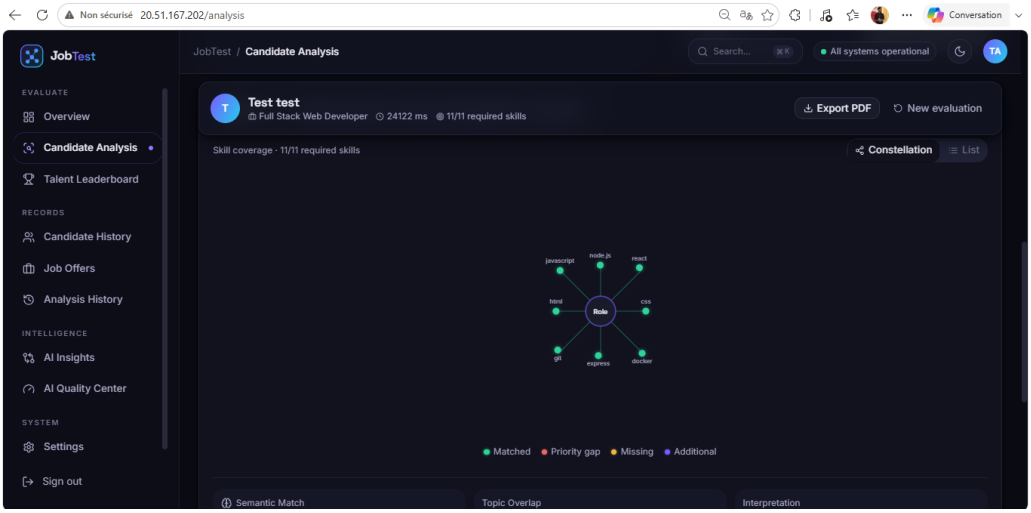


Figure 5.14: Visualisation complémentaire du résultat d'évaluation

La figure 5.15 montre la suite du parcours, une fois l'analyse approfondie demandée.

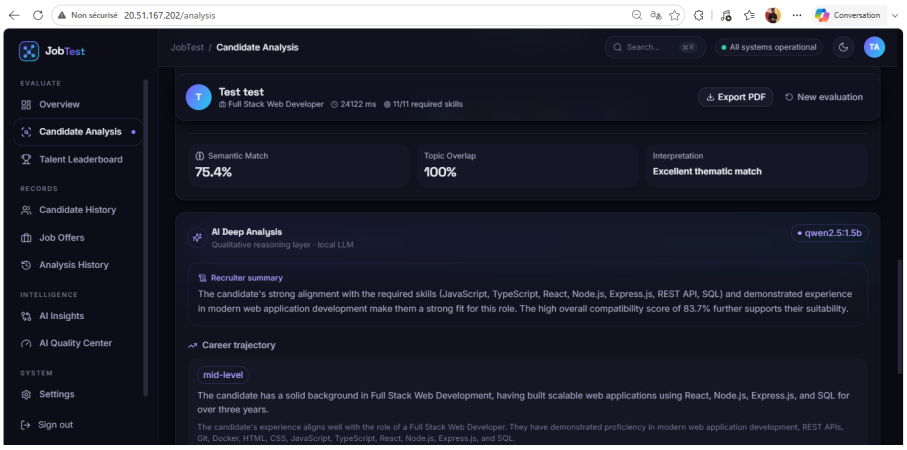


Figure 5.15: Poursuite de l'analyse approfondie dans le parcours candidat

5.7.5 Historique, comparaison et rapport PDF

La page *History* consulte les résultats persistés, propose un filtrage et donne accès au détail enregistré. *AI Insights* compare les quatre méthodes sur un même couple CV–offre sans créer un nouveau résultat persistant. Le *Talent Leaderboard* accepte plusieurs CV pour une offre et restitue un classement ; le *Quality Center* expose les résultats d'évaluation hors ligne et les figures associées. Enfin, le bouton *Export PDF* de l'écran de résultat déclenche la génération ReportLab et le téléchargement du rapport. Le document reprend les éléments de décision déjà visibles, au lieu de fournir une décision autonome supplémentaire.

La figure 5.16 montre la sélection de l'offre et des CV à comparer.

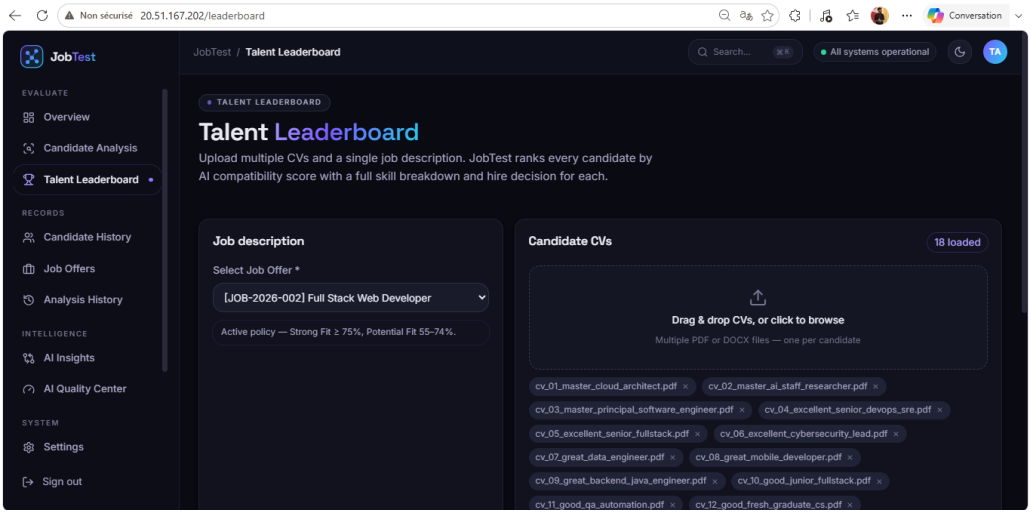


Figure 5.16: Sélection de l'offre et des CV dans Talent Leaderboard

Le classement obtenu est reproduit sur la figure 5.17.



Figure 5.17: Classement initial des candidats dans Talent Leaderboard

La figure 5.18 en présente la lecture synthétique et le détail d'un candidat.

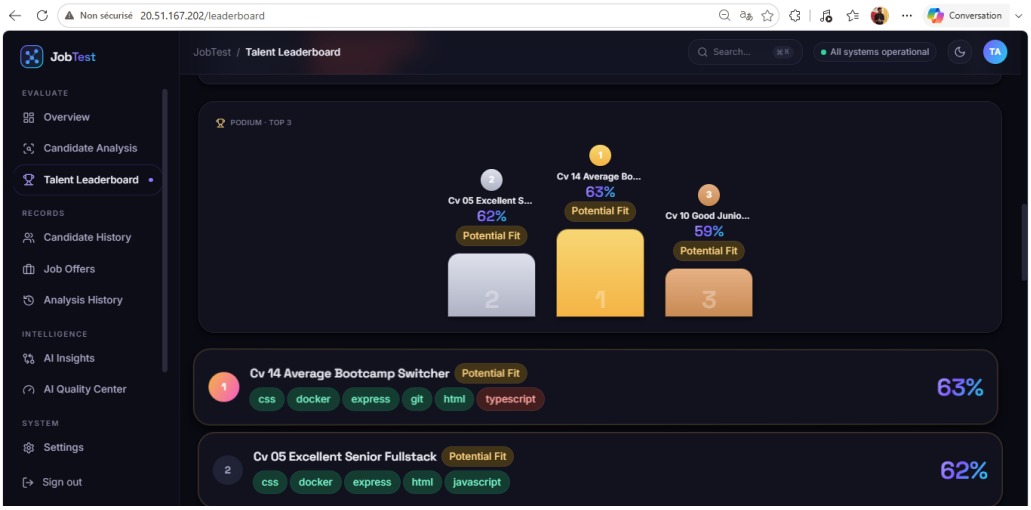


Figure 5.18: Podium et détail d'un classement multi-CV

La figure 5.19 complète cette restitution.

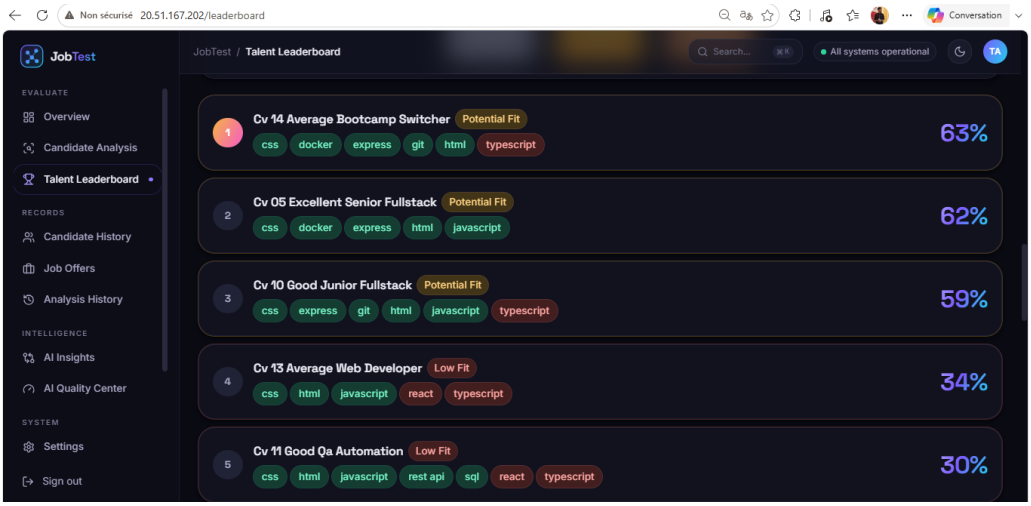


Figure 5.19: Résultats complémentaires du classement multi-CV

La figure 5.20 montre l'écran de réglage des seuils et l'état des composants de la plateforme.

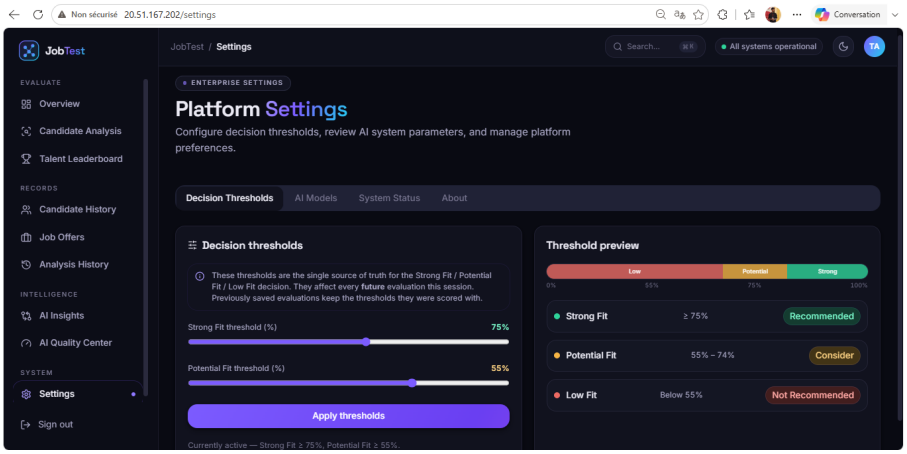


Figure 5.20: Écran de paramètres et état des composants de la plateforme

5.8 Parcours fonctionnel de démonstration

Un scénario de démonstration se déroule de façon continue : le recruteur se connecte, crée ou sélectionne une offre ouverte, fournit le texte ou le fichier d'un CV, puis lance l'évaluation. Il lit ensuite le score et les raisons associées, consulte si besoin l'analyse approfondie, puis exporte le rapport PDF. L'historique permet enfin de retrouver une évaluation persistée. Ce parcours maintient une séparation claire entre les données métier (offre, candidat et CV), le calcul de correspondance et l'aide à l'interprétation.

5.9 Difficultés de réalisation et solutions apportées

Les difficultés retenues ci-dessous sont celles qui apparaissent dans le code ou dans l'historique technique du projet. Elles concernent la réalisation de la plateforme, non les incidents de déploiement qui seront détaillés au chapitre suivant.

Le tableau 5.4 les présente avec la réponse apportée à chacune.

Tableau 5.4: Difficultés de réalisation et réponses apportées

Difficulté	**Conséquence**	**Solution retenue**
Prototype Streamlit et interface finale distincts	Risque que le moteur ne soit pas accessible par l'interface Web actuelle.	Création du routeur `/api/v1/ui`, réutilisant les services métier existants depuis Next.js.
CV PDF et DOCX hétérogènes	Extraction incomplète possible selon le document.	Parsers dédiés avec replis PDF et DOCX avant le prétraitement commun.
Modèles d'embeddings indisponibles ou contraints	Le composant sémantique ne doit pas empêcher le service de répondre.	Chargement différé et stratégie de repli locale documentée dans le moteur.
Layer 2 potentiellement lente ou indisponible	Une réponse LLM ne doit pas bloquer le verdict calculé.	Appel séparé, enveloppe d'état contrôlée et conservation du niveau 1 comme source du score.
Évolution SQLite vers PostgreSQL	Les entités et relations doivent rester cohérentes entre environnements.	Modèles SQLAlchemy communs et configuration par URL de base de données.
Export exploitable hors application	Le recruteur doit partager une synthèse lisible.	Génération serveur ReportLab à partir du résultat déjà obtenu.

Conclusion

JobTest est réalisé comme une plateforme Web modulaire dans laquelle Next.js porte le parcours du recruteur et FastAPI centralise les règles métier, les calculs et la persistance. Les écrans transforment un résultat de matching en éléments interprétables : score, compétences, explications, recommandations, analyse qualitative conditionnelle et rapport PDF. Le système soutient ainsi l'évaluation du recruteur sans se substituer à sa décision. Le chapitre suivant présentera les tests, les mesures de sécurité, la conteneurisation et le déploiement Azure.

Chapter 6

Tests, sécurité, DevOps et déploiement Azure

Plan

1 Stratégie de validation 85
2 Environnement et données de test 86
3 Tests fonctionnels 87
4 Validation du moteur intelligent 88
5 Tests de l'API et de la persistance 88
6 Validation du rapport PDF 89
7 Tests de santé et stabilité des services 89
8 Résultats de validation de bout en bout 89
9 Sécurité applicative 89
10 Sécurité réseau et infrastructure 90
11 Analyse honnête des limites de sécurité 91
12 Traitement des données à caractère personnel 92
13 Conteneurisation 93
14 Déploiement sur Microsoft Azure 94
15 Architecture de déploiement 97
16 Exploitation et maîtrise des coûts 97
17 Procédures d'exploitation 98
18 Bilan du déploiement 98

Introduction

Après la réalisation de la plateforme, sa valeur dépend autant de la fiabilité de la chaîne complète que de la cohérence du moteur de rapprochement. Ce chapitre présente les contrôles effectivement disponibles dans JobTest, les vérifications effectuées lors du déploiement et les protections visibles dans le code et la configuration. Il distingue les tests automatisés, les vérifications manuelles et les observations ponctuelles faites sur la machine de démonstration. L'objectif n'est pas de certifier une disponibilité de production, mais de documenter de manière traçable le périmètre validé, les choix d'exploitation et les limites restantes.

6.1 Stratégie de validation

La validation combine une suite `pytest`, des contrôles d'API et une vérification de bout en bout sur l'environnement conteneurisé. Les essais de benchmark du chapitre 4 restent séparés : ils étudient un corpus synthétique et ne constituent pas une suite de tests fonctionnels. Le tableau 6.1 synthétise le statut des éléments examinés.

Tableau 6.1: Niveaux de validation retenus pour JobTest

Niveau	Objectif	Outil ou méthode	Statut
Unitaires	Vérifier extraction, fusion, décision, PDF et règles de cohérence.	`pytest` ; modèles de langue et Ollama simulés lorsque nécessaire.	Partiel : 106 réussites sur 113 items.
API et intégration	Vérifier routes, statuts HTTP, JWT, persistance et contrats.	Tests FastAPI et contrôles d'API documentés.	Partiel : 6 tests hérités à adapter.
Fonctionnel	Vérifier le parcours recruteur depuis l'inscription jusqu'au rapport.	Scénario manuel enregistré sur la VM Azure.	Réalisé sur un jeu synthétique.
Chaîne IA	Vérifier sous-scores, verdict, explication et statut de la couche 2.	Tests du moteur, réponses de l'API et contrôle du résultat persistant.	Réalisé ; benchmark distinct au chapitre 4.
Déploiement	Vérifier les conteneurs, le proxy, les health checks et les ports.	`docker compose`, `nginx -t`, requêtes HTTP et journaux.	Réalisé ponctuellement.

Le dernier lancement documenté de `pytest -q` a produit 106 tests réussis, 6 échecs et 1 test ignoré (113 items, environ 45 secondes). Les six échecs sont concentrés dans `test_api_extended.py` : des fixtures de routes historiques ne satisfont plus le contrôle d'appartenance introduit pour les offres et résultats. Ils retournent donc `404` au lieu du résultat attendu par ces tests anciens. Ce constat ne permet pas de déclarer la suite entièrement verte ; sa mise à jour constitue une action de maintenance.

6.2 Environnement et données de test

Le développement a été conduit localement sous Windows avec Python, Node.js et SQLite, puis vérifié dans une topologie Docker Compose utilisant PostgreSQL. La validation intégrée a ensuite été

observée sur une machine virtuelle Azure. Les parcours fonctionnels ont utilisé un compte recruteur de démonstration, une offre et un CV synthétiques. Aucun résultat présenté dans ce chapitre ne requiert l'affichage d'un mot de passe, d'un jeton, d'une clé SSH, d'une chaîne de connexion ou du contenu du fichier `.env`. Les éléments visibles dans les traces de déploiement sont des données de vérification et non un jeu de CV réels.

6.3 Tests fonctionnels

La matrice suivante reprend seulement les scénarios dont l'exécution ou le résultat est attesté par les tests du dépôt ou le rapport de déploiement. Les cas non exécutés ne sont pas convertis artificiellement en succès.

Le tableau 6.2 en dresse la liste.

Tableau 6.2: Matrice des contrôles fonctionnels attestés

ID	Fonction	Précondition	Résultat attendu	Résultat obtenu	Statut
F01	Inscription et connexion	Compte de vérification.	Création puis obtention d'un jeton sans l'exposer.	Compte créé et JWT utilisé pendant la vérification Azure.	OK
F02	Refus d'identifiants invalides	Identifiants incorrects.	Réponse d'authentification refusée.	Couvert par les tests `test_auth`.	OK
F03	Consultation du profil	Jeton Bearer valide.	Retour du recruteur connecté.	Route protégée couverte par les tests d'authentification.	OK
F04	Création d'offre	Recruteur authentifié.	Offre enregistrée et rattachée au recruteur.	Offre de contrôle créée (identifiant 1 dans l'environnement observé).	OK
F05	Évaluation d'un CV	Offre et CV de démonstration.	Sous-scores, explication et recommandation générés.	Résultat `MatchResult` 1 créé, verdict `CONSIDER`.	OK
F06	Persistance	Évaluation réussie.	Résultat récupérable avec relations cohérentes.	Un résultat et une analyse LLM valide comptabilisés dans PostgreSQL.	OK
F07	Analyse approfondie	Résultat persistant, Ollama disponible.	Enveloppe d'analyse sans modification du score.	`status=ok` observé et analyse enregistrée.	OK
F08	Rapport PDF	Résultat calculé.	Flux PDF ouvrable avec informations d'évaluation.	PDF 1.4, deux pages et 112 039 octets dans ce cas.	OK

ID	Fonction	Précondition	Résultat attendu	Résultat obtenu	Statut
F09	Routes API historiques	Fixtures anciennes de test.	Réponse des routes historiques.	Six attentes sont incompatibles avec le contrôle de propriété.	À adapter

6.4 Validation du moteur intelligent

Le contrôle fonctionnel du moteur concerne ici sa mise en œuvre, non la qualité prédictive discutée au chapitre 4. Une évaluation retournée par l'API contient les trois sous-scores utilisés dans la fusion, les paramètres de pondération, l'explication structurée et la recommandation algorithmique. Les tests couvrent la fusion hybride, les frontières de décision, la cohérence entre moteur et module d'explication ainsi que les états `disabled`, `unavailable`, `error`, `ok` et `inconsistent` de la couche 2.

La couche générative reçoit le résultat déterministe mais ne calcule ni le score officiel ni le verdict `HIRE`, `CONSIDER` ou `REJECT`. Les tests de raisonnement emploient un Ollama simulé ; la disponibilité réelle de la couche 2 a, elle, été vérifiée une fois sur Azure. Les cas de CV vide, d'offre inexistante et de ressources appartenant à un autre recruteur sont contrôlés par les routes concernées. Les résultats de cas de seuil exact 55 ou 75 ne sont pas présentés séparément ici, faute de trace de démonstration dédiée.

6.5 Tests de l'API et de la persistance

Les schémas Pydantic valident les corps de requête et FastAPI retourne notamment des erreurs `401`, `404`, `422` ou `429` selon le contrat concerné. Chaque route de données passe par l'authentification Bearer. La fonction `_assert_owned` vérifie que l'offre et le candidat visés appartiennent au recruteur connecté ; une ressource d'un autre périmètre est traitée comme absente. Les requêtes courantes sont exprimées par SQLAlchemy plutôt que par concaténation de SQL ; cette pratique réduit l'exposition à des chaînes SQL construites manuellement, sans constituer à elle seule une garantie universelle.

La vérification Azure a confirmé l'écriture d'un `MatchResult` et d'une analyse LLM valide, puis leur comptage dans PostgreSQL. Les relations candidat, offre et résultat ont donc été parcourues dans un scénario réel de démonstration. Swagger était accessible derrière Nginx à `/api/v1/docs` ; aucune capture n'est ajoutée afin d'éviter de répéter les écrans du chapitre 5.

6.6 Validation du rapport PDF

La génération est testée dans le dépôt par des contrôles de structure binaire et a également été validée dans le parcours déployé. Pour un résultat précis, la réponse HTTP a été reçue sous forme de PDF 1.4, le fichier s'est ouvert sur deux pages et sa taille était de 112 039 octets. Ces valeurs caractérisent un exemple de démonstration : le nombre de pages et le poids peuvent évoluer avec les données du candidat et de l'offre. L'export utilise les informations déjà calculées ; il ne lance pas un nouveau rapprochement.

6.7 Tests de santé et stabilité des services

Les cinq conteneurs ont été observés à l'état *healthy*, avec une politique `unless-stopped` et sans redémarrage inattendu lors du contrôle. La configuration Nginx a passé `nginx -t`; l'accueil et Swagger ont retourné HTTP 200 via le proxy. L'endpoint FastAPI de santé et son contrôle détaillé ont retourné `ok`; la couche 1 était disponible, Ollama répondait et le modèle `qwen2.5:1.5b` était installé. Les journaux examinés ne montraient ni OOM, ni erreur fatale, ni boucle de redémarrage. Il s'agit d'une observation à un instant donné et non d'un engagement de disponibilité permanente.

6.8 Résultats de validation de bout en bout

Le scénario intégré attesté suit les étapes suivantes : utilisation d'un compte recruteur, authentification, création d'une offre, évaluation d'un CV synthétique, enregistrement du résultat, lecture de l'explication, analyse approfondie puis génération du PDF servi par Nginx. Cette chaîne a produit les trois composantes de score, une recommandation `CONSIDER`, un `MatchResult` persistant, une analyse LLM `ok` et le rapport PDF décrit précédemment.

La mesure de l'analyse approfondie est de 54 290,6 ms sur cette VM CPU. Elle doit être interprétée comme une mesure ponctuelle sur une infrastructure de démonstration et non comme un objectif de temps de réponse. La couche 1 demeurait opérationnelle pendant que la couche qualitative est, par conception, additive et consultative.

6.9 Sécurité applicative

Les mots de passe sont traités par `passlib` avec le schéma `bcrypt` visible dans le code. Les jetons JWT sont signés en HS256, portent une expiration et sont vérifiés à chaque dépendance

`get_current_user`. La clé provient de l'environnement ; en son absence, l'application fabrique une valeur aléatoire au démarrage, ce qui invalide les sessions au redémarrage et produit un avertissement. En déploiement, une valeur durable est donc fournie hors dépôt.

La séparation par `recruiter_id`, les contrôles de propriété et les schémas Pydantic constituent les protections principales des opérations métier. Les fichiers sont acceptés selon l'extension PDF/DOCX et passent par un fichier temporaire ; Nginx limite le corps des requêtes à 15 Mo. Cette dernière limite ne remplace pas une validation de signature de fichier : celle-ci n'est pas implémentée. Un limiteur en mémoire applique par défaut 60 requêtes par minute et par adresse ; il ne remplace ni une politique de verrouillage de compte ni une protection distribuée.

Les secrets réels sont exclus du dépôt et des contextes Docker. La configuration de la VM place `/opt/jobtest/.env` avec le mode 0600 observé. Les erreurs de la couche LLM sont ramenées à une enveloppe d'état contrôlée : la chaîne statistique reste utilisable si Ollama est indisponible. Ces mécanismes sont des mesures concrètes, mais ne justifient aucune revendication de conformité ou de sécurité totale.

6.10 Sécurité réseau et infrastructure

Un groupe de sécurité réseau Azure (NSG) filtre le trafic entrant et sortant au niveau des ressources virtuelles ; ses règles peuvent notamment spécifier le protocole, le port, l'action et la priorité [20]. Dans le déploiement observé, une règle effective autorise HTTP TCP 80 et SSH TCP 22 est ouvert pour l'administration. Les détails administratifs de la règle (nom, priorité et association exacte) n'ont pas été récupérés depuis le portail Azure et ne sont donc pas reconstitués.

Nginx est le seul conteneur publiant un port hôte. Il relaie `/api/v1/` vers FastAPI et les autres chemins vers Next.js. Compose attache les services au réseau `jobtest` de type bridge ; ils sont joignables par leurs noms de service, tandis qu'un port hôte n'est nécessaire que pour l'accès extérieur [21]. Ainsi, 3000, 8000, 5432 et 11434 ne sont pas publiquement exposés. UFW a été observé inactif sur la VM : l'isolation actuelle repose donc sur le NSG, l'absence de mappings de ports et le proxy, plutôt que sur une seconde couche active de filtrage local.

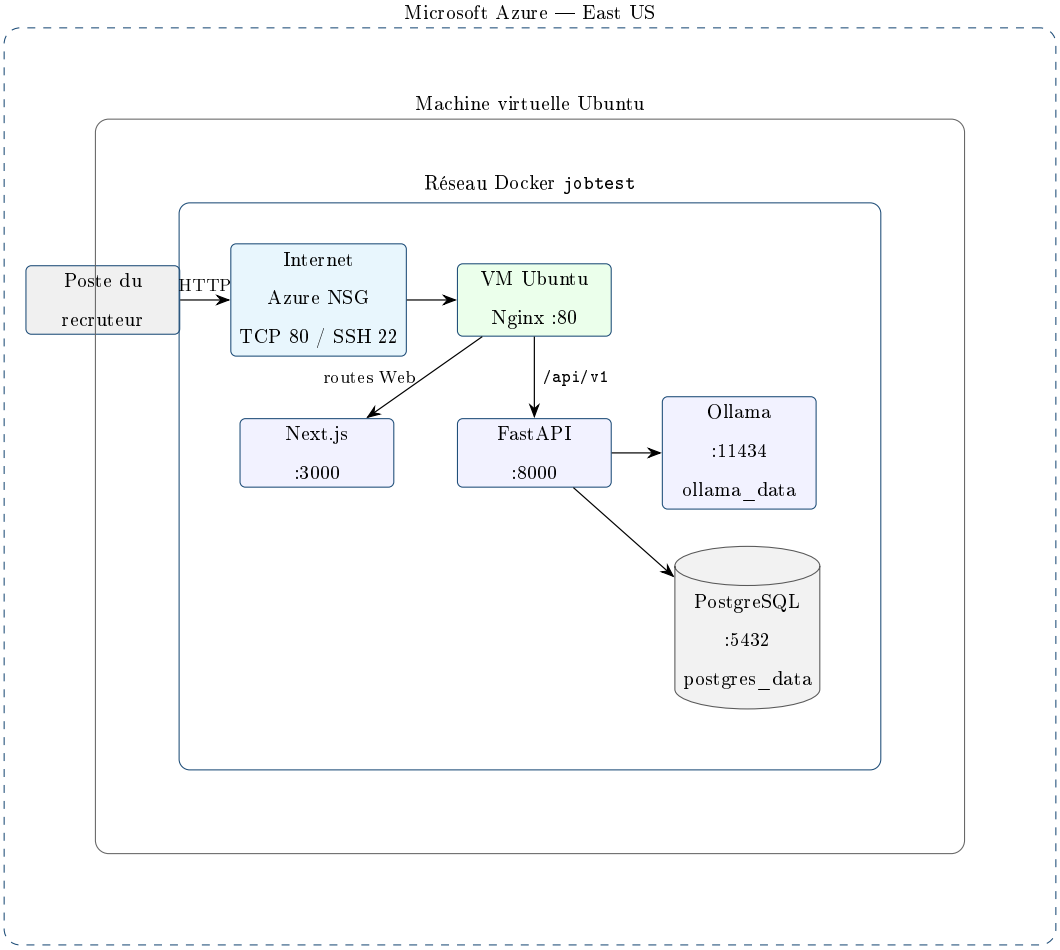


Figure 6.1: Architecture concrète de déploiement : accès public, VM et réseau Docker interne

6.11 Analyse honnête des limites de sécurité

Le démonstrateur utilise HTTP, sans domaine ni certificat TLS. Le jeton Bearer est conservé côté frontend dans `localStorage`; une injection de script réussie pourrait donc le lire. OWASP recommande de ne pas placer de données sensibles dans ce stockage lorsque des attaques XSS sont à considérer [22]. Un passage vers des cookies `HttpOnly`, `Secure` et `SameSite` est une amélioration souhaitable, associée à une protection XSS approfondie.

Le port SSH exposé doit être limité à une adresse d'administration connue ; `fail2ban` peut compléter cette mesure selon le contexte. Aucune rotation automatisée des secrets, aucun WAF, aucune supervision centralisée ni sauvegarde PostgreSQL planifiée ne sont attestés. L'infrastructure n'a qu'une VM, une base et un point d'entrée : elle ne fournit ni haute disponibilité ni reprise automatisée. Les téléversements restent perfectibles (contrôle de type fondé surtout sur l'extension, pas de validation par signature), et les textes envoyés à la couche LLM doivent être considérés comme

non fiables. Une politique de conservation, d'anonymisation et de suppression contrôlée des CV est également nécessaire avant un emploi sur des candidatures réelles.

6.12 Traitement des données à caractère personnel

Un CV n'est pas un document technique ordinaire : il identifie une personne et décrit son parcours. Toute plateforme qui l'ingère traite donc des données à caractère personnel, et il serait malhonnête de discuter la sécurité de JobTest sans nommer cette dimension. Les catégories manipulées par la plateforme sont l'identité déclarée du candidat, son adresse électronique et son téléphone lorsqu'ils figurent dans le dossier, le texte intégral du CV avec les employeurs, établissements et dates qu'il contient, puis les données produites par le système lui-même : score, verdict, compétences détectées ou absentes et analyse qualitative éventuelle. Cette dernière catégorie mérite attention, car une appréciation calculée sur une personne devient à son tour une donnée la concernant.

Le cadre applicable en Tunisie est la loi organique n° 2004-63 du 27 juillet 2004 relative à la protection des données à caractère personnel [23]. Sans en reprendre le détail, son économie générale repose sur quelques principes : le traitement suppose en règle générale le consentement de la personne concernée, il doit poursuivre une finalité déterminée et légitime, les données collectées doivent rester en rapport avec cette finalité, et la personne conserve des droits d'accès, de rectification et de suppression sur ce qui la concerne. La loi institue également une autorité de contrôle, l'Instance nationale de protection des données à caractère personnel, chargée de recevoir les déclarations de traitement, de délivrer les autorisations prévues par le texte et d'instruire les réclamations [24]. Un usage réel de JobTest sur des candidatures authentiques relèverait de ce cadre et supposerait les démarches correspondantes auprès de cette instance ; le présent projet, conduit sur des CV synthétiques dans un contexte académique, ne les a pas engagées.

Plusieurs choix d'architecture décrits dans ce mémoire ont néanmoins un effet direct sur la protection des candidats, et il est utile de les relire sous cet angle plutôt que de les présenter comme de simples mesures techniques. Le cloisonnement par `recruiter_id` fait qu'un dossier n'est lisible que dans l'espace du recruteur qui l'a créé : une offre ou un candidat appartenant à un autre compte est traité comme inexistant, ce qui limite la circulation interne des données. L'exécution locale du modèle de langage est le second point notable : la couche 2 s'adresse à un conteneur Ollama joint par le réseau interne, de sorte que le contenu d'un CV n'est jamais transmis à un fournisseur d'intelligence artificielle externe, contrairement à ce qu'impliquerait l'appel d'une API

tierce. Enfin, la suppression d'un dossier candidat est une action exposée par l'interface, ce qui fournit un premier support à une demande d'effacement, même si sa portée reste bornée par la règle qui protège l'historique des évaluations.

Ces éléments sont des dispositions de conception, non une mise en conformité. Plusieurs exigences que la loi rend centrales ne sont pas couvertes par la version étudiée : aucun mécanisme de recueil ni de preuve du consentement n'existe, puisque le candidat n'interagit jamais avec la plateforme ; aucune durée de conservation n'est définie ni appliquée automatiquement aux CV et aux résultats ; aucune information n'est délivrée à la personne évaluée sur l'existence du traitement ou sur la logique du score qui la concerne ; et la démonstration circule en HTTP, sans chiffrement du transport. À cela s'ajoute que le texte d'un CV est conservé intégralement dans la base, alors qu'une partie de son contenu n'est pas nécessaire au calcul de correspondance.

Les travaux à mener avant tout usage sur des candidatures réelles se déduisent de cette liste : définir et appliquer une durée de conservation avec purge automatique, informer les candidats et organiser le recueil de leur consentement par le recruteur, réduire les données stockées à ce que le rapprochement exige réellement, ouvrir une voie d'exercice des droits d'accès et de suppression, chiffrer le transport, et effectuer les formalités requises auprès de l'autorité de contrôle. Ces points ne sont pas des perspectives lointaines mais des conditions d'entrée en service ; les mentionner explicitement vaut mieux que de laisser croire qu'un prototype validé techniquement serait, pour cette raison, utilisable sur des personnes.

6.13 Conteneurisation

Docker Compose décrit cinq services et les volumes nommés qui conservent les données hors du cycle de vie d'un conteneur. Les Dockerfiles de l'API et de Next.js construisent respectivement les images applicatives ; les images Nginx, PostgreSQL et Ollama sont référencées par Compose. Les dépendances conditionnent le démarrage de l'API à la santé de PostgreSQL, et celui de Nginx à la santé de l'API et du frontend. Les health checks contrôlent l'API, la page web, `pg_isready`, `ollama list` et la réponse HTTP locale de Nginx.

Le tableau 6.3 précise le rôle, le port et la persistance de chacun d'eux.

Tableau 6.3: Services conteneurisés de JobTest

Conteneur	Image ou build	Rôle	Port	Persistance
Nginx	`nginx:1.27-alpine`	Entrée HTTP et proxy inverse.	80 public	Configuration montée en lecture seule.
Web	Build `web/Dockerfile`	Interface Next.js.	3000 interne	Aucune donnée métier.
API	Build `Dockerfile.api`	API, moteur et génération PDF.	8000 interne	`embedding_cache`.
PostgreSQL	`postgres:16-alpine`	Base relationnelle.	5432 interne	`postgres_data`.
Ollama	`ollama/ollama:0.5.13`	Analyse qualitative locale.	11434 interne	`ollama_data`.

6.14 Déploiement sur Microsoft Azure

Le déploiement a suivi un enchaînement simple : création du groupe de ressources, création de la VM Ubuntu, accès SSH, préparation de Docker, transfert de la version retenue, création locale du fichier `.env`, construction des images, téléchargement du modèle Ollama, démarrage Compose puis vérification des services et de Nginx. Le port HTTP a été ouvert de façon contrôlée pour la démonstration. La VM `vm-jobtest-pfe`, dans le groupe `rg-jobtest-pfe`, est localisée en East US. L'instance publique de démonstration est accessible à l'adresse `http://20.51.167.202/`. Cette adresse ne doit pas être considérée comme stable en cas de désallocation ou de redémarrage de la VM.

QR code d'accès à la démonstration publique de JobTest

`http://20.51.167.202/`

Tableau 6.4: Configuration observée de la machine virtuelle de démonstration

Paramètre	**Valeur observée ou déclarée dans le rapport de déploiement**
Fournisseur et région	Microsoft Azure, East US.
Machine et système	`vm-jobtest-pfe`, Ubuntu Server 24.04 LTS.
Taille	Standard D2as_v7 ; 2 vCPU et 8 Gio de mémoire.
Stockage et mémoire virtuelle	Disque Standard SSD ; swap persistante de 4 Gio.
Accès	HTTP TCP 80 public ; SSH TCP 22 pour l'administration.
Arrêt automatique	Configuré autour de 02 h, fuseau *W. Europe Standard Time*.

Les captures du portail Azure ci-dessous sont recadrées afin de ne pas afficher l'adresse e-mail de connexion ni l'identifiant d'abonnement. Elles constituent des constats visuels de l'environnement de démonstration et complètent les valeurs synthétisées dans le tableau 6.4.

La figure 6.2 présente l'abonnement utilisé pour la démonstration.

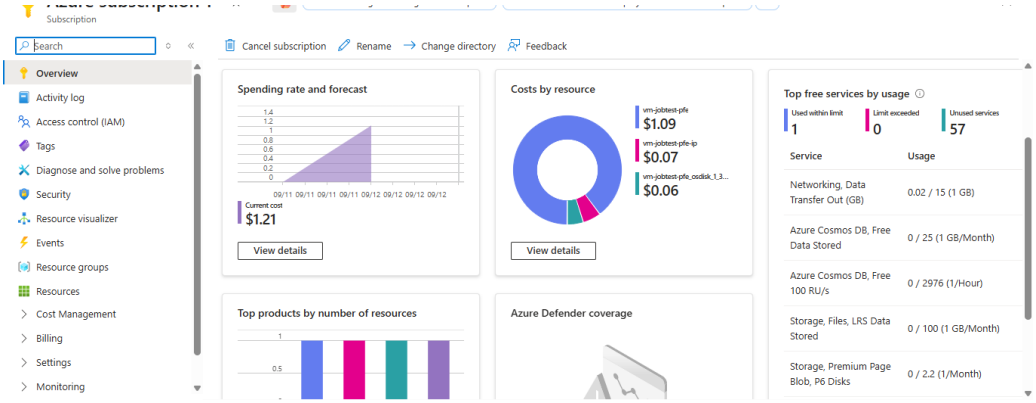


Figure 6.2: Vue de l'abonnement Azure, recadrée

La figure 6.3 donne l'aperçu de consommation associé.

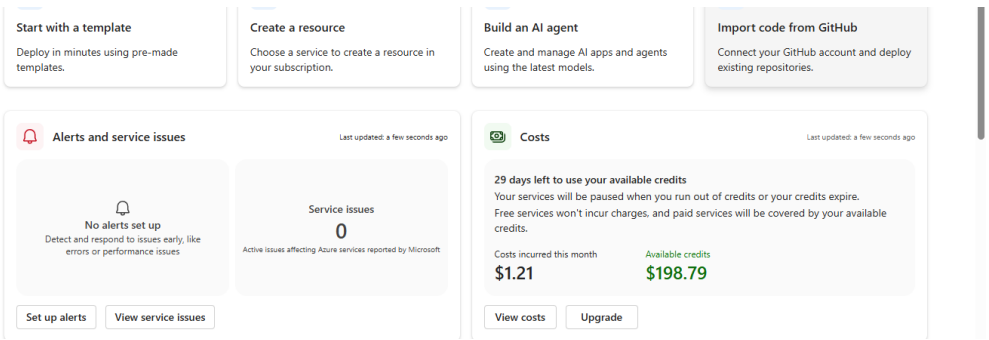


Figure 6.3: Aperçu de consommation Azure, recadré

Les ressources créées pour le projet sont visibles sur la figure 6.4.

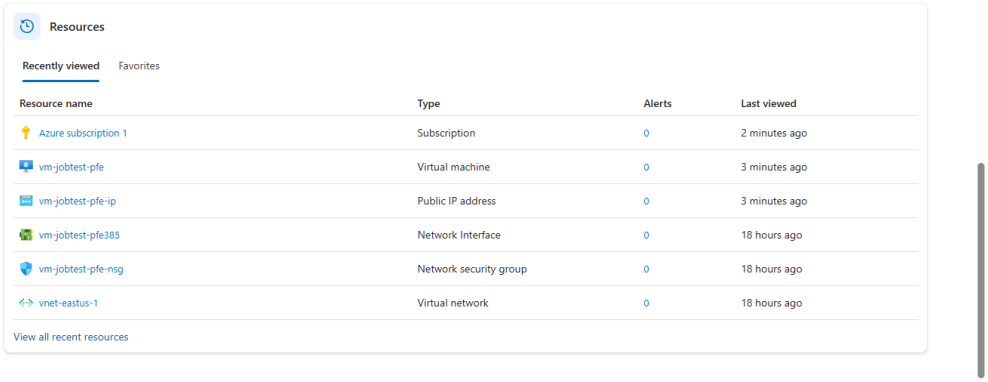


Figure 6.4: Ressources Azure associées à la démonstration

La figure 6.5 reprend les propriétés de la machine virtuelle.

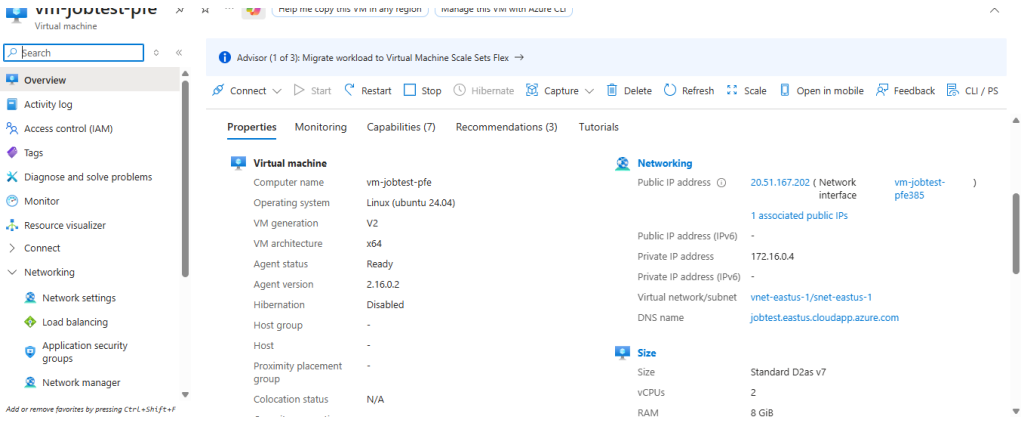


Figure 6.5: Propriétés observées de la machine virtuelle

Son état d'exécution apparaît sur la figure 6.6.

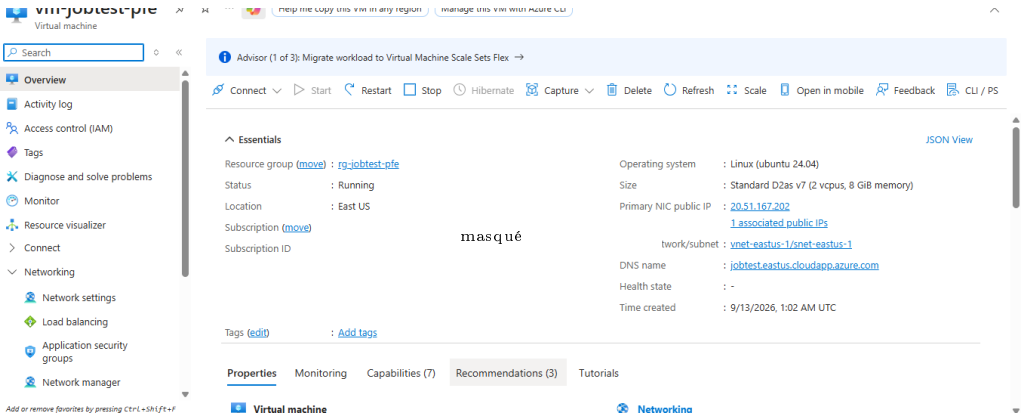


Figure 6.6: État de la machine virtuelle, recadré

La figure 6.7 montre enfin la ressource d'adresse IP publique rattachée à la machine.

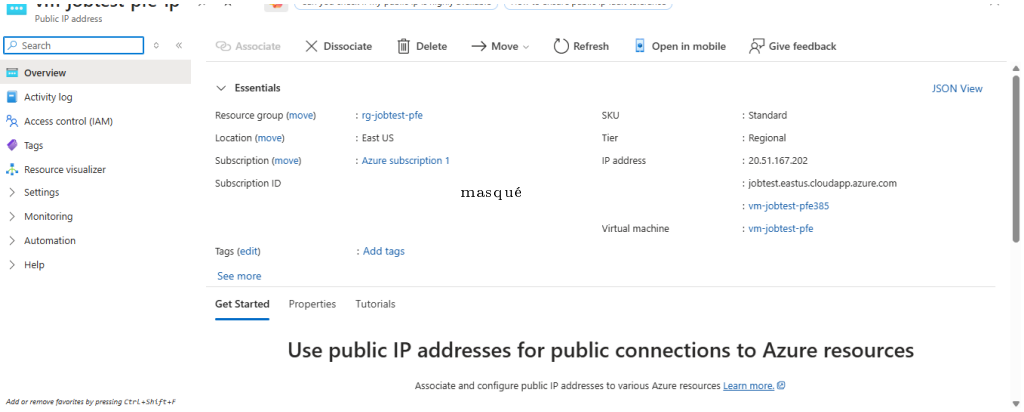


Figure 6.7: Ressource d'adresse IP publique, recadrée

Le format D2as_v7 a été retenu comme compromis pour une démonstration de PFE et pour l'exécution locale de `qwen2.5:1.5b`. Il n'est pas dimensionné pour une charge réelle soutenue : la latence de la couche 2 sur CPU illustre cette contrainte. La création précise de la règle NSG et son niveau d'attachement n'ont pas été archivés ; seul son effet, l'accès HTTP externe, est confirmé.

6.15 Architecture de déploiement

La figure 6.1 matérialise les trois frontières utiles à l'exploitation : Azure filtre l'entrée publique, la VM héberge le proxy et le réseau Docker porte les communications de service à service. Les volumes `postgres_data`, `ollama_data` et `embedding_cache` séparent les données persistantes des conteneurs recréables. Le navigateur ne communique ni avec PostgreSQL, ni avec Ollama, ni directement avec FastAPI ou Next.js : il atteint Nginx, qui distribue ensuite les routes internes.

6.16 Exploitation et maîtrise des coûts

Le contexte de démonstration mentionne un crédit Azure Free Trial de 200 USD, sans calcul mensuel figé : le coût affiché dépend de la région, du temps d'allocation et des ressources associées. L'arrêt automatique est activé mais ne doit pas être assimilé sans vérification à une désallocation. Microsoft indique que la fonctionnalité d'auto-arrêt est configurable selon un horaire et un fuseau horaire [25]. Après une démonstration, il faut contrôler l'état réel de la VM et privilégier *Stop (deallocate)* pour libérer le calcul. Le disque et, selon sa configuration, l'adresse IP peuvent continuer à générer des coûts ; la persistance de ces ressources explique qu'une désallocation ne soit pas une suppression.

6.17 Procédures d'exploitation

Les commandes suivantes correspondent aux opérations prévues par la procédure de déploiement. Elles ne révèlent pas le contenu de l'environnement ni les secrets.

```
cd /opt/jobtest
docker compose ps
docker compose logs --tail=200 nginx api web ollama db
docker compose restart
docker compose up -d --build
docker compose down
docker compose exec -T db sh -c 'pg_dump -U "$POSTGRES_USER" "$POSTGRES_DB"' | gzip > jobtest-b
```

La dernière commande est la forme documentée de sauvegarde PostgreSQL. La commande `docker compose down` arrête les conteneurs tout en conservant les volumes nommés ; l'option `-v` n'est donc pas utilisée, car elle supprimerait les données persistantes et les caches.

6.18 Bilan du déploiement

Le tableau 6.5 met en regard chaque résultat observé et la limite qui l'accompagne.

Tableau 6.5: Synthèse des résultats observés et des limites

Élément	**Résultat observé**	**Limite**
Interface Next.js	Accessible derrière Nginx.	HTTP uniquement.
API FastAPI	Santé et routes publiques de démonstration disponibles.	Ressources CPU/RAM limitées.
PostgreSQL	Persistance d'un résultat et d'une analyse validée.	Sauvegarde à automatiser.
Couche 1	Évaluation et explication fonctionnelles.	Validation réelle avec recruteurs encore nécessaire.
Couche 2	Analyse `ok` disponible.	Latence CPU élevée ; contenu consultatif.
PDF	Export valide sur un exemple de deux pages.	Test ponctuel, taille variable.
Réseau	Ports applicatifs internes non publiés.	SSH à restreindre ; UFW inactif observé.
Disponibilité	Démonstration réussie avec cinq services sains.	VM unique, sans haute disponibilité.

Conclusion

Les preuves réunies confirment le fonctionnement d'une chaîne complète de démonstration : l'évaluation, la persistance, l'analyse qualitative et l'export PDF sont servis par une pile conteneurisée, avec Nginx comme point d'entrée et des services métier non publiés directement. Le déploiement Azure est donc opérationnel dans son périmètre de PFE. Il ne constitue toutefois ni une certification de sécurité ni une infrastructure de production : HTTPS, le durcissement de SSH, les sauvegardes, la supervision et la tolérance aux pannes restent à mettre en œuvre. Ces résultats permettent de replacer, dans la conclusion générale, les apports du prototype et les conditions de son évolution.

Conclusion générale

Le recrutement confronte les organisations à un volume important de CV, à des offres exprimées de manière hétérogène et à la nécessité de justifier un premier tri. Dans ce contexte, rapprocher automatiquement un CV et une offre ne doit pas réduire une candidature à un classement opaque. Le besoin auquel répond JobTest est celui d'une aide structurée : mettre en évidence les éléments de compatibilité, les écarts et les raisons d'une recommandation, tout en laissant au recruteur la décision finale.

Le projet avait pour objectif de concevoir et réaliser une plateforme Web d'aide au recrutement capable de gérer les offres, les candidats et leurs CV, de produire une évaluation hybride, d'en restituer une explication et d'en préserver la traçabilité. La solution livrée propose un frontend Next.js, une API FastAPI, une persistance SQLAlchemy utilisant SQLite en développement et PostgreSQL dans l'environnement conteneurisé, ainsi qu'un export PDF. Le recruteur peut créer ses offres, gérer ses candidats, lancer une évaluation, consulter son historique et obtenir un rapport. Les contrôles d'appartenance des données maintiennent la séparation entre les espaces des recruteurs.

La démarche a suivi les étapes de l'étude du contexte, de l'analyse des besoins, de la conception UML et architecturale, du développement, de la conception du moteur de rapprochement, puis de la validation et du déploiement. Elle s'inscrit dans le cycle en cascade présenté au début du rapport, tout en intégrant les retours techniques nécessaires au fil des vérifications. L'industrialisation a complété le développement par Docker Compose, Nginx comme point d'entrée et un déploiement de démonstration sur une VM Microsoft Azure.

La contribution centrale est un moteur hybride qui combine les signaux TF–IDF, sémantique et compétences, puis applique des règles de décision explicites. Le score est reproductible et s'accompagne de sous-scores, de forces, de lacunes et de recommandations structurées. Une seconde couche locale, exécutée avec Ollama, produit une analyse qualitative séparée : elle enrichit la lecture du recruteur, mais ne peut ni modifier le score officiel ni déterminer seule la recommandation algorithmique `HIRE`, `CONSIDER` ou `REJECT`. Cette séparation est un résultat important du projet, car elle maintient une distinction claire entre calcul déterministe et contenu génératif.

Les validations réunies montrent qu'un parcours complet peut être exécuté sur des données synthétiques : authentification, création d'offre, évaluation, persistance, analyse approfondie et génération PDF. Le déploiement a également confirmé les cinq services conteneurisés, le proxy Nginx

et l'isolation des ports métier du réseau public. Le benchmark présenté au chapitre 4 fournit des résultats encourageants sur un corpus synthétique ; ses métriques, dont le F1 et l'AUC, ne doivent cependant pas être interprétées comme une validation sur des décisions de recrutement réelles. La valeur démontrée est celle d'un prototype fonctionnel et traçable, non celle d'un système déjà validé en situation opérationnelle.

Cette prudence est essentielle. Le jeu d'évaluation est synthétique, les poids et seuils restent empiriques, le dictionnaire de compétences est nécessairement incomplet et aucune évaluation humaine approfondie avec des recruteurs n'a été menée. Les biais possibles dans les données, les libellés ou les règles doivent être étudiés avant tout usage réel. La couche 2 présente une latence notable sur CPU et son texte, bien que contrôlé par des statuts, reste consultatif. Du point de vue de l'infrastructure, la démonstration est hébergée sur une VM unique, sans haute disponibilité, sous HTTP sans TLS, sans sauvegarde automatisée ni supervision avancée. Une politique explicite de protection, de conservation, d'anonymisation et de suppression des CV demeure également indispensable.

Les évolutions prioritaires à court terme sont l'association d'un domaine et d'un certificat TLS, la sécurisation des sessions par cookies adaptés, la restriction de SSH, les sauvegardes PostgreSQL, le monitoring et la correction des limites de téléversement et des tests hérités. L'amélioration de l'interface et l'optimisation de la couche 2 peuvent progresser en parallèle. À moyen terme, il sera nécessaire de confronter le système à un corpus réel anonymisé et autorisé, de travailler avec des recruteurs, d'enrichir le dictionnaire de compétences, de calibrer les seuils, d'évaluer les biais et de formaliser l'évaluation des explications génératives, y compris dans plusieurs langues.

À plus long terme, une charge réelle justifierait des traitements asynchrones, une file de tâches, une meilleure observabilité et, si nécessaire, la séparation de certains services sur une infrastructure évolutive. Des intégrations avec des systèmes de gestion de recrutement pourraient être envisagées, accompagnées de mécanismes de gouvernance et d'audit des décisions assistées. Ces perspectives ne sont pas des fonctions déjà réalisées ; elles dessinent les conditions d'une évolution responsable.

JobTest constitue ainsi un prototype fonctionnel et une base technique cohérente pour l'aide à l'analyse des candidatures. Il organise des signaux et des explications utiles au recruteur, sans prétendre remplacer son jugement, l'appréciation du contexte ou sa responsabilité dans la décision finale.

Bibliographie

[1] CyberCloud Solutions. "Cybercloud solutions : Page entreprise," LinkedIn, Accessed: Sep. 13, 2026. [Online]. Available: https://www.linkedin.com/company/cybercloudsolutions/

[2] J. Berg. "The messy business of managing people at work: Is AI the solution?" International Labour Organization, Accessed: Sep. 13, 2026. [Online]. Available: https://www.ilo.org/resource/article/messy-business-managing-people-work-ai-solution

[3] National Institute of Standards and Technology. "Artificial intelligence — glossary," NIST, Accessed: Sep. 13, 2026. [Online]. Available: https://csrc.nist.gov/glossary/term/artificial_intelligence

[4] A. Koechling and M. C. Wehner, "Discriminated by an algorithm: A systematic review of discrimination and fairness by algorithmic decision-making in the context of HR recruitment and HR development," *Business Research*, vol. 13, pp. 795–848, 2020. DOI: 10.1007/s40685-020-00134-w [Online]. Available: https://doi.org/10.1007/s40685-020-00134-w

[5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2019, pp. 4171–4186. DOI: 10.18653/v1/N19-1423 [Online]. Available: https://aclanthology.org/N19-1423/

[6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 2019, pp. 3982–3992. DOI: 10.18653/v1/D19-1410 [Online]. Available: https://aclanthology.org/D19-1410/

[7] LinkedIn. "Search with AI-assisted search in recruiter," LinkedIn Recruiter Help, Accessed: Sep. 13, 2026. [Online]. Available: https://www.linkedin.com/help/recruiter/answer/a6509735

[8] LinkedIn. "AI features in LinkedIn Recruiter," LinkedIn Recruiter Help, Accessed: Sep. 13, 2026. [Online]. Available: https://www.linkedin.com/help/recruiter/answer/a7784112

[9] Workable. "Candidate profile in pipeline view overview," Workable Help, Accessed: Sep. 13, 2026. [Online]. Available: https://help.workable.com/hc/en-us/articles/115012857047-Candidate-profile-in-pipeline-view-overview

[10] Workable. "Workable security and data privacy," Workable Help, Accessed: Sep. 13, 2026. [Online]. Available: https://help.workable.com/hc/en-us/articles/115011942227-Workable-security-and-data-privacy

[11] Greenhouse. "Greenhouse AI features," Greenhouse Support, Accessed: Sep. 13, 2026. [Online]. Available: https://support.greenhouse.io/hc/en-us/articles/33043749845403-Greenhouse-AI-features

[12] Greenhouse. "Scorecard overview," Greenhouse Support, Accessed: Sep. 13, 2026. [Online]. Available: https://support.greenhouse.io/hc/en-us/articles/4414777492891-Scorecard-overview

[13] Greenhouse. "Essential candidate report," Greenhouse Support, Accessed: Sep. 13, 2026. [Online]. Available: https://support.greenhouse.io/hc/en-us/articles/360042740232-Essential-candidate-report

[14] W. W. Royce, "Managing the development of large software systems," in *Proceedings of IEEE WESCON*, 1970, pp. 1–9. Accessed: Sep. 13, 2026. [Online]. Available: https://www.praxisframework.org/files/royce1970.pdf

[15] ISO/IEC. "Iso/iec 25010:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — product quality model," International Organization for Standardization, Accessed: Sep. 13, 2026. [Online]. Available: https://www.iso.org/standard/78176.html

[16] Object Management Group. "Unified modeling language, version 2.5.1," OMG, Accessed: Sep. 13, 2026. [Online]. Available: https://www.omg.org/spec/UML/2.5.1/PDF

[17] scikit-learn developers. "TfidfVectorizer," scikit-learn, Accessed: Sep. 13, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

[18] scikit-learn developers. "cosine_similarity," scikit-learn, Accessed: Sep. 13, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html

[19] scikit-learn developers. "Metrics and scoring: Quantifying the quality of predictions," scikit-learn, Accessed: Sep. 13, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/model_evaluation.html

[20] Microsoft. "Azure network security groups overview," Microsoft Learn, Accessed: Sep. 13, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/virtual-network/network-security-groups-overview

[21] Docker Inc. "Networking in compose," Docker Docs, Accessed: Sep. 13, 2026. [Online]. Available: https://docs.docker.com/compose/how-tos/networking/

[22] OWASP Foundation. "HTML5 security cheat sheet," OWASP Cheat Sheet Series, Accessed: Sep. 13, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html

[23] République Tunisienne, *Loi organique 2004-63 du 27 juillet 2004, portant sur la protection des données à caractère personnel*, Journal Officiel de la République Tunisienne, Texte instituant le régime tunisien de protection des données à caractère personnel et l'autorité de contrôle correspondante, Jul. 2004.

[24] Instance Nationale de Protection des Données à Caractère Personnel. "Instance nationale de protection des données à caractère personnel (inpdp)," INPDP, Accessed: Sep. 17, 2026. [Online]. Available: https://www.inpdp.nat.tn/

[25] Microsoft. "Auto-shutdown a virtual machine," Microsoft Learn, Accessed: Sep. 13, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/virtual-machines/auto-shutdown-vm

Résumé

Ce mémoire présente JobTest, une plateforme Web d'aide à la présélection qui rapproche des CV et des offres d'emploi. La solution combine des signaux TF–IDF, sémantiques et de compétences, applique une décision explicite et restitue ses composantes au recruteur. Elle intègre un frontend Next.js, une API FastAPI, une persistance relationnelle, un export PDF et une analyse générative locale optionnelle, séparée du verdict officiel. Les validations couvrent les parcours principaux, un benchmark sur données synthétiques et un déploiement de démonstration conteneurisé sur Azure. JobTest assiste le recruteur sans remplacer son jugement ni une décision humaine.

Mots-clés : recrutement, appariement CV-offre, IA explicable, FastAPI, Next.js.

Abstract

This thesis presents JobTest, a web platform that assists recruiters in matching resumes with job offers. The solution combines TF–IDF, semantic and skill-based signals, applies an explicit decision policy, and exposes the components of the result to the recruiter. It includes a Next.js frontend, a FastAPI API, relational persistence, PDF export, and an optional local generative analysis that is kept separate from the official verdict. The validation covers the main user flows, a synthetic-data benchmark, and a containerized demonstration deployment on Azure. JobTest supports recruiters without replacing their judgement or human decision-making.

Keywords: recruitment, CV-job matching, explainable AI, FastAPI, Next.js.

Accéder à la démonstration publique de JobTest

`http://20.51.167.202/`